XVI-O1-CONE: Python scripts (v1–v6)

Ngo Cheung

18 October 2026

All Python scripts in `/workspace/XVI-O1-CONE/code/` used for XVI-O1-CONE v1–v6, grouped by purpose and reproduced verbatim. Each listing carries its SHA-256. The scripts import from one another and read frozen XVI-P/XVI-Q engine files and run logs that are not included here.

Contents

1 Apex BRST jets and cohomology (v1–v3) 1
1.1 apex_jets.py . . . 1
1.2 apex_jets_v1_frozen.py . . . 4
1.3 apex_run.py . . . 8
1.4 tri_count.py . . . 11
1.5 tri_count_all.py . . . 11
1.6 tri_run.py . . . 12
1.7 tri_run_all.py . . . 13
1.8 resonances.py . . . 15
1.9 inner_degrees.py . . . 15
1.10 inner_degrees_q24.py . . . 16
1.11 locality.py . . . 17

2 Flat-space controls 18
2.1 flat_count.py . . . 18
2.2 flat_pole.py . . . 18
2.3 flat_vs_p18.py . . . 20

3 Kernels, coincidence limits and heat coefficients 21
3.1 kernel.py . . . 21
3.2 tensor.py . . . 22
3.3 coinc.py . . . 22
3.4 xcheck_coinc.py . . . 23
3.5 heatcoef.py . . . 24
3.6 weighted_apex.py . . . 24
3.7 weighted_checks.py . . . 25

4 Tadpoles D02, D12, D05a and the hh structures 25
4.1 cone_tadpole.py . . . 25
4.2 tadpole2.py . . . 27
4.3 tadpole2_fast.py . . . 28
4.4 hh_structures.py . . . 29

5 Spectral apex reduction of the bubbles (v4) 30
5.1 spectral_apex.py . . . 30
5.2 spec_compare.py . . . 32
5.3 catalogue_check.py . . . 32

6 Slavnov–Taylor and Ward diagnostics **33**
6.1 st_test_const.py . . . 33
6.2 st_scan.py . . . 34
6.3 st_norm.py . . . 35
6.4 st_v6.py . . . 35
6.5 st_v6_stats.py . . . 36
6.6 ward_scan.py . . . 37
6.7 ward_xcheck.py . . . 37

7 $H_I obstruction for D01(v6)$ **38**
7.1 hi_d01.py . . . 38
7.2 hi_d01_exact.py . . . 38

8 Build and artifact scripts for v6 **39**
8.1 build_v6.py . . . 39
8.2 artifacts_v6.py . . . 42

9 Debug scratch **43**
9.1 dbg.py . . . 43
9.2 dbg2.py . . . 44

1 Apex BRST jets and cohomology (v1–v3)

1.1 apex_jets.py

SHA-256: 63d179822a7042df0382c26e5cedd4b2ad929bc7f69440ad522f9098cd83d437

```
"""Apex (worldsheet W) counterterm inventory and BRST cohomology on the cone C_alpha x R^2 (Euclidean), stage 1.

Fields near W are apex-polyhomogeneous spectral packets of the declared realization (I-B-G-S-v2 smooth packets):
component of helicity weight s, angular channel n:  u = sum_j c_j(t,z) r^(eps*nu+2j) e^{i a theta},  a = n/alpha + s,
nu=|a|, eps=-1 only in the curvature-regular E0 channels (s,n)=(+2,-1),(-2,+1) for alpha>1/2 (graviton and its
antifield K), eps=+1 otherwise (Friedrichs).  Jets J=(comp,n,m) carry dimension [F]+m.
An apex functional is  int_W  J1 . M(d_par) . J2  with n1+n2=0 (cone rotation), O(2)_par invariance (delta pairings,
no epsilon: reflection), phi-reflection invariance, Bose symmetry; power counting kappa^2 int_W: [F1]+m1+[F2]+m2+k=4.
Linear BRST (free gauge-fixed EH + b-field, frozen-gauge structure): s h=K2 c, s cb=b, s K=E h + C2^T b,
s c* = K2^T K - (C2 K2)^T cb ; s c = s b = s phi = 0.
Exact arithmetic mod two primes p=1 mod 4 (i = sqrt(-1) mod p); alpha a rational with large denominator (generic)."""
import itertools, random, json, sys
from fractions import Fraction as Fr

PRIMES = [1000000009, 998244353, 1000000021]
def sqrtm1(p):
    for g in range(2, 200):
        x = pow(g, (p - 1) // 4, p)
        if x * x % p == p - 1: return x
P = PRIMES[0]; II = sqrtm1(P)
def setprime(p):
    global P, II
    P = p; II = sqrtm1(p)
def fr(x):   # Fraction -> mod P
    x = Fr(x); return x.numerator % P * pow(x.denominator % P, P - 2, P) % P

# ---------------- components: name -> (weight, longitudinal rank)
VEC = {'L': (0, 1), 'P': (1, 0), 'M': (-1, 0)}
TEN = {'LL': (0, 2), 'PM': (0, 0), 'LP': (1, 1), 'LM': (-1, 1), 'PP': (2, 0), 'MM': (-2, 0)}
SCA = {'S': (0, 0)}
REFL = {'L': 'L', 'P': 'M', 'M': 'P', 'LL': 'LL', 'PM': 'PM', 'LP': 'LM', 'LM': 'LP', 'PP': 'MM', 'MM': 'PP', 'S': 'S'}
FIELDS = {  # name: (component table, dimension, kind)
    'h': (TEN, 1, 'graviton'), 'K': (TEN, 2, 'graviton'), 'c': (VEC, 1, 'vector'), 'cb': (VEC, 1, 'vector'),
    'b': (VEC, 2, 'vector'), 'cs': (VEC, 2, 'vector'), 'phi': (SCA, 1, 'scalar')}

class Setup:
    def __init__(self, alpha, branch, nmax=4, mcap=Fr(7, 2)):
        self.al = Fr(alpha); self.branch = branch; self.nmax = nmax; self.mcap = mcap
    def a(self, s, n): return Fr(n) / self.al + s
    def eps(self, kind, s, n):
        if kind == 'graviton' and self.branch == 'CR' and self.al > Fr(1, 2) and (s, n) in ((2, -1), (-2, 1)):
            return -1
        return 1
    def lattice_ok(self, kind, s, n, m):
        nu = abs(self.a(s, n)); e = self.eps(kind, s, n)
```

```
        d = (Fr(m) - e * nu) / 2
        return d.denominator == 1 and d >= 0
    def jets(self, F, dmax):
        comps, dim, kind = FIELDS[F]; out = []
        for C, (s, rk) in comps.items():
            for n in range(-self.nmax, self.nmax + 1):
                nu = abs(self.a(s, n)); e = self.eps(kind, s, n)
                j = 0
                while True:
                    m = e * nu + 2 * j
                    if dim + m > dmax: break
                    out.append((C, n, m)); j += 1
        return out

# ---------------- configurations: dict comp -> dict (n,m) -> coefficient tensor (flat tuple over 2^rank indices)
def zeros(rk): return [0] * (2 ** rk)
def rand_tensor(rk, sym):
    v = [random.randrange(P) for _ in range(2 ** rk)]
    if rk == 2 and sym: v[2] = v[1]
    return v
def add_into(cfg, C, key, vec, fac=1):
    d = cfg.setdefault(C, {}); cur = d.setdefault(key, zeros({1: 0, 2: 1, 4: 2}[len(vec)]))
    for i in range(len(vec)): cur[i] = (cur[i] + fac * vec[i]) % P
def scal(vec, f): return [x * f % P for x in vec]

class Ops:
    """flat parallel-frame operators acting on atom configurations; q = longitudinal momentum (mod P ints)"""
    def __init__(self, S, q): self.S = S; self.q = q
    def Dw(self, d, s):      # d: {(n,m):vec} for a component of weight s -> weight s-1
        out = {}
        for (n, m), v in d.items():
            c = fr((Fr(m) + self.S.a(s, n)) / 2)
            if c: out[(n, m - 1)] = scal(v, c)
        return out
    def Dwb(self, d, s):
        out = {}
        for (n, m), v in d.items():
            c = fr((Fr(m) - self.S.a(s, n)) / 2)
            if c: out[(n, m - 1)] = scal(v, c)
        return out
    # longitudinal derivative i q_a: rank r -> r+1 (new index FIRST)
    def Dl(self, d):
        out = {}
        for k, v in d.items():
            r = {1: 0, 2: 1, 4: 2}[len(v)]
            w = []
            for a in range(2):
                w += [II * self.q[a] % P * x % P for x in v]
            out[k] = w
        return out
    def contract_first(self, d):   # sum_a  T[a, a, ...] for rank>=2 (first two indices)
        out = {}
        for k, v in d.items():
            n = len(v) // 4
            out[k] = [(v[0 * 2 * n + 0 * n + i] + v[1 * 2 * n + 1 * n + i]) % P for i in range(n)]
        return out
    @staticmethod
    def addd(*ds, facs=None):
        out = {}
        for i, d in enumerate(ds):
            f = 1 if facs is None else facs[i]
            for k, v in d.items():
                cur = out.setdefault(k, [0] * len(v))
                for j in range(len(v)): cur[j] = (cur[j] + f * v[j]) % P
        return out
    @staticmethod
    def sc(d, f): return {k: scal(v, f % P) for k, v in d.items()}
    @staticmethod
    def swap01(d):   # transpose first two indices of rank-2
        return {k: [v[0], v[2], v[1], v[3]] for k, v in d.items()}
    # --- scalar/vector/tensor operators
    def grad(self, f):   # scalar comp dict -> vector
        return {'L': self.Dl(f.get('S', {})), 'P': self.sc(self.Dwb(f.get('S', {}), 0), 2), 'M': self.sc(self.Dw(f.get('S', {}), 0),
     2)}
    def divv(self, V):
        return {'S': self.addd(self.contract_first_vec(self.Dl(V.get('L', {}))), self.Dwb(V.get('M', {}), -1), self.Dw(V.get('P', {}),
       1))}
    def contract_first_vec(self, d):  # rank2 from Dl of rank1: sum_a T[a,a]
        return {k: [(v[0] + v[3]) % P] for k, v in d.items()}
    def K2(self, c):
        cL = c.get('L', {}); cP = c.get('P', {}); cM = c.get('M', {})
        dL = self.Dl(cL)                        # [a][b] = i q_a c_b
        LL = self.addd(dL, self.swap01(dL))
        LP = self.addd(self.Dl(cP), self.sc(self.Dwb(cL, 0), 2))
```

```
        LM = self.addd(self.Dl(cM), self.sc(self.Dw(cL, 0), 2))
        PP = self.sc(self.Dwb(cP, 1), 4); MM = self.sc(self.Dw(cM, -1), 4)
        PM = self.addd(self.sc(self.Dwb(cM, -1), 2), self.sc(self.Dw(cP, 1), 2))
        return {'LL': LL, 'LP': LP, 'LM': LM, 'PP': PP, 'MM': MM, 'PM': PM}
    def divt(self, X):   # (div X)_nu = d_a X_{a nu} + dwb X_{- nu} + dw X_{+ nu}
        LL = X.get('LL', {}); LP = X.get('LP', {}); LM = X.get('LM', {}); PP = X.get('PP', {}); MM = X.get('MM', {}); PM = X.get('PM', {})
        L = self.addd(self.contract_first(self.Dl(LL)), self.Dwb(LM, -1), self.Dw(LP, 1))
        Pp = self.addd(self.contract_first_vec(self.Dl(LP)), self.Dwb(PM, 0), self.Dw(PP, 2))
        Mm = self.addd(self.contract_first_vec(self.Dl(LM)), self.Dwb(MM, -2), self.Dw(PM, 0))
        return {'L': L, 'P': Pp, 'M': Mm}
    def tr(self, X):
        LL = X.get('LL', {})
        t = {k: [(v[0] + v[3]) % P] for k, v in LL.items()}
        return {'S': self.addd(t, X.get('PM', {}))}
    def gtimes(self, f):  # metric times scalar: LL=delta f, PM = 2 f
        fs = f.get('S', {})
        return {'LL': {k: [v[0], 0, 0, v[0]] for k, v in fs.items()}, 'PM': self.sc(fs, 2)}
    def tadd(self, *Xs, facs=None):
        out = {}
        for i, X in enumerate(Xs):
            f = 1 if facs is None else facs[i]
            for C, d in X.items():
                out[C] = self.addd(out.get(C, {}), d, facs=[1, f])
        return out
    def box(self, X, wt):   # wt: comp->weight
        q2 = (self.q[0] * self.q[0] + self.q[1] * self.q[1]) % P
        out = {}
        for C, d in X.items():
            s = wt[C]
            out[C] = self.addd(self.sc(d, -q2), self.sc(self.Dw(self.Dwb(d, s), s + 1), 4))
        return out
    def Ptr(self, X):
        return self.tadd(X, self.gtimes(self.tr(X)), facs=[1, P - pow(2, P - 2, P)])
    def E(self, h):
        wt = {C: TEN[C][0] for C in TEN}
        dv = self.divt(h); th = self.tr(h)
        dd = self.divv(dv); bt = self.box(th, {'S': 0})
        return self.tadd(self.box(h, wt), self.K2(dv), self.K2(self.grad(th)), self.gtimes(dd), self.gtimes(bt),
                         facs=[1, P - 1, pow(2, P - 2, P), 1, P - 1])
    def C2(self, h): return self.divt(self.Ptr(h))
    def C2T(self, b): return self.tadd(self.Ptr(self.K2(b)), facs=[P - pow(2, P - 2, P)])
    def K2T(self, K): return {C: self.sc(d, -2) for C, d in self.divt(K).items()}
    def ghost(self, c): return self.C2(self.K2(c))
    def Ric(self, h):
        dv = self.divt(h); th = self.tr(h); wt = {C: TEN[C][0] for C in TEN}
        return self.tadd(self.K2(dv), self.box(h, wt), self.K2(self.grad(th)), facs=[pow(2, P - 2, P), P - pow(2, P - 2, P), P - pow(4, P - 2, P)])

def rand_cfg(S, F, dmax):
    comps, dim, kind = FIELDS[F]; cfg = {}
    for (C, n, m) in S.jets(F, dmax):
        rk = comps[C][1]
        cfg.setdefault(C, {})[(n, m)] = rand_tensor(rk, C == 'LL')
    return cfg

def foreign(S, F, cfg):
    """atoms of cfg (a field of type F) outside F's realization lattice"""
    comps, dim, kind = FIELDS[F]; bad = []
    for C, d in cfg.items():
        s = comps[C][0]
        for (n, m), v in d.items():
            if any(v) and not S.lattice_ok(kind, s, n, m): bad.append((C, n, str(m)))
    return bad

def reflect(cfg):
    return {REFL[C]: {(-n, m): list(v) for (n, m), v in d.items()} for C, d in cfg.items()}

# ---------------- candidate functionals
def pairings(slots):
    if not slots: yield []; return
    a = slots[0]
    for i in range(1, len(slots)):
        rest = slots[1:i] + slots[i + 1:]
        for pr in pairings(rest): yield [(a, slots[i])] + pr

def candidates(S, F1, F2, T):
    c1, d1, _ = FIELDS[F1]; c2, d2, _ = FIELDS[F2]; out = []
    J1 = S.jets(F1, T); J2 = S.jets(F2, T)   # v2: cap T (a CR-flipped partner can have m in (-1,0)); k>=0 enforces the budget
    for (C1, n1, m1) in J1:
        for (C2, n2, m2) in J2:
            if n1 + n2 != 0: continue
            k = Fr(T) - d1 - m1 - d2 - m2
            if k.denominator != 1 or k < 0: continue
```

```
            k = int(k); r1 = c1[C1][1]; r2 = c2[C2][1]
            slots = [('A', i) for i in range(r1)] + [('B', i) for i in range(r2)] + [('p', i) for i in range(k)]
            if len(slots) % 2: continue
            seen = set()
            for pr in pairings(slots):
                key = tuple(sorted(tuple(sorted((x[0], x[1] if x[0] != 'p' else 0) for x in pair)) for pair in pr))
                if key in seen: continue
                seen.add(key); out.append(((C1, n1, m1), (C2, n2, m2), r1, r2, k, pr))
    return out

def eval_cand(cand, cfg1, cfg2, q):
    (C1, n1, m1), (C2, n2, m2), r1, r2, k, pr = cand
    v1 = cfg1.get(C1, {}).get((n1, m1)); v2 = cfg2.get(C2, {}).get((n2, m2))
    if v1 is None or v2 is None: return 0
    tot = 0
    for idx in itertools.product(range(2), repeat=r1 + r2 + k):
        A = idx[:r1]; B = idx[r1:r1 + r2]; Pp = idx[r1 + r2:]
        lab = {('A', i): A[i] for i in range(r1)}; lab.update({('B', i): B[i] for i in range(r2)}); lab.update({('p', i): Pp[i] for i
     in range(k)})
        if any(lab[x] != lab[y] for x, y in pr): continue
        ia = 0
        for i in A: ia = ia * 2 + i
        ib = 0
        for i in B: ib = ib * 2 + i
        t = v1[ia] * v2[ib] % P
        for i in Pp: t = t * q[i] % P
        tot = (tot + t) % P
    return tot

def negq(q): return [(-x) % P for x in q]

def eval_sym(cand, cfg1, cfg2, q, bose):
    v = (eval_cand(cand, cfg1, cfg2, q) + eval_cand(cand, reflect(cfg1), reflect(cfg2), q)) % P
    if bose:
        v = (v + eval_cand(cand, cfg2, cfg1, negq(q)) + eval_cand(cand, reflect(cfg2), reflect(cfg1), negq(q))) % P
    return v

# ---------------- modular linear algebra
def rank_and_basis(rows):
    """rows: list of vectors (mod P). returns rank and pivot row indices (independent subset)"""
    M = [list(r) for r in rows]; piv = []; R = []
    for i, r in enumerate(M):
        v = list(r)
        for (pc, prow) in R:
            if v[pc]:
                f = v[pc]
                v = [(x - f * y) % P for x, y in zip(v, prow)]
        nz = next((j for j, x in enumerate(v) if x), None)
        if nz is not None:
            inv = pow(v[nz], P - 2, P); v = [x * inv % P for x in v]
            R.append((nz, v)); piv.append(i)
    return len(R), piv

def nullspace(A, ncols):
    """A: list of rows (mod P). returns basis of {x: A x = 0}"""
    M = [list(r) for r in A]; pivcols = []; r = 0
    for c in range(ncols):
        pr = next((i for i in range(r, len(M)) if M[i][c]), None)
        if pr is None: continue
        M[r], M[pr] = M[pr], M[r]
        inv = pow(M[r][c], P - 2, P); M[r] = [x * inv % P for x in M[r]]
        for i in range(len(M)):
            if i != r and M[i][c]:
                f = M[i][c]; M[i] = [(x - f * y) % P for x, y in zip(M[i], M[r])]
        pivcols.append(c); r += 1
        if r == len(M): break
    free = [c for c in range(ncols) if c not in pivcols]; basis = []
    for fcol in free:
        x = [0] * ncols; x[fcol] = 1
        for i, pc in enumerate(pivcols): x[pc] = (-M[i][fcol]) % P
        basis.append(x)
    return basis
```

1.2 apex_jets_v1_frozen.py

SHA-256: d2dd83d18d709e89c614ec397c52fc86cc3019d111c9336796995d03b122c2d9

```
"""Apex (worldsheet W) counterterm inventory and BRST cohomology on the cone C_alpha x R^2 (Euclidean), stage 1.

Fields near W are apex-polyhomogeneous spectral packets of the declared realization (I-B-G-S-v2 smooth packets):
component of helicity weight s, angular channel n:  u = sum_j c_j(t,z) r^(eps*nu+2j) e^{i a theta},  a = n/alpha + s,
nu=|a|, eps=-1 only in the curvature-regular E0 channels (s,n)=(+2,-1),(-2,+1) for alpha>1/2 (graviton and its
```

```
antifield K), eps=+1 otherwise (Friedrichs).  Jets J=(comp,n,m) carry dimension [F]+m.
An apex functional is  int_W  J1 . M(d_par) . J2  with n1+n2=0 (cone rotation), O(2)_par invariance (delta pairings,
no epsilon: reflection), phi-reflection invariance, Bose symmetry; power counting kappa^2 int_W: [F1]+m1+[F2]+m2+k=4.
Linear BRST (free gauge-fixed EH + b-field, frozen-gauge structure): s h=K2 c, s cb=b, s K=E h + C2^T b,
s c* = K2^T K - (C2 K2)^T cb ; s c = s b = s phi = 0.
Exact arithmetic mod two primes p=1 mod 4 (i = sqrt(-1) mod p); alpha a rational with large denominator (generic)."""
import itertools, random, json, sys
from fractions import Fraction as Fr

PRIMES = [1000000009, 998244353]
def sqrtm1(p):
    for g in range(2, 200):
        x = pow(g, (p - 1) // 4, p)
        if x * x % p == p - 1: return x
P = PRIMES[0]; II = sqrtm1(P)
def setprime(p):
    global P, II
    P = p; II = sqrtm1(p)
def fr(x):   # Fraction -> mod P
    x = Fr(x); return x.numerator % P * pow(x.denominator % P, P - 2, P) % P

# ---------------- components: name -> (weight, longitudinal rank)
VEC = {'L': (0, 1), 'P': (1, 0), 'M': (-1, 0)}
TEN = {'LL': (0, 2), 'PM': (0, 0), 'LP': (1, 1), 'LM': (-1, 1), 'PP': (2, 0), 'MM': (-2, 0)}
SCA = {'S': (0, 0)}
REFL = {'L': 'L', 'P': 'M', 'M': 'P', 'LL': 'LL', 'PM': 'PM', 'LP': 'LM', 'LM': 'LP', 'PP': 'MM', 'MM': 'PP', 'S': 'S'}
FIELDS = {  # name: (component table, dimension, kind)
    'h': (TEN, 1, 'graviton'), 'K': (TEN, 2, 'graviton'), 'c': (VEC, 1, 'vector'), 'cb': (VEC, 1, 'vector'),
    'b': (VEC, 2, 'vector'), 'cs': (VEC, 2, 'vector'), 'phi': (SCA, 1, 'scalar')}

class Setup:
    def __init__(self, alpha, branch, nmax=4, mcap=Fr(7, 2)):
        self.al = Fr(alpha); self.branch = branch; self.nmax = nmax; self.mcap = mcap
    def a(self, s, n): return Fr(n) / self.al + s
    def eps(self, kind, s, n):
        if kind == 'graviton' and self.branch == 'CR' and self.al > Fr(1, 2) and (s, n) in ((2, -1), (-2, 1)):
            return -1
        return 1
    def lattice_ok(self, kind, s, n, m):
        nu = abs(self.a(s, n)); e = self.eps(kind, s, n)
        d = (Fr(m) - e * nu) / 2
        return d.denominator == 1 and d >= 0
    def jets(self, F, dmax):
        comps, dim, kind = FIELDS[F]; out = []
        for C, (s, rk) in comps.items():
            for n in range(-self.nmax, self.nmax + 1):
                nu = abs(self.a(s, n)); e = self.eps(kind, s, n)
                j = 0
                while True:
                    m = e * nu + 2 * j
                    if dim + m > dmax: break
                    out.append((C, n, m)); j += 1
        return out

# ---------------- configurations: dict comp -> dict (n,m) -> coefficient tensor (flat tuple over 2^rank indices)
def zeros(rk): return [0] * (2 ** rk)
def rand_tensor(rk, sym):
    v = [random.randrange(P) for _ in range(2 ** rk)]
    if rk == 2 and sym: v[2] = v[1]
    return v
def add_into(cfg, C, key, vec, fac=1):
    d = cfg.setdefault(C, {}); cur = d.setdefault(key, zeros({1: 0, 2: 1, 4: 2}[len(vec)]))
    for i in range(len(vec)): cur[i] = (cur[i] + fac * vec[i]) % P
def scal(vec, f): return [x * f % P for x in vec]

class Ops:
    """flat parallel-frame operators acting on atom configurations; q = longitudinal momentum (mod P ints)"""
    def __init__(self, S, q): self.S = S; self.q = q
    def Dw(self, d, s):      # d: {(n,m):vec} for a component of weight s -> weight s-1
        out = {}
        for (n, m), v in d.items():
            c = fr((Fr(m) + self.S.a(s, n)) / 2)
            if c: out[(n, m - 1)] = scal(v, c)
        return out
    def Dwb(self, d, s):
        out = {}
        for (n, m), v in d.items():
            c = fr((Fr(m) - self.S.a(s, n)) / 2)
            if c: out[(n, m - 1)] = scal(v, c)
        return out
    # longitudinal derivative i q_a: rank r -> r+1 (new index FIRST)
    def Dl(self, d):
        out = {}
        for k, v in d.items():
```

```
            r = {1: 0, 2: 1, 4: 2}[len(v)]
            w = []
            for a in range(2):
                w += [II * self.q[a] % P * x % P for x in v]
            out[k] = w
        return out
    def contract_first(self, d):   # sum_a  T[a, a, ...] for rank>=2 (first two indices)
        out = {}
        for k, v in d.items():
            n = len(v) // 4
            out[k] = [(v[0 * 2 * n + 0 * n + i] + v[1 * 2 * n + 1 * n + i]) % P for i in range(n)]
        return out
    @staticmethod
    def addd(*ds, facs=None):
        out = {}
        for i, d in enumerate(ds):
            f = 1 if facs is None else facs[i]
            for k, v in d.items():
                cur = out.setdefault(k, [0] * len(v))
                for j in range(len(v)): cur[j] = (cur[j] + f * v[j]) % P
        return out
    @staticmethod
    def sc(d, f): return {k: scal(v, f % P) for k, v in d.items()}
    @staticmethod
    def swap01(d):   # transpose first two indices of rank-2
        return {k: [v[0], v[2], v[1], v[3]] for k, v in d.items()}
    # --- scalar/vector/tensor operators
    def grad(self, f):   # scalar comp dict -> vector
        return {'L': self.Dl(f.get('S', {})), 'P': self.sc(self.Dwb(f.get('S', {}), 0), 2), 'M': self.sc(self.Dw(f.get('S', {}), 0),
     2)}
    def divv(self, V):
        return {'S': self.addd(self.contract_first_vec(self.Dl(V.get('L', {}))), self.Dwb(V.get('M', {}), -1), self.Dw(V.get('P', {}),
       1))}
    def contract_first_vec(self, d):  # rank2 from Dl of rank1: sum_a T[a,a]
        return {k: [(v[0] + v[3]) % P] for k, v in d.items()}
    def K2(self, c):
        cL = c.get('L', {}); cP = c.get('P', {}); cM = c.get('M', {})
        dL = self.Dl(cL)                        # [a][b] = i q_a c_b
        LL = self.addd(dL, self.swap01(dL))
        LP = self.addd(self.Dl(cP), self.sc(self.Dwb(cL, 0), 2))
        LM = self.addd(self.Dl(cM), self.sc(self.Dw(cL, 0), 2))
        PP = self.sc(self.Dwb(cP, 1), 4); MM = self.sc(self.Dw(cM, -1), 4)
        PM = self.addd(self.sc(self.Dwb(cM, -1), 2), self.sc(self.Dw(cP, 1), 2))
        return {'LL': LL, 'LP': LP, 'LM': LM, 'PP': PP, 'MM': MM, 'PM': PM}
    def divt(self, X):   # (div X)_nu = d_a X_{a nu} + dwb X_{- nu} + dw X_{+ nu}
        LL = X.get('LL', {}); LP = X.get('LP', {}); LM = X.get('LM', {}); PP = X.get('PP', {}); MM = X.get('MM', {}); PM = X.get('PM',
       {})
        L = self.addd(self.contract_first(self.Dl(LL)), self.Dwb(LM, -1), self.Dw(LP, 1))
        Pp = self.addd(self.contract_first_vec(self.Dl(LP)), self.Dwb(PM, 0), self.Dw(PP, 2))
        Mm = self.addd(self.contract_first_vec(self.Dl(LM)), self.Dwb(MM, -2), self.Dw(PM, 0))
        return {'L': L, 'P': Pp, 'M': Mm}
    def tr(self, X):
        LL = X.get('LL', {})
        t = {k: [(v[0] + v[3]) % P] for k, v in LL.items()}
        return {'S': self.addd(t, X.get('PM', {}))}
    def gtimes(self, f):  # metric times scalar: LL=delta f, PM = 2 f
        fs = f.get('S', {})
        return {'LL': {k: [v[0], 0, 0, v[0]] for k, v in fs.items()}, 'PM': self.sc(fs, 2)}
    def tadd(self, *Xs, facs=None):
        out = {}
        for i, X in enumerate(Xs):
            f = 1 if facs is None else facs[i]
            for C, d in X.items():
                out[C] = self.addd(out.get(C, {}), d, facs=[1, f])
        return out
    def box(self, X, wt):   # wt: comp->weight
        q2 = (self.q[0] * self.q[0] + self.q[1] * self.q[1]) % P
        out = {}
        for C, d in X.items():
            s = wt[C]
            out[C] = self.addd(self.sc(d, -q2), self.sc(self.Dw(self.Dwb(d, s), s + 1), 4))
        return out
    def Ptr(self, X):
        return self.tadd(X, self.gtimes(self.tr(X)), facs=[1, P - pow(2, P - 2, P)])
    def E(self, h):
        wt = {C: TEN[C][0] for C in TEN}
        dv = self.divt(h); th = self.tr(h)
        dd = self.divv(dv); bt = self.box(th, {'S': 0})
        return self.tadd(self.box(h, wt), self.K2(dv), self.K2(self.grad(th)), self.gtimes(dd), self.gtimes(bt),
                         facs=[1, P - 1, pow(2, P - 2, P), 1, P - 1])
    def C2(self, h): return self.divt(self.Ptr(h))
    def C2T(self, b): return self.tadd(self.Ptr(self.K2(b)), facs=[P - pow(2, P - 2, P)])
    def K2T(self, K): return {C: self.sc(d, -2) for C, d in self.divt(K).items()}
    def ghost(self, c): return self.C2(self.K2(c))
```

```
    def Ric(self, h):
        dv = self.divt(h); th = self.tr(h); wt = {C: TEN[C][0] for C in TEN}
        return self.tadd(self.K2(dv), self.box(h, wt), self.K2(self.grad(th)), facs=[pow(2, P - 2, P), P - pow(2, P - 2, P), P - pow
     (4, P - 2, P)])

def rand_cfg(S, F, dmax):
    comps, dim, kind = FIELDS[F]; cfg = {}
    for (C, n, m) in S.jets(F, dmax):
        rk = comps[C][1]
        cfg.setdefault(C, {})[(n, m)] = rand_tensor(rk, C == 'LL')
    return cfg

def foreign(S, F, cfg):
    """atoms of cfg (a field of type F) outside F's realization lattice"""
    comps, dim, kind = FIELDS[F]; bad = []
    for C, d in cfg.items():
        s = comps[C][0]
        for (n, m), v in d.items():
            if any(v) and not S.lattice_ok(kind, s, n, m): bad.append((C, n, str(m)))
    return bad

def reflect(cfg):
    return {REFL[C]: {(-n, m): list(v) for (n, m), v in d.items()} for C, d in cfg.items()}

# ---------------- candidate functionals
def pairings(slots):
    if not slots: yield []; return
    a = slots[0]
    for i in range(1, len(slots)):
        rest = slots[1:i] + slots[i + 1:]
        for pr in pairings(rest): yield [(a, slots[i])] + pr

def candidates(S, F1, F2, T):
    c1, d1, _ = FIELDS[F1]; c2, d2, _ = FIELDS[F2]; out = []
    J1 = S.jets(F1, T - 1); J2 = S.jets(F2, T - 1)
    for (C1, n1, m1) in J1:
        for (C2, n2, m2) in J2:
            if n1 + n2 != 0: continue
            k = Fr(T) - d1 - m1 - d2 - m2
            if k.denominator != 1 or k < 0: continue
            k = int(k); r1 = c1[C1][1]; r2 = c2[C2][1]
            slots = [('A', i) for i in range(r1)] + [('B', i) for i in range(r2)] + [('p', i) for i in range(k)]
            if len(slots) % 2: continue
            seen = set()
            for pr in pairings(slots):
                key = tuple(sorted(tuple(sorted((x[0], x[1] if x[0] != 'p' else 0) for x in pair)) for pair in pr))
                if key in seen: continue
                seen.add(key); out.append(((C1, n1, m1), (C2, n2, m2), r1, r2, k, pr))
    return out

def eval_cand(cand, cfg1, cfg2, q):
    (C1, n1, m1), (C2, n2, m2), r1, r2, k, pr = cand
    v1 = cfg1.get(C1, {}).get((n1, m1)); v2 = cfg2.get(C2, {}).get((n2, m2))
    if v1 is None or v2 is None: return 0
    tot = 0
    for idx in itertools.product(range(2), repeat=r1 + r2 + k):
        A = idx[:r1]; B = idx[r1:r1 + r2]; Pp = idx[r1 + r2:]
        lab = {('A', i): A[i] for i in range(r1)}; lab.update({('B', i): B[i] for i in range(r2)}); lab.update({('p', i): Pp[i] for i
     in range(k)})
        if any(lab[x] != lab[y] for x, y in pr): continue
        ia = 0
        for i in A: ia = ia * 2 + i
        ib = 0
        for i in B: ib = ib * 2 + i
        t = v1[ia] * v2[ib] % P
        for i in Pp: t = t * q[i] % P
        tot = (tot + t) % P
    return tot

def negq(q): return [(-x) % P for x in q]

def eval_sym(cand, cfg1, cfg2, q, bose):
    v = (eval_cand(cand, cfg1, cfg2, q) + eval_cand(cand, reflect(cfg1), reflect(cfg2), q)) % P
    if bose:
        v = (v + eval_cand(cand, cfg2, cfg1, negq(q)) + eval_cand(cand, reflect(cfg2), reflect(cfg1), negq(q))) % P
    return v

# ---------------- modular linear algebra
def rank_and_basis(rows):
    """rows: list of vectors (mod P). returns rank and pivot row indices (independent subset)"""
    M = [list(r) for r in rows]; piv = []; R = []
    for i, r in enumerate(M):
        v = list(r)
        for (pc, prow) in R:
```

```
            if v[pc]:
                f = v[pc]
                v = [(x - f * y) % P for x, y in zip(v, prow)]
        nz = next((j for j, x in enumerate(v) if x), None)
        if nz is not None:
            inv = pow(v[nz], P - 2, P); v = [x * inv % P for x in v]
            R.append((nz, v)); piv.append(i)
    return len(R), piv

def nullspace(A, ncols):
    """A: list of rows (mod P). returns basis of {x: A x = 0}"""
    M = [list(r) for r in A]; pivcols = []; r = 0
    for c in range(ncols):
        pr = next((i for i in range(r, len(M)) if M[i][c]), None)
        if pr is None: continue
        M[r], M[pr] = M[pr], M[r]
        inv = pow(M[r][c], P - 2, P); M[r] = [x * inv % P for x in M[r]]
        for i in range(len(M)):
            if i != r and M[i][c]:
                f = M[i][c]; M[i] = [(x - f * y) % P for x, y in zip(M[i], M[r])]
        pivcols.append(c); r += 1
        if r == len(M): break
    free = [c for c in range(ncols) if c not in pivcols]; basis = []
    for fcol in free:
        x = [0] * ncols; x[fcol] = 1
        for i, pc in enumerate(pivcols): x[pc] = (-M[i][fcol]) % P
        basis.append(x)
    return basis
```

1.3 apex_run.py

SHA-256: d82c87c959cacdea7e2384b9a192767d11bd7716b258722eedfa387e99c3175f

```
"""Driver: apex counterterm inventory, ST constraints (sigma_0-closure), exact directions, gh1 cohomology,
kappa^1 one-point apex sector, and the flat-bulk sanity check (reproduces the XVI-P p14r 5/2/3/1 counts).
usage: python3 apex_run.py <alpha as fraction p/q> <branch CR|F> [prime_index]"""
import sys, json, random, time
from fractions import Fraction as Fr
sys.path.insert(0, '/workspace/XVI-01-CONE/code')
import apex_jets as AJ
from apex_jets import Setup, Ops, rand_cfg, candidates, eval_sym, eval_cand, rank_and_basis, nullspace, foreign, negq, reflect
alpha = Fr(sys.argv[1]); branch = sys.argv[2]; pi = int(sys.argv[3]) if len(sys.argv) > 3 else 0
AJ.setprime(AJ.PRIMES[pi]); P = AJ.P
random.seed(12345 + pi)
S = Setup(alpha, branch)
T0 = time.time()
def rq(): return [random.randrange(1, 1000), random.randrange(1, 1000)]
NS = 60
DM = 6
SECT = {'HH': ('h', 'h', True), 'CB': ('cb', 'c', False), 'CK': ('K', 'c', False), 'PP': ('phi', 'phi', False)}
SECT['PP'] = ('phi', 'phi', True)
res = {'alpha': str(alpha), 'branch': branch, 'prime': P}
# ---------- candidate bases per gh0 sector
basis = {}; dims = {}; jetpairs = {}
for name, (F1, F2, bose) in SECT.items():
    cands = candidates(S, F1, F2, 4)
    samples = []
    for _ in range(max(NS, len(cands) + 10)):
        samples.append((rand_cfg(S, F1, DM), rand_cfg(S, F2, DM), rq()))
    rows = [[eval_sym(c, a, b, q, bose) for (a, b, q) in samples] for c in cands]
    r, piv = rank_and_basis(rows)
    basis[name] = [cands[i] for i in piv]; dims[name] = r
    jetpairs[name] = sorted(set((str(c[0]), str(c[1]), c[4]) for c in basis[name]))
res['gh0_candidate_dims'] = dims; res['gh0_jet_pairs'] = jetpairs
print('dims', dims, round(time.time() - T0, 1), flush=True)
# ---------- ST rows (sigma_0 closure)
order = [(name, i) for name in SECT for i in range(len(basis[name]))]
def C(name, i, a, b, q):
    F1, F2, bose = SECT[name]; return eval_sym(basis[name][i], a, b, q, bose)
rows = []; foreign_hits = []
nrow = len(order) + 20
for _ in range(nrow):   # row [h c]: 2 C_hh(h, K2 c) + C_ck(E h, c)
    q = rq(); h = rand_cfg(S, 'h', DM); c = rand_cfg(S, 'c', DM)
    K2c = Ops(S, q).K2(c); Eh = Ops(S, negq(q)).E(h)
    fh = foreign(S, 'h', K2c)
    if fh: foreign_hits.append(('K2c', fh[:4]))
    row = []
    for (name, i) in order:
        if name == 'HH': row.append(2 * C(name, i, h, K2c, q) % P)
        elif name == 'CK': row.append(C(name, i, Eh, c, q))
        else: row.append(0)
    rows.append(row)
```

```
for _ in range(nrow):   # row [b c]: C_cb(b, c) + C_ck(C2^T b, c)
    q = rq(); b = rand_cfg(S, 'b', DM); c = rand_cfg(S, 'c', DM)
    C2Tb = Ops(S, negq(q)).C2T(b)
    row = []
    for (name, i) in order:
        if name == 'CB': row.append(C(name, i, b, c, q))
        elif name == 'CK': row.append(C(name, i, C2Tb, c, q))
        else: row.append(0)
    rows.append(row)
Z = nullspace(rows, len(order))
res['N_total'] = len(order); res['Z_dim'] = len(Z)
# sector-resolved: which sectors are entirely free / constrained
res['Z_by_sector_support'] = {}
for name in SECT:
    idx = [k for k, (nm, i) in enumerate(order) if nm == name]
    sub = [[z[k] for k in idx] for z in Z]
    res['Z_by_sector_support'][name] = rank_and_basis(sub)[0] if sub else 0
res['foreign_atoms_in_K2c'] = foreign_hits[:3]; res['n_foreign_samples'] = len(foreign_hits)
print('Z', len(Z), 'of', len(order), 'foreign', len(foreign_hits), round(time.time() - T0, 1), flush=True)
# ---------- exact directions (gh -1): X1=(h,cb), X2=(h,K), X3=(cs,c)
XS = {'X1': ('h', 'cb', 3), 'X2': ('h', 'K', 3), 'X3': ('cs', 'c', 3)}
xb = {}
for nm, (F1, F2, T) in XS.items():
    cands = candidates(S, F1, F2, T)
    samples = [(rand_cfg(S, F1, DM), rand_cfg(S, F2, DM), rq()) for _ in range(len(cands) + 10)]
    rws = [[eval_sym(c, a, b, q, False) for (a, b, q) in samples] for c in cands]
    r, piv = rank_and_basis(rws); xb[nm] = [cands[i] for i in piv]
res['gh-1_dims'] = {k: len(v) for k, v in xb.items()}
xorder = [(nm, i) for nm in XS for i in range(len(xb[nm]))]
def X(nm, i, a, b, q): return eval_sym(xb[nm][i], a, b, q, False)
# sigma_0 images evaluated on gh0 sample pairs of sectors HH, CB, CK and the non-permitted (h,b)
def img_samples(n):
    out = []
    for _ in range(n):
        q = rq()
        out.append(('HH', rand_cfg(S, 'h', DM), rand_cfg(S, 'h', DM), q))
        out.append(('CB', rand_cfg(S, 'cb', DM), rand_cfg(S, 'c', DM), q))
        out.append(('CK', rand_cfg(S, 'K', 3), rand_cfg(S, 'c', DM), q))
        out.append(('HB', rand_cfg(S, 'h', DM), rand_cfg(S, 'b', DM), q))
    return out
smp = img_samples(len(order) + len(xorder) + 10)
def sigma_X(nm, i, sec, a, b, q):
    """value of sigma_0 X on a gh0 sample (sec; slot A cfg a at -q, slot B cfg b at q)"""
    if nm == 'X1':   # X1(h,cb): s -> X1(K2 c, cb) + X1(h, b)
        if sec == 'CB':   # (cb slot A, c slot B): -X1(K2 c [momentum q, plays slot A], cb [slot B, momentum -q]) (odd swap)
            return (-X(nm, i, Ops(S, q).K2(b), a, negq(q))) % P
        if sec == 'HB': return X(nm, i, a, b, q)
        return 0
    if nm == 'X2':   # X2(h,K): s -> X2(K2 c, K) + X2(h, E h' + C2^T b)
        if sec == 'CK': return (-X(nm, i, Ops(S, q).K2(b), a, negq(q))) % P   # odd swap
        if sec == 'HH':
            v = X(nm, i, a, Ops(S, q).E(b), q) + X(nm, i, b, Ops(S, negq(q)).E(a), negq(q))
            return v * pow(2, P - 2, P) % P   # polarization convention: F(h)=C_sym(h,h)
        if sec == 'HB': return X(nm, i, a, Ops(S, q).C2T(b), q)
        return 0
    if nm == 'X3':   # X3(cs,c): s -> X3(K2^T K - (C2K2)^T cb, c)
        if sec == 'CK': return X(nm, i, Ops(S, negq(q)).K2T(a), b, q)
        if sec == 'CB': return (-X(nm, i, Ops(S, negq(q)).ghost(a), b, q)) % P
        return 0
Mimg = [[sigma_X(nm, i, sec, a, b, q) for (sec, a, b, q) in smp] for (nm, i) in xorder]
Mbas = [[(C(name, i, a, b, q) if sec == name else 0) for (sec, a, b, q) in smp] for (name, i) in order]
# constraint: zero HB component
hbcols = [k for k, s in enumerate(smp) if s[0] == 'HB']
A_hb = [[Mimg[r][k] for r in range(len(xorder))] for k in hbcols]
Xok = nullspace(A_hb, len(xorder))
imgs = []
for xv in Xok:
    vec = [sum(xv[r] * Mimg[r][k] for r in range(len(xorder))) % P for k in range(len(smp))]
    imgs.append(vec)
rB, _ = rank_and_basis(imgs) if imgs else (0, [])
rBas, _ = rank_and_basis(Mbas)
rBoth, _ = rank_and_basis(Mbas + imgs)
res['exact_dim_B'] = rB; res['B_in_candidate_span'] = (rBoth == rBas)
# B subset of Z? express images in basis and test rows
if imgs and rBoth == rBas:
    # solve coefficients: for each image vec find y with y.Mbas = vec  (use nullspace of [Mbas; -vec]^T)
    inZ = True; Zrows = rows
    for vec in imgs:
        aug = [[Mbas[r][k] for r in range(len(order))] + [(-vec[k]) % P] for k in range(len(smp))]
        ns = nullspace(aug, len(order) + 1)
        sol = next((v for v in ns if v[-1]), None)
        if sol is None: inZ = False; break
        inv = pow(sol[-1], P - 2, P); y = [x * inv % P for x in sol[:-1]]
        if any(sum(rw[k] * y[k] for k in range(len(order))) % P for rw in Zrows): inZ = False; break
```

```
    res['B_subset_Z'] = inZ
res['H0_dim'] = len(Z) - rB if res.get('B_subset_Z') else None
print('B', rB, res.get('B_subset_Z'), round(time.time() - T0, 1), flush=True)
# ---------- gh1 cohomology (anomaly candidates): sectors HC=(h,c), BC=(b,c)
G1 = {'HC': ('h', 'c'), 'BC': ('b', 'c')}
gb = {}
for nm, (F1, F2) in G1.items():
    cands = candidates(S, F1, F2, 5)
    samples = [(rand_cfg(S, F1, DM), rand_cfg(S, F2, DM), rq()) for _ in range(len(cands) + 10)]
    rws = [[eval_sym(c, a, b, q, False) for (a, b, q) in samples] for c in cands]
    r, piv = rank_and_basis(rws); gb[nm] = [cands[i] for i in piv]
res['gh1_dims'] = {k: len(v) for k, v in gb.items()}
gorder = [(nm, i) for nm in G1 for i in range(len(gb[nm]))]
# WZ consistency: Delta_hc(K2 c1, c2) - Delta_hc(K2 c2, c1) = 0 ; BC part unconstrained
wz = []
for _ in range(len(gorder) + 15):
    q = rq(); c1 = rand_cfg(S, 'c', DM); c2 = rand_cfg(S, 'c', DM)
    row = []
    for (nm, i) in gorder:
        if nm == 'HC':
            v = eval_sym(gb[nm][i], Ops(S, negq(q)).K2(c1), c2, q, False) - eval_sym(gb[nm][i], Ops(S, q).K2(c2), c1, negq(q), False)
            row.append(v % P)
        else: row.append(0)
    wz.append(row)
Zg1 = nullspace(wz, len(gorder))
# exact gh1 = sigma_0 (gh0 candidates) : evaluate the row maps on gh1 samples and fit into gh1 basis
gsm = []
for _ in range(len(gorder) + len(order) + 10):
    q = rq(); gsm.append(('HC', rand_cfg(S, 'h', DM), rand_cfg(S, 'c', DM), q)); gsm.append(('BC', rand_cfg(S, 'b', DM), rand_cfg(S, '
     c', DM), q))
def sig0(name, i, sec, a, b, q):
    if sec == 'HC':
        if name == 'HH': return 2 * C(name, i, a, Ops(S, q).K2(b), q) % P
        if name == 'CK': return C(name, i, Ops(S, negq(q)).E(a), b, q)
        return 0
    if sec == 'BC':
        if name == 'CB': return C(name, i, a, b, q)
        if name == 'CK': return C(name, i, Ops(S, negq(q)).C2T(a), b, q)
        return 0
Gimg = [[sig0(name, i, sec, a, b, q) for (sec, a, b, q) in gsm] for (name, i) in order]
Gbas = [[(eval_sym(gb[nm][i], a, b, q, False) if sec == nm else 0) for (sec, a, b, q) in gsm] for (nm, i) in gorder]
rG, _ = rank_and_basis(Gimg); rGb, _ = rank_and_basis(Gbas); rGB, _ = rank_and_basis(Gbas + Gimg)
res['gh1_Z_dim'] = len(Zg1); res['gh1_exact_dim'] = rG; res['gh1_image_in_span'] = (rGB == rGb)
# images in gh1 coordinates -> WZ rows must annihilate them (sigma_0^2 = 0 check)
closed = None
if rGB == rGb:
    closed = True
    for vec in Gimg:
        aug = [[Gbas[r][k] for r in range(len(gorder))] + [(-vec[k]) % P] for k in range(len(gsm))]
        ns = nullspace(aug, len(gorder) + 1); sol = next((v for v in ns if v[-1]), None)
        if sol is None: closed = False; break
        inv = pow(sol[-1], P - 2, P); y = [x * inv % P for x in sol[:-1]]
        if any(sum(rw[k] * y[k] for k in range(len(gorder))) % P for rw in wz): closed = False; break
res['gh1_images_WZ_closed'] = closed
res['H1_dim'] = len(Zg1) - rG if (rGB == rGb and closed) else None
print('gh1', len(gorder), len(Zg1), rG, rGB == rGb, round(time.time() - T0, 1), flush=True)
# ---------- kappa^1 one-point apex sector (single h, n=0, [h]+m=3, k=0)
one = []
for (Cc, n, m) in S.jets('h', 3):
    if n != 0 or 1 + m != 3: continue
    rk = AJ.TEN[Cc][1]
    if rk == 2: one.append((Cc, m, 'trace'))
    elif rk == 0: one.append((Cc, m, 'value'))
def ev1(c, cfg):
    Cc, m, kind = c; tot = 0
    for cf in (cfg, reflect(cfg)):
        v = cf.get(Cc, {}).get((0, m))
        if v is None: continue
        tot += (v[0] + v[3]) if kind == 'trace' else v[0]
    return tot % P
samp1 = [rand_cfg(S, 'h', DM) for _ in range(20)]
r1, piv1 = rank_and_basis([[ev1(c, h) for h in samp1] for c in one]); one = [one[i] for i in piv1]
q0 = [0, 0]
st1 = []
for _ in range(20):
    k2c = Ops(S, q0).K2(rand_cfg(S, 'c', DM)); st1.append([ev1(c, k2c) for c in one])
Z1 = nullspace(st1, len(one))
# linearized curvature invariants at W: R, R_ii (=Ric_PM), R_ijij (2D transverse curvature)
def jet0(X, comp='S'): return X.get(comp, {}).get((0, 0), [0])[0]
def curv(h):
    o = Ops(S, q0)
    R = o.tadd(o.divv(o.divt(h)), o.box(o.tr(h), {'S': 0}), facs=[1, P - 1])
    Ric = o.Ric(h)
    # transverse 2D curvature: div_perp div_perp h_perp - Lap_perp tr_perp h
```

```
    dP = o.addd(o.Dwb(h.get('PM', {}), 0), o.Dw(h.get('PP', {}), 2)); dM = o.addd(o.Dwb(h.get('MM', {}), -2), o.Dw(h.get('PM', {}), 0))
    ddp = o.addd(o.Dwb(dM, -1), o.Dw(dP, 1)); lap = o.sc(o.Dw(o.Dwb(h.get('PM', {}), 0), 1), 4)
    R2 = o.addd(ddp, lap, facs=[1, P - 1])
    return [jet0(R), jet0(Ric, 'PM'), R2.get((0, 0), [0])[0]]
cv = [curv(h) for h in samp1]
M1 = [[ev1(c, h) for h in samp1] for c in one] + [[cv[s][j] for s in range(len(samp1))] for j in range(3)]
rc, _ = rank_and_basis([[cv[s][j] for s in range(len(samp1))] for j in range(3)])
rall, _ = rank_and_basis(M1)
res['kappa1_onepoint'] = {'candidates': [list(map(str, c)) for c in one], 'N': len(one), 'ST_closed_dim': len(Z1),
                          'curvature_invariants_rank': rc, 'curvatures_in_candidate_span': (rall == len(one))}
print('1pt', res['kappa1_onepoint'], flush=True)
res['seconds'] = round(time.time() - T0, 1)
json.dump(res, open('/workspace/XVI-01-CONE/logs/v2apex/apex_%s_%s_p%d.json' % (str(alpha).replace('/', '_'), branch, pi), 'w'),
      indent=1)
print(json.dumps({k: v for k, v in res.items() if k != 'gh0_jet_pairs'}, indent=1))
```

1.4 tri_count.py

SHA-256: 6f0996dd4220a3d64194d80add4911206b216f22a02c4e63bca1dee6df75e6df

```
import sys
from fractions import Fraction as Fr
sys.path.insert(0, '/workspace/XVI-01-CONE/code')
import apex_jets as AJ
from apex_jets import Setup, pairings, FIELDS, SCA
FIELDS['phis'] = (SCA, 2, 'scalar')
def jets0(S, F, dmax):
    return [j for j in S.jets(F, dmax) if j[1] == 0]
def tri_cands(S, F1, F2, F3, T):
    out = []
    d = [FIELDS[F][1] for F in (F1, F2, F3)]
    for j1 in jets0(S, F1, T):
        for j2 in jets0(S, F2, T):
            for j3 in jets0(S, F3, T):
                k = Fr(T) - sum(d) - j1[2] - j2[2] - j3[2]
                if k.denominator != 1 or k < 0: continue
                k = int(k); r = [FIELDS[F][0][j[0]][1] for F, j in ((F1, j1), (F2, j2), (F3, j3))]
                for k1 in range(k + 1):
                    k2 = k - k1
                    slots = [('A', i) for i in range(r[0])] + [('B', i) for i in range(r[1])] + [('C', i) for i in range(r[2])] + [('p1', i) for i in range(k1)] + [('p2', i) for i in range(k2)]
                    if len(slots) % 2: continue
                    seen = set()
                    for pr in pairings(slots):
                        key = tuple(sorted(tuple(sorted((x[0], x[1] if x[0] not in ('p1', 'p2') else 0) for x in pair)) for pair in pr))
                        if key in seen: continue
                        seen.add(key); out.append((j1, j2, j3, r, k1, k2, pr))
    return out
if __name__ == '__main__':
    S = Setup(Fr(7071, 10000), 'CR')
    for sec, (F1, F2, F3, T) in {'HHH': ('h','h','h',5), 'PPH': ('phi','phi','h',5), 'KCH': ('K','c','h',5), 'SCC': ('cs','c','c',5), 'FPC': ('phis','phi','c',5),
                                 'HHC': ('h','h','c',6), 'PPC': ('phi','phi','c',6), 'KCC': ('K','c','c',6), 'HCC': ('h','c','c',7)}.items():
        print(sec, len(tri_cands(S, F1, F2, F3, T)), flush=True)
```

1.5 tri_count_all.py

SHA-256: 0010edb17e2b34e4be18c4b0f8673e5faa0b7a0c8616eb25155f229bc45b6d31

```
"""v3 item (5): trilinear apex candidates with ALL lattice jets (n != 0 allowed), rotation charge n1+n2+n3=0.
Needed at resonant alpha, where n != 0 jets can enter the integer power-counting window."""
import sys
from fractions import Fraction as Fr
sys.path.insert(0, '/workspace/XVI-01-CONE/code')
import apex_jets as AJ
from apex_jets import pairings, FIELDS, SCA
FIELDS['phis'] = (SCA, 2, 'scalar')
def tri_cands(S, F1, F2, F3, T):
    out = []
    d = [FIELDS[F][1] for F in (F1, F2, F3)]
    J = [S.jets(F, T) for F in (F1, F2, F3)]
    for j1 in J[0]:
        for j2 in J[1]:
            for j3 in J[2]:
                if j1[1] + j2[1] + j3[1] != 0: continue
                k = Fr(T) - sum(d) - Fr(j1[2]) - Fr(j2[2]) - Fr(j3[2])
```

```
                if k.denominator != 1 or k < 0: continue
                k = int(k); r = [FIELDS[F][0][j[0]][1] for F, j in ((F1, j1), (F2, j2), (F3, j3))]
                for k1 in range(k + 1):
                    k2 = k - k1
                    slots = [('A', i) for i in range(r[0])] + [('B', i) for i in range(r[1])] + [('C', i) for i in range(r[2])] + [('p1', i) for i in range(k1)] + [('p2', i) for i in range(k2)]
                    if len(slots) % 2: continue
                    seen = set()
                    for pr in pairings(slots):
                        key = tuple(sorted(tuple(sorted((x[0], x[1] if x[0] not in ('p1', 'p2') else 0) for x in pair)) for pair in pr))
                        if key in seen: continue
                        seen.add(key); out.append((j1, j2, j3, r, k1, k2, pr))
    return out
```

1.6 tri_run.py

SHA-256: b8d010a7805b489747c1857d06119ef1db2157005d0f5755d82ed62f4c9551c7

```
"""(E) Trilinear (three-field) linearised BRST cohomology at the apex, ghost number 1, generic alpha, CR branch.
Minimal BV complex (non-minimal pair cbar,b contractible and dropped): s h = K2 c, s K = E h, s c* = K2^T K,
s phi* = box phi, s c = s phi = 0.  Sectors (phi-parity even, n=0 jets only: proved sufficient at non-resonant alpha):
gh0: HHH(h,h,h) PPH(phi,phi,h) KCH(K,c,h) SCC(c*,c,c) FPC(phi*,phi,c)  [T=5]
gh1: HHC(h,h,c) PPC(phi,phi,c) KCC(K,c,c)                              [T=6]
closure into gh2 HCC(h,c,c) [T=7]; commuting-placeholder conventions: identical bosons polarised with 1/n!,
identical ghosts antisymmetrised; relative signs fixed by sigma0^2=0 (checked).  usage: tri_run.py <alpha> <prime_index>"""
import sys, random, json, time, itertools
from fractions import Fraction as Fr
sys.path.insert(0, '/workspace/XVI-01-CONE/code')
import apex_jets as AJ
from apex_jets import Setup, Ops, rand_cfg, reflect, rank_and_basis, nullspace, FIELDS, SCA
from tri_count import tri_cands
FIELDS['phis'] = (SCA, 2, 'scalar')
alpha = Fr(sys.argv[1]); pi = int(sys.argv[2]) if len(sys.argv) > 2 else 0
AJ.setprime(AJ.PRIMES[pi]); P = AJ.P; random.seed(4242 + pi)
S = Setup(alpha, 'CR'); T0 = time.time(); DM = 7
def rq(): return [random.randrange(1, 1000), random.randrange(1, 1000)]
def neg(q): return [(-x) % P for x in q]
def add(a, b): return [(x + y) % P for x, y in zip(a, b)]
def raw(c, cfgs, q1, q2):
    j1, j2, j3, r, k1, k2, pr = c
    vs = []
    for cfg, j in zip(cfgs, (j1, j2, j3)):
        v = cfg.get(j[0], {}).get((j[1], j[2]))
        if v is None: return 0
        vs.append(v)
    tot = 0
    for vals in itertools.product(range(2), repeat=len(pr)):
        lab = {}
        for (x, y), v in zip(pr, vals): lab[x] = v; lab[y] = v
        ia = 0
        for i in range(r[0]): ia = ia * 2 + lab[('A', i)]
        ib = 0
        for i in range(r[1]): ib = ib * 2 + lab[('B', i)]
        ic = 0
        for i in range(r[2]): ic = ic * 2 + lab[('C', i)]
        t = vs[0][ia] * vs[1][ib] % P * vs[2][ic] % P
        for i in range(k1): t = t * q1[lab[('p1', i)]] % P
        for i in range(k2): t = t * q2[lab[('p2', i)]] % P
        tot += t
    return tot % P
def ev(c, cfgs, q1, q2):   # reflection-symmetrised raw evaluation; legs momenta q1, q2, q3=-q1-q2
    return (raw(c, cfgs, q1, q2) + raw(c, [reflect(x) for x in cfgs], q1, q2)) % P
I2 = pow(2, P - 2, P); I6 = pow(6, P - 2, P)
def pol(sec, c, cfgs, qs):
    """polarised evaluation in the sector convention; cfgs/qs = 3 legs (q3 = -q1-q2 implied by qs[2])"""
    a, b, d = cfgs; qa, qb, qd = qs
    if sec in ('HHH',):
        tot = 0
        for p in itertools.permutations(range(3)):
            cc = [cfgs[i] for i in p]; qq = [qs[i] for i in p]
            tot += ev(c, cc, qq[0], qq[1])
        return tot % P * I6 % P
    if sec in ('PPH', 'HHC', 'PPC'):
        return (ev(c, [a, b, d], qa, qb) + ev(c, [b, a, d], qb, qa)) % P * I2 % P
    if sec in ('KCC', 'SCC'):
        return (ev(c, [a, b, d], qa, qb) - ev(c, [a, d, b], qa, qd)) % P
    return ev(c, [a, b, d], qa, qb)   # KCH, FPC
SECT = {'HHH': ('h', 'h', 'h', 5), 'PPH': ('phi', 'phi', 'h', 5), 'KCH': ('K', 'c', 'h', 5), 'SCC': ('cs', 'c', 'c', 5), 'FPC': ('phis', 'phi', 'c', 5),
        'HHC': ('h', 'h', 'c', 6), 'PPC': ('phi', 'phi', 'c', 6), 'KCC': ('K', 'c', 'c', 6)}
def sample(sec):
```

```
    F1, F2, F3, _ = SECT[sec]; q1 = rq(); q2 = rq(); q3 = neg(add(q1, q2))
    return [rand_cfg(S, F1, DM), rand_cfg(S, F2, DM), rand_cfg(S, F3, DM)], [q1, q2, q3]
basis = {}; res = {'alpha': str(alpha), 'prime': P}
for sec, (F1, F2, F3, T) in SECT.items():
    cl = tri_cands(S, F1, F2, F3, T)
    sm = [sample(sec) for _ in range(len(cl) + 12)]
    rows = [[pol(sec, c, cf, qs) for (cf, qs) in sm] for c in cl]
    rk, piv = rank_and_basis(rows); basis[sec] = [cl[i] for i in piv]
    res.setdefault('raw', {})[sec] = len(cl); res.setdefault('dims', {})[sec] = rk
    print(sec, len(cl), rk, round(time.time() - T0, 1), flush=True)
G1 = ['HHC', 'PPC', 'KCC']; G0 = ['HHH', 'PPH', 'KCH', 'SCC', 'FPC']
gorder = [(s, i) for s in G1 for i in range(len(basis[s]))]
# ---- closure rows on gh2 samples (h@qh, c1@q1, c2@q2)
cl_rows = []
for _ in range(len(gorder) + 15):
    qh = rq(); q1 = rq(); q2 = neg(add(qh, q1))
    h = rand_cfg(S, 'h', DM); c1 = rand_cfg(S, 'c', DM); c2 = rand_cfg(S, 'c', DM)
    K2c1 = Ops(S, q1).K2(c1); K2c2 = Ops(S, q2).K2(c2); Eh = Ops(S, qh).E(h)
    row = []
    for (s, i) in gorder:
        c = basis[s][i]
        if s == 'HHC':
            v = 2 * (pol('HHC', c, [K2c1, h, c2], [q1, qh, q2]) - pol('HHC', c, [K2c2, h, c1], [q2, qh, q1]))
        elif s == 'KCC':
            v = pol('KCC', c, [Eh, c1, c2], [qh, q1, q2])
        else: v = 0
        row.append(v % P)
    cl_rows.append(row)
Zg1 = nullspace(cl_rows, len(gorder)); res['gh1_closed_dim'] = len(Zg1)
print('closed', len(Zg1), 'of', len(gorder), round(time.time() - T0, 1), flush=True)
# ---- exact: images of gh0 basis on gh1 samples
gs = []
for _ in range(len(gorder) + 20):
    for s in G1: gs.append((s,) + tuple(sample(s)))
def image(s0, c, smp):
    s1, cfgs, qs = smp; a, b, d = cfgs; qa, qb, qd = qs
    if s0 == 'HHH' and s1 == 'HHC':
        return 3 * pol('HHH', c, [Ops(S, qd).K2(d), a, b], [qd, qa, qb]) % P
    if s0 == 'PPH' and s1 == 'PPC':
        return pol('PPH', c, [a, b, Ops(S, qd).K2(d)], [qa, qb, qd])
    if s0 == 'KCH' and s1 == 'HHC':
        return (ev(c, [Ops(S, qa).E(a), d, b], qa, qd) + ev(c, [Ops(S, qb).E(b), d, a], qb, qd)) % P * I2 % P
    if s0 == 'KCH' and s1 == 'KCC':
        return (ev(c, [a, b, Ops(S, qd).K2(d)], qa, qb) - ev(c, [a, d, Ops(S, qb).K2(b)], qa, qd)) % P
    if s0 == 'SCC' and s1 == 'KCC':
        return pol('SCC', c, [Ops(S, qa).K2T(a), b, d], [qa, qb, qd])
    if s0 == 'FPC' and s1 == 'PPC':
        bx = lambda f, q: {'S': Ops(S, q).box(f, {'S': 0})['S']}
        return (ev(c, [bx(a, qa), b, d], qa, qb) + ev(c, [bx(b, qb), a, d], qb, qa)) % P * I2 % P
    return 0
order0 = [(s, i) for s in G0 for i in range(len(basis[s]))]
Gimg = [[image(s, basis[s][i], smp) for smp in gs] for (s, i) in order0]
Gbas = [[(pol(s, basis[s][i], smp[1], smp[2]) if smp[0] == s else 0) for smp in gs] for (s, i) in gorder]
rG, _ = rank_and_basis(Gimg); rGb, _ = rank_and_basis(Gbas); rGB, _ = rank_and_basis(Gbas + Gimg)
res['gh1_exact_dim'] = rG; res['images_in_span'] = (rGB == rGb); res['gh1_basis_rank_on_samples'] = rGb
closed = None
if rGB == rGb:
    closed = True
    for vec in Gimg:
        aug = [[Gbas[r][k] for r in range(len(gorder))] + [(-vec[k]) % P] for k in range(len(gs))]
        ns = nullspace(aug, len(gorder) + 1); sol = next((v for v in ns if v[-1]), None)
        if sol is None: closed = False; break
        inv = pow(sol[-1], P - 2, P); y = [x * inv % P for x in sol[:-1]]
        if any(sum(rw[k] * y[k] for k in range(len(gorder))) % P for rw in cl_rows): closed = False; break
res['images_closed_sigma0sq'] = closed
res['H1_trilinear'] = len(Zg1) - rG if closed else None
# sector-resolved: H1 in the PPC block alone and HHC+KCC block
print(json.dumps(res), round(time.time() - T0, 1), flush=True)
json.dump(res, open('/workspace/XVI-01-CONE/logs/v2_tri_%s_p%d.json' % (str(alpha).replace('/', '_'), pi), 'w'), indent=1)
```

1.7 tri_run_all.py

SHA-256: 54f7e86a629bf5edaad447c58eb2f35c40e10ca78117cb56c3853b77bb1049c5

```
"""(E) Trilinear (three-field) linearised BRST cohomology at the apex, ghost number 1, generic alpha, CR branch.
Minimal BV complex (non-minimal pair cbar,b contractible and dropped): s h = K2 c, s K = E h, s c* = K2^T K,
s phi* = box phi, s c = s phi = 0.  Sectors (phi-parity even, n=0 jets only: proved sufficient at non-resonant alpha):
gh0: HHH(h,h,h) PPH(phi,phi,h) KCH(K,c,h) SCC(c*,c,c) FPC(phi*,phi,c)  [T=5]
gh1: HHC(h,h,c) PPC(phi,phi,c) KCC(K,c,c)                              [T=6]
closure into gh2 HCC(h,c,c) [T=7]; commuting-placeholder conventions: identical bosons polarised with 1/n!,
identical ghosts antisymmetrised; relative signs fixed by sigma0^2=0 (checked).  usage: tri_run.py <alpha> <prime_index>"""
import sys, random, json, time, itertools
```

```
from fractions import Fraction as Fr
sys.path.insert(0, '/workspace/XVI-01-CONE/code')
import apex_jets as AJ
from apex_jets import Setup, Ops, rand_cfg, reflect, rank_and_basis, nullspace, FIELDS, SCA
from tri_count_all import tri_cands
FIELDS['phis'] = (SCA, 2, 'scalar')
alpha = Fr(sys.argv[1]); pi = int(sys.argv[2]) if len(sys.argv) > 2 else 0
AJ.setprime(AJ.PRIMES[pi]); P = AJ.P; random.seed(4242 + pi)
S = Setup(alpha, "CR"); T0 = time.time(); DM = 7
def rq(): return [random.randrange(1, 1000), random.randrange(1, 1000)]
def neg(q): return [(-x) % P for x in q]
def add(a, b): return [(x + y) % P for x, y in zip(a, b)]
def raw(c, cfgs, q1, q2):
    j1, j2, j3, r, k1, k2, pr = c
    vs = []
    for cfg, j in zip(cfgs, (j1, j2, j3)):
        v = cfg.get(j[0], {}).get((j[1], j[2]))
        if v is None: return 0
        vs.append(v)
    tot = 0
    for vals in itertools.product(range(2), repeat=len(pr)):
        lab = {}
        for (x, y), v in zip(pr, vals): lab[x] = v; lab[y] = v
        ia = 0
        for i in range(r[0]): ia = ia * 2 + lab[('A', i)]
        ib = 0
        for i in range(r[1]): ib = ib * 2 + lab[('B', i)]
        ic = 0
        for i in range(r[2]): ic = ic * 2 + lab[('C', i)]
        t = vs[0][ia] * vs[1][ib] % P * vs[2][ic] % P
        for i in range(k1): t = t * q1[lab[('p1', i)]] % P
        for i in range(k2): t = t * q2[lab[('p2', i)]] % P
        tot += t
    return tot % P
def ev(c, cfgs, q1, q2):   # reflection-symmetrised raw evaluation; legs momenta q1, q2, q3=-q1-q2
    return (raw(c, cfgs, q1, q2) + raw(c, [reflect(x) for x in cfgs], q1, q2)) % P
I2 = pow(2, P - 2, P); I6 = pow(6, P - 2, P)
def pol(sec, c, cfgs, qs):
    """polarised evaluation in the sector convention; cfgs/qs = 3 legs (q3 = -q1-q2 implied by qs[2])"""
    a, b, d = cfgs; qa, qb, qd = qs
    if sec in ('HHH',):
        tot = 0
        for p in itertools.permutations(range(3)):
            cc = [cfgs[i] for i in p]; qq = [qs[i] for i in p]
            tot += ev(c, cc, qq[0], qq[1])
        return tot % P * I6 % P
    if sec in ('PPH', 'HHC', 'PPC'):
        return (ev(c, [a, b, d], qa, qb) + ev(c, [b, a, d], qb, qa)) % P * I2 % P
    if sec in ('KCC', 'SCC'):
        return (ev(c, [a, b, d], qa, qb) - ev(c, [a, d, b], qa, qd)) % P
    return ev(c, [a, b, d], qa, qb)   # KCH, FPC
SECT = {'HHH': ('h', 'h', 'h', 5), 'PPH': ('phi', 'phi', 'h', 5), 'KCH': ('K', 'c', 'h', 5), 'SCC': ('cs', 'c', 'c', 5), 'FPC': ('phis', 'phi', 'c', 5),
        'HHC': ('h', 'h', 'c', 6), 'PPC': ('phi', 'phi', 'c', 6), 'KCC': ('K', 'c', 'c', 6)}
def sample(sec):
    F1, F2, F3, _ = SECT[sec]; q1 = rq(); q2 = rq(); q3 = neg(add(q1, q2))
    return [rand_cfg(S, F1, DM), rand_cfg(S, F2, DM), rand_cfg(S, F3, DM)], [q1, q2, q3]
basis = {}; res = {'alpha': str(alpha), 'prime': P}
for sec, (F1, F2, F3, T) in SECT.items():
    cl = tri_cands(S, F1, F2, F3, T)
    sm = [sample(sec) for _ in range(len(cl) + 12)]
    rows = [[pol(sec, c, cf, qs) for (cf, qs) in sm] for c in cl]
    rk, piv = rank_and_basis(rows); basis[sec] = [cl[i] for i in piv]
    res.setdefault('raw', {})[sec] = len(cl); res.setdefault('dims', {})[sec] = rk
    print(sec, len(cl), rk, round(time.time() - T0, 1), flush=True)
G1 = ['HHC', 'PPC', 'KCC']; G0 = ['HHH', 'PPH', 'KCH', 'SCC', 'FPC']
gorder = [(s, i) for s in G1 for i in range(len(basis[s]))]
# ---- closure rows on gh2 samples (h@qh, c1@q1, c2@q2)
cl_rows = []
for _ in range(len(gorder) + 15):
    qh = rq(); q1 = rq(); q2 = neg(add(qh, q1))
    h = rand_cfg(S, 'h', DM); c1 = rand_cfg(S, 'c', DM); c2 = rand_cfg(S, 'c', DM)
    K2c1 = Ops(S, q1).K2(c1); K2c2 = Ops(S, q2).K2(c2); Eh = Ops(S, qh).E(h)
    row = []
    for (s, i) in gorder:
        c = basis[s][i]
        if s == 'HHC':
            v = 2 * (pol('HHC', c, [K2c1, h, c2], [q1, qh, q2]) - pol('HHC', c, [K2c2, h, c1], [q2, qh, q1]))
        elif s == 'KCC':
            v = pol('KCC', c, [Eh, c1, c2], [qh, q1, q2])
        else: v = 0
        row.append(v % P)
    cl_rows.append(row)
Zg1 = nullspace(cl_rows, len(gorder)); res['gh1_closed_dim'] = len(Zg1)
```

```
print('closed', len(Zg1), 'of', len(gorder), round(time.time() - T0, 1), flush=True)
# ---- exact: images of gh0 basis on gh1 samples
gs = []
for _ in range(len(gorder) + 20):
    for s in G1: gs.append((s,) + tuple(sample(s)))
def image(s0, c, smp):
    s1, cfgs, qs = smp; a, b, d = cfgs; qa, qb, qd = qs
    if s0 == 'HHH' and s1 == 'HHC':
        return 3 * pol('HHH', c, [Ops(S, qd).K2(d), a, b], [qd, qa, qb]) % P
    if s0 == 'PPH' and s1 == 'PPC':
        return pol('PPH', c, [a, b, Ops(S, qd).K2(d)], [qa, qb, qd])
    if s0 == 'KCH' and s1 == 'HHC':
        return (ev(c, [Ops(S, qa).E(a), d, b], qa, qd) + ev(c, [Ops(S, qb).E(b), d, a], qb, qd)) % P * I2 % P
    if s0 == 'KCH' and s1 == 'KCC':
        return (ev(c, [a, b, Ops(S, qd).K2(d)], qa, qb) - ev(c, [a, d, Ops(S, qb).K2(b)], qa, qd)) % P
    if s0 == 'SCC' and s1 == 'KCC':
        return pol('SCC', c, [Ops(S, qa).K2T(a), b, d], [qa, qb, qd])
    if s0 == 'FPC' and s1 == 'PPC':
        bx = lambda f, q: {'S': Ops(S, q).box(f, {'S': 0})['S']}
        return (ev(c, [bx(a, qa), b, d], qa, qb) + ev(c, [bx(b, qb), a, d], qb, qa)) % P * I2 % P
    return 0
order0 = [(s, i) for s in G0 for i in range(len(basis[s]))]
Gimg = [[image(s, basis[s][i], smp) for smp in gs] for (s, i) in order0]
Gbas = [[(pol(s, basis[s][i], smp[1], smp[2]) if smp[0] == s else 0) for smp in gs] for (s, i) in gorder]
rG, _ = rank_and_basis(Gimg); rGb, _ = rank_and_basis(Gbas); rGB, _ = rank_and_basis(Gbas + Gimg)
res['gh1_exact_dim'] = rG; res['images_in_span'] = (rGB == rGb); res['gh1_basis_rank_on_samples'] = rGb
closed = None
if rGB == rGb:
    closed = True
    for vec in Gimg:
        aug = [[Gbas[r][k] for r in range(len(gorder))] + [(-vec[k]) % P] for k in range(len(gs))]
        ns = nullspace(aug, len(gorder) + 1); sol = next((v for v in ns if v[-1]), None)
        if sol is None: closed = False; break
        inv = pow(sol[-1], P - 2, P); y = [x * inv % P for x in sol[:-1]]
        if any(sum(rw[k] * y[k] for k in range(len(gorder))) % P for rw in cl_rows): closed = False; break
res['images_closed_sigma0sq'] = closed
res['H1_trilinear'] = len(Zg1) - rG if closed else None
# sector-resolved: H1 in the PPC block alone and HHC+KCC block
print(json.dumps(res), round(time.time() - T0, 1), flush=True)
json.dump(res, open('/workspace/XVI-01-CONE/logs/v3/tri_all_%s_p%d.json' % (str(alpha).replace('/', '_'), pi), 'w'), indent=1)
```

1.8 resonances.py

SHA-256: c7bebc7a997a4b5780f6028cf09c6861065b696a2ab7937a3e5d1482a5d4d8da

```
"""(A3) Exhaustive resonance set.  CR lemma: for alpha<1 (t=1/alpha>1) every jet of every field obeys
m = |n| t + A, A integer >= -2 (Friedrichs channels: sgn(nt+s)=sgn(n) unless |n|=1,|s|=2,t<2; those two channels are
exactly the CR-flipped ones, m=t-2+2j).  A pairing with some n!=0 needs N t in Z with N=sum|n|>=2 and
sum m <= M (sector budget), hence N < K := N t <= M + 2J (J = #jets with n!=0 = 2).  So the CR-resonant alphas are
among N/K.  We enumerate those and confirm with the engine's own jet lists which ones actually admit an n!=0 pair
(candidate with integer k>=0) in any sector (gh-1 T=3, gh0 T=4, gh1 T=5); for F we scan the same rational set plus the
chamber representatives."""
import sys, json
from fractions import Fraction as Fr
sys.path.insert(0, '/workspace/XVI-01-CONE/code')
from apex_jets import Setup, candidates
SECTORS = [('h','h',4),('cb','c',4),('K','c',4),('phi','phi',4),('h','cb',3),('h','K',3),('cs','c',3),('h','c',5),('b','c',5)]
def nz_pairs(al, br):
    S = Setup(al, br); out = {}
    for F1, F2, T in SECTORS:
        c = [x for x in candidates(S, F1, F2, T) if x[0][1] != 0]
        if c: out['%s.%s(T=%d)' % (F1, F2, T)] = sorted(set((str(x[0]), str(x[1]), x[4]) for x in c))
    return out
cand = set()
for N in range(2, 12, 2):
    for K in range(N + 1, 3 + 4 + 1):
        cand.add(Fr(N, K))
cand = sorted(a for a in cand if 0 < a < 1)
res = {'CR_candidate_set_N_over_K': [str(a) for a in cand], 'CR': {}, 'F': {}}
for a in cand:
    p = nz_pairs(a, 'CR')
    res['CR'][str(a)] = {k: len(v) for k, v in p.items()}
# generic control points (should be empty for CR) and F scan on a fine rational grid
grid = sorted(set(cand) | {Fr(k, 97) for k in range(10, 97)} | {Fr(3, 10), Fr(7071, 10000), Fr(8501, 10000)})
res['CR_nonempty_on_grid'] = [str(a) for a in grid if nz_pairs(a, 'CR')]
res['F_nonempty_on_grid'] = {str(a): {k: len(v) for k, v in nz_pairs(a, 'F').items()} for a in grid if nz_pairs(a, 'F')}
print(json.dumps({'CR': {k: v for k, v in res['CR'].items() if v}, 'CR_grid': res['CR_nonempty_on_grid']}, indent=0))
print('F grid nonempty count', len(res['F_nonempty_on_grid']), 'min', min(map(Fr, res['F_nonempty_on_grid'])) if res['
    F_nonempty_on_grid'] else None)
json.dump(res, open('/workspace/XVI-01-CONE/logs/v2_resonances.json', 'w'), indent=1)
```

1.9 inner_degrees.py

SHA-256: 4988220d048ec5c88ff5fda3961f846fa626817af3776fea3de557f80208c2d5

```
"""v3 item (1): one-point ('inner') apex degrees of bubble integrands.  At a cubic vertex x with two propagator legs
(other ends fixed away from W) and one external leg, the integrand near W is a sum of terms r^s e^{i(sum n)phi} with
s = m1 + m2 + m3 - k, k <= 2 transverse derivatives, m from the I-B-G-S v2 lattice:
  nu = |n t + sigma|, F jet m = nu + 2j;  CR flips (sigma,n) in {(2,-1),(-2,1)} to E0 (m = -nu + 2j) when t<2.
Angular integral forces n1+n2+n3 = 0.  Non-local-integrability: s <= -2.  We list, on a grid of alpha and on both
branches, the minimal s with sum n = 0 and whether some s <= -2 is an INTEGER (unregulated log; needs a new regulator)
or only non-integer (unique homogeneous extension, Hormander Thm 3.2.3).  Legs: graviton sigma in {0,+-1,+-2},
ghost/vector sigma in {0,+-1}, scalar sigma=0."""
import json, itertools
from fractions import Fraction as Fr
SIG = {'h': [0, 1, -1, 2, -2], 'c': [0, 1, -1], 'phi': [0]}
def jets(field, t, br, nmax=3, jmax=1):
    out = []
    for s in SIG[field]:
        for n in range(-nmax, nmax + 1):
            nu = abs(n * t + s)
            flip = (br == 'CR' and field == 'h' and (s, n) in ((2, -1), (-2, 1)) and t < 2)
            for j in range(jmax + 1):
                m = (-nu if flip else nu) + 2 * j
                if not flip and nu == 0 and j == 0: pass
                out.append((n, m, flip))
    return out
GRAPHS = {'D01': ('h', 'h', 'h'), 'D03': ('c', 'c', 'h'), 'D04a': ('h', 'phi', 'phi'), 'D04b': ('phi', 'h', 'phi'),
          'D11': ('phi', 'phi', 'h'), 'D13a': ('c', 'h', 'K'), 'D13b': ('c', 'h', 'c')}
def scan(alpha, br):
    t = 1 / alpha; res = {}
    for gname, (L1, L2, E) in GRAPHS.items():
        Ej = 'h' if E in ('h', 'K') else E
        J1 = jets(L1, t, br); J2 = jets(L2, t, br); J3 = jets(Ej, t, br)
        worst = None; integer_bad = []
        for (n1, m1, f1), (n2, m2, f2), (n3, m3, f3) in itertools.product(J1, J2, J3):
            if n1 + n2 + n3 != 0: continue
            for k in (0, 1, 2):
                s = m1 + m2 + m3 - k
                if worst is None or s < worst: worst = s
                if s <= -2 + 1e-12:
                    isint = abs(s - round(s)) < 1e-9
                    if isint and (n1, n2, n3) != (0, 0, 0): integer_bad.append((n1, n2, n3, round(s, 6), k, f1, f2, f3))
        res[gname] = {'min_s': round(worst, 6), 'integer_nonlocal': integer_bad[:6], 'n_integer_nonlocal': len(integer_bad)}
    return res
out = {}
grid = [Fr(k, 97) for k in range(1, 97)] + [Fr(1, 3), Fr(1, 2), Fr(2, 3), Fr(3, 4), Fr(4, 5), Fr(4, 7), Fr(3, 5), Fr(7071, 10000)]
for a in grid:
    for br in ('CR', 'F'):
        r = scan(float(a), br)
        bad = {g: v['n_integer_nonlocal'] for g, v in r.items() if v['n_integer_nonlocal']}
        out[f'{a}_{br}'] = {'bad': bad, 'min_s': {g: v['min_s'] for g, v in r.items()}, 'examples': {g: v['integer_nonlocal'] for g, v
      in r.items() if v['n_integer_nonlocal']}}
json.dump(out, open('/workspace/XVI-01-CONE/logs/v3/inner_degrees.json', 'w'), indent=1)
crbad = sorted({k.split('_')[0] for k, v in out.items() if k.endswith('_CR') and v['bad']}, key=lambda s: float(Fr(s)))
fbad = sorted({k.split('_')[0] for k, v in out.items() if k.endswith('_F') and v['bad']}, key=lambda s: float(Fr(s)))
print('CR alphas with integer non-local inner degree:', crbad)
print('F  alphas with integer non-local inner degree (count):', len(fbad), fbad[:12], '...')
for a in ['7071/10000', '2/3', '1/2', '1/3']:
    print(a, 'CR', out[a + '_CR']['bad'], out[a + '_CR']['min_s']); print(a, 'F', out[a + '_F']['bad'])
```

1.10 inner_degrees_q24.py

SHA-256: 327d8ef4185c403a1f8b807bc0d9c6f3f4f4659d79793b2682bb96f87a1cd0ec

```
"""v3 item (1): one-point ('inner') apex degrees of bubble integrands.  At a cubic vertex x with two propagator legs
(other ends fixed away from W) and one external leg, the integrand near W is a sum of terms r^s e^{i(sum n)phi} with
s = m1 + m2 + m3 - k, k <= 2 transverse derivatives, m from the I-B-G-S v2 lattice:
  nu = |n t + sigma|, F jet m = nu + 2j;  CR flips (sigma,n) in {(2,-1),(-2,1)} to E0 (m = -nu + 2j) when t<2.
Angular integral forces n1+n2+n3 = 0.  Non-local-integrability: s <= -2.  We list, on a grid of alpha and on both
branches, the minimal s with sum n = 0 and whether some s <= -2 is an INTEGER (unregulated log; needs a new regulator)
or only non-integer (unique homogeneous extension, Hormander Thm 3.2.3).  Legs: graviton sigma in {0,+-1,+-2},
ghost/vector sigma in {0,+-1}, scalar sigma=0."""
import json, itertools
from fractions import Fraction as Fr
SIG = {'h': [0, 1, -1, 2, -2], 'c': [0, 1, -1], 'phi': [0]}
def jets(field, t, br, nmax=5, jmax=1):
    out = []
    for s in SIG[field]:
        for n in range(-nmax, nmax + 1):
            nu = abs(n * t + s)
            flip = (br == 'CR' and field == 'h' and (s, n) in ((2, -1), (-2, 1)) and t < 2)
            for j in range(jmax + 1):
```

```
                m = (-nu if flip else nu) + 2 * j
                if not flip and nu == 0 and j == 0: pass
                out.append((n, m, flip))
    return out
GRAPHS = {'D01': ('h', 'h', 'h'), 'D03': ('c', 'c', 'h'), 'D04a': ('h', 'phi', 'phi'), 'D04b': ('phi', 'h', 'phi'),
          'D11': ('phi', 'phi', 'h'), 'D13a': ('c', 'h', 'K'), 'D13b': ('c', 'h', 'c')}
def scan(alpha, br):
    t = 1 / alpha; res = {}
    for gname, (L1, L2, E) in GRAPHS.items():
        Ej = 'h' if E in ('h', 'K') else E
        J1 = jets(L1, t, br); J2 = jets(L2, t, br); J3 = jets(Ej, t, br)
        worst = None; integer_bad = []
        for (n1, m1, f1), (n2, m2, f2), (n3, m3, f3) in itertools.product(J1, J2, J3):
            if n1 + n2 + n3 != 0: continue
            for k in (0, 1, 2):
                s = m1 + m2 + m3 - k
                if worst is None or s < worst: worst = s
                if s <= -2 + 1e-12:
                    isint = abs(s - round(s)) < 1e-9
                    if isint and (n1, n2, n3) != (0, 0, 0): integer_bad.append((n1, n2, n3, round(s, 6), k, f1, f2, f3))
        res[gname] = {'min_s': round(worst, 6), 'integer_nonlocal': integer_bad[:6], 'n_integer_nonlocal': len(integer_bad)}
    return res
out = {}
grid = sorted({Fr(k, 97) for k in range(1, 97)} | {Fr(p, q) for q in range(2, 25) for p in range(1, q)} | {Fr(7071, 10000)})
for a in grid:
    for br in ('CR', 'F'):
        r = scan(float(a), br)
        bad = {g: v['n_integer_nonlocal'] for g, v in r.items() if v['n_integer_nonlocal']}
        out[f'{a}_{br}'] = {'bad': bad, 'min_s': {g: v['min_s'] for g, v in r.items()}, 'examples': {g: v['integer_nonlocal'] for g, v
      in r.items() if v['n_integer_nonlocal']}}
json.dump(out, open('/workspace/XVI-01-CONE/logs/v3/inner_degrees_q24.json', 'w'), indent=1)
crbad = sorted({k.split('_')[0] for k, v in out.items() if k.endswith('_CR') and v['bad']}, key=lambda s: float(Fr(s)))
fbad = sorted({k.split('_')[0] for k, v in out.items() if k.endswith('_F') and v['bad']}, key=lambda s: float(Fr(s)))
print('CR alphas with integer non-local inner degree:', crbad)
print('F  alphas with integer non-local inner degree (count):', len(fbad), fbad[:12], '...')
for a in ['7071/10000', '2/3', '1/2', '1/3']:
    print(a, 'CR', out[a + '_CR']['bad'], out[a + '_CR']['min_s']); print(a, 'F', out[a + '_F']['bad'])
```

1.11 locality.py

SHA-256: a6ea1ee23102af01e94df979ac12ce7c1cb5a6b1e267fe085d915265917d1420

```
"""Stage-1 regular-region locality checks (Euclidean cone C_alpha x R^2, I-B-G-S-v2 labels).
(a) coincidence: r^2 R_b(x,x) = (B2(b)/alpha^2 - 1/6)/(8 pi^2), R_b = W_b - W_flat(direct);
(b) smoothness of R_b at the diagonal: direction independence of value and gradient, bounded 2nd differences;
(c) bubble log-coefficient: J(rho)=2 pi^2 rho^4 G(x,x+rho n)^2 -> 1/(8 pi^2) with (J-J_flat)/rho^2 -> R_b(x,x).
Run: python3 locality.py <alpha>   (writes logs/locality_<alpha>.json)"""
import sys, json, mpmath as mp
sys.path.insert(0, '/workspace/XVI-01-CONE/code')
from tensor import Gb
mp.mp.dps = 40
al = mp.mpf(sys.argv[1])
labels = {'b0': mp.mpf(0), 'b+1': al, 'b-1': 1 - al, 'b+2': 2 * al, 'b-2': 1 - 2 * al}
if al <= mp.mpf(1) / 2:   # curvature-regular = Friedrichs labels (fractional parts) when alpha<=1/2
    labels['b+2'] = 2 * al - mp.floor(2 * al); labels['b-2'] = (1 - 2 * al) - mp.floor(1 - 2 * al)
else:                     # also record the componentwise-Friedrichs label for comparison
    labels['b+2_F'] = 2 * al - 1; labels['b-2_F'] = 2 - 2 * al
r0 = mp.mpf('0.9'); th0 = mp.mpf('0.15')
x = (mp.mpf('0.1'), r0 * mp.cos(th0), r0 * mp.sin(th0), mp.mpf('-0.05'))
def Wf(x, y): return 1 / (4 * mp.pi**2 * sum((a - b)**2 for a, b in zip(x, y)))
def B2(b): return b * b - b + mp.mpf(1) / 6
dirs = [(1, 0, 0, 0), (0, 1, 0, 0), (0, 0, 1, 0), (0, 0, 0, 1), (1, 2, -1, 3), (-2, 1, 3, 1), (1, -1, -1, 2)]
dirs = [tuple(mp.mpf(c) / mp.sqrt(sum(mp.mpf(d)**2 for d in v)) for c in v) for v in dirs]
out = {'alpha': str(al), 'x': [mp.nstr(c, 8) for c in x], 'labels': {}}
for name, b in labels.items():
    R = lambda y: (Gb(b, al, x, y) - Wf(x, y)).real
    pred = (B2(b) / al**2 - mp.mpf(1) / 6) / (8 * mp.pi**2) / r0**2
    rec = {'b': mp.nstr(b, 12), 'pred_R(x,x)': mp.nstr(pred, 15)}
    # (a)+(b): value and directional derivative from symmetric differences at two scales
    vals = []; grads = []
    for n in dirs:
        h = mp.mpf('1e-6')
        yp = tuple(a + h * c for a, c in zip(x, n)); ym = tuple(a - h * c for a, c in zip(x, n))
        Rp, Rm = R(yp), R(ym)
        vals.append((Rp + Rm) / 2); grads.append((Rp - Rm) / (2 * h))
    rec['R0_spread_over_dirs'] = mp.nstr(max(vals) - min(vals), 5)
    rec['R0_relerr_vs_pred'] = mp.nstr(abs(sum(vals) / len(vals) - pred) / abs(pred) if pred != 0 else abs(sum(vals) / len(vals)), 5)
    g = grads[:4]   # axis gradient
    lin = [abs(grads[k] - sum(g[i] * dirs[k][i] for i in range(4))) for k in range(4, len(dirs))]
    rec['grad_linearity_residual_max'] = mp.nstr(max(lin), 5)
    rec['grad_norm'] = mp.nstr(mp.sqrt(sum(v * v for v in g)), 8)
    # (c) bubble log coefficient along the random directions
```

```
    J = []
    for rho in [mp.mpf('1e-1'), mp.mpf('1e-2'), mp.mpf('1e-3')]:
        row = []
        for n in dirs[4:]:
            y = tuple(a + rho * c for a, c in zip(x, n))
            G = Gb(b, al, x, y).real; Gf = Wf(x, y)
            Jc = 2 * mp.pi**2 * rho**4 * G * G; Jf = 2 * mp.pi**2 * rho**4 * Gf * Gf
            row.append({'J_flat': mp.nstr(Jf, 12), 'J_cone-1/(8pi^2)': mp.nstr(Jc - 1 / (8 * mp.pi**2), 6),
                        '(Jc-Jf)/rho^2 / R0': mp.nstr((Jc - Jf) / rho**2 / pred, 8) if pred != 0 else mp.nstr((Jc - Jf) / rho**2, 6)})
        J.append({'rho': mp.nstr(rho, 3), 'dirs': row})
    rec['bubble'] = J
    out['labels'][name] = rec
    print(name, rec['R0_relerr_vs_pred'], rec['R0_spread_over_dirs'], rec['grad_linearity_residual_max'], flush=True)
json.dump(out, open('/workspace/XVI-01-CONE/logs/locality_%s.json' % mp.nstr(al, 6), 'w'), indent=1)
```

2 Flat-space controls

2.1 `flat_count.py`

SHA-256: `069a9c8b1efb78db20311f8663b5de5ea02391b3dd53229fabe98bce42df30e4`

```
"""(A1) Flat-bulk sanity count inside the apex-engine machinery: same slot/pairing enumerator (apex_jets.pairings),
same modular rank routine, index range D=4 (no transverse jets; all directions 'longitudinal'), power counting of the
bulk kappa^2 d^4x two-point counterterm: [F1]+[F2]+k = 6.  Expected XVI-P p14r domain: h.h 5, c.cb 2, c.K 3, phi.phi 1."""
import sys, random, itertools, json
sys.path.insert(0, '/workspace/XVI-01-CONE/code')
import apex_jets as AJ
from apex_jets import pairings, rank_and_basis
D = 4
FL = {'h': (2, 1, True), 'K': (2, 2, True), 'c': (1, 1, False), 'cb': (1, 1, False), 'phi': (0, 1, False)}  # rank, dim, symmetric
def cands(F1, F2, T=6):
    r1, d1, _ = FL[F1]; r2, d2, _ = FL[F2]; k = T - d1 - d2
    slots = [('A', i) for i in range(r1)] + [('B', i) for i in range(r2)] + [('p', i) for i in range(k)]
    if len(slots) % 2: return [], k
    seen = set(); out = []
    for pr in pairings(slots):
        key = tuple(sorted(tuple(sorted((x[0], x[1] if x[0] != 'p' else 0) for x in pair)) for pair in pr))
        if key in seen: continue
        seen.add(key); out.append(pr)
    return out, k
def rt(F):
    r, _, sym = FL[F]; P = AJ.P
    t = {idx: random.randrange(P) for idx in itertools.product(range(D), repeat=r)}
    if sym and r == 2:
        for a in range(D):
            for b in range(a): t[(a, b)] = t[(b, a)]
    return t
def ev(pr, F1, F2, t1, t2, q):
    r1 = FL[F1][0]; r2 = FL[F2][0]; P = AJ.P
    k = sum(1 for pair in pr for x in pair if x[0] == 'p')
    tot = 0
    for idx in itertools.product(range(D), repeat=r1 + r2 + k):
        lab = {('A', i): idx[i] for i in range(r1)}; lab.update({('B', i): idx[r1 + i] for i in range(r2)})
        lab.update({('p', i): idx[r1 + r2 + i] for i in range(k)})
        if any(lab[x] != lab[y] for x, y in pr): continue
        v = t1[tuple(idx[:r1])] * t2[tuple(idx[r1:r1 + r2])] % P
        for i in range(k): v = v * q[idx[r1 + r2 + i]] % P
        tot = (tot + v) % P
    return tot
res = {}
for pi, p in enumerate(AJ.PRIMES):
    AJ.setprime(p); random.seed(7 + pi); P = AJ.P
    for name, (F1, F2, bose) in {'h.h': ('h', 'h', True), 'c.cb': ('cb', 'c', False), 'c.K': ('K', 'c', False), 'phi.phi': ('phi', '
     phi', True)}.items():
        cl, k = cands(F1, F2)
        smp = [(rt(F1), rt(F2), [random.randrange(P) for _ in range(D)]) for _ in range(len(cl) + 8)]
        rows = []
        for pr in cl:
            row = []
            for (a, b, q) in smp:
                v = ev(pr, F1, F2, a, b, q)
                if bose: v = (v + ev(pr, F1, F2, b, a, [(-x) % P for x in q])) % P
                row.append(v)
            rows.append(row)
        res.setdefault(name, []).append({'raw_patterns': len(cl), 'k': k, 'rank': rank_and_basis(rows)[0]})
print(json.dumps(res))
json.dump(res, open('/workspace/XVI-01-CONE/logs/v2_flat_count.json', 'w'), indent=1)
```

2.2 flat_pole.py

SHA-256: 7d77f198d4cd6874c6a88f8bff55483597b22ed74be4dc66bc7a2bbfd9423dc2

```
"""v4 item (1): spectral apex reduction for one-loop kappa^2 two-point graphs on the cone, constant-profile
(n=0, m=0) external fields with longitudinal momentum p.
Hypotheses: (H_I) intertwining of transverse derivatives between helicity bundles on CR; then the apex part of
Tr_perp f(-Lap_b) = C_b f(0) with C_b = (1/12)(1/a - a) - b(1-b)/(2a)  (scale-invariant heat-trace constant).
Hence: apex pole = sum_S C_{b(S)} x [UV pole of the (d-2)-dim LONGITUDINAL loop integral of the flat XVI-P integrand
at k_perp = 0], each internal line weighted by the helicity class of its propagator components.
Graph conventions copied (read-only) from XVI-P amplitude_rules.build_integrands and XVI-Q Q02 bulk_graphs (D04, D06),
D05a (Q02 symmetry factor 1/2).  i Gamma = pref * int d^dl/(2pi)^d N/dens  ->  Gamma = -i pref int ...
Wick: l0 = i y0, l3 = y3;  int d^Dl_M/(2pi)^D f  pole  = i/(8 pi^2 eps) int_0^{2pi} g(theta) dtheta,  eps=(4-d)/2,
where g = lim R^2 * [degree -2 part of f].  Output: coefficient of 1/(4-d) of Gamma_{ab}(p), per unit area of W.
usage: flat_pole.py <graph>  (v6: FLAT 4D bulk UV pole, same graph builder; validation of ST conventions)"""
import sys, json, time, itertools
sys.path.insert(0, '/tmp/o1cone_ro/XVI-P/code/p5_entry')
import sympy as sp
import amplitude_rules as AR
F = AR.F
I = sp.I
g = sys.argv[1]; br = 'UNIT'
T0 = time.time()
LS, PS = sp.symbols('l0:4'), sp.symbols('p0:4')
NAMES = {**{r: LS[r] for r in range(4)}, **{4 + r: PS[r] for r in range(4)}}
kap = F.KAPPA
al = sp.Symbol('alpha', positive=True)
u = sp.symbols('u0:3'); v = sp.symbols('v0:3')
def sym(poly): return sp.expand(poly.to_sympy(NAMES)) if hasattr(poly, 'to_sympy') else sp.expand(poly)
HC = list(F.HC)
def ctype(i):
    m, n = HC[i]; L = (m in (0, 3)) + (n in (0, 3))
    return {2: 'L', 1: 'M', 0: 'T'}[L]
def cls(i, j):
    a, b = ctype(i), ctype(j)
    if a == b == 'M': return 1
    if a == b == 'T': return 2
    return 0
Pi = AR.pinv_sparse(); Gh = AR.ghost_num()
def Piw(i, j, mk): return sp.Rational(Pi[(i, j)].numerator, Pi[(i, j)].denominator) * mk[cls(i, j)]
def Ghw(i, j, mk): return sp.Rational(Gh[(i, j)].numerator, Gh[(i, j)].denominator) * mk[0 if i in (0, 3) else 1]
V = F.vertices()
def route(vt, r): return {k: sym(p) for k, p in F.vt_route(vt, r).items()}
def C_of(b): return sp.Rational(1, 12) * (1 / al - al) - b * (1 - b) / (2 * al)
def lab(S):
    if br == 'CR': return S * al if S >= 0 else 1 + S * al
    return {0: 0, 1: al, 2: 2 * al - 1}[S]       # F, 1/2 < alpha < 1 (|S| only; C symmetric under b -> 1-b)
CS = [sp.Integer(1)]*3 if br == "UNIT" else [C_of(lab(S)) for S in range(3)]
N = {}; dens = None
if g == 'D01':
    V3 = V['V3_grav']; VA = route(V3, [(0, 1), (-1, 0), (1, -1)]); VB = route(V3, [(0, -1), (1, 0), (-1, 1)])
    for (a, c, e), pa in VA.items():
        for (b, d, f), pb in VB.items():
            if (c, d) in Pi and (e, f) in Pi:
                N[(a, b)] = N.get((a, b), 0) + pa * pb * Piw(c, d, u) * Piw(e, f, v)
    pref = AR.base_rules()['graphs']['D01']['S_f'] * (I * kap) ** 2 * I ** (2 * V3.phase) * I ** 2; dens = 'll-p'
elif g == 'D03':
    Vg = V['V_cb_c_h']; VA = route(Vg, [(-1, 0), (1, -1), (0, 1)]); VB = route(Vg, [(-1, 1), (1, 0), (0, -1)])
    for (rho, mu, a), pa in VA.items():
        for (s, nu, b), pb in VB.items():
            if (mu, s) in Gh and (nu, rho) in Gh:
                N[(a, b)] = N.get((a, b), 0) + pa * pb * Ghw(mu, s, u) * Ghw(nu, rho, v)
    pref = -1 * (I * kap) ** 2 * I ** (2 * Vg.phase); dens = 'll-p'
elif g == 'D11':
    Vs = V['V_h_phi_phi']; VA = route(Vs, [(0, 1), (-1, 0), (1, -1)]); VB = route(Vs, [(0, -1), (1, 0), (-1, 1)])
    for (a, _, _), pa in VA.items():
        for (b, _, _), pb in VB.items():
            N[(a, b)] = N.get((a, b), 0) + pa * pb * u[0] * v[0]
    pref = sp.Rational(1, 2) * (I * kap) ** 2 * I ** (2 * Vs.phase) * I ** 2; dens = 'll-p'
elif g == 'D04':   # Q02: VA h_c:-l, phi_ext:p, phi:l-p ; VB h_d:l, phi_ext:-p, phi:p-l ; graviton line l, scalar line l-p
    Vs = V['V_h_phi_phi']; VA = route(Vs, [(-1, 0), (0, 1), (1, -1)]); VB = route(Vs, [(1, 0), (0, -1), (-1, 1)])
    for (c, _, _), pa in VA.items():
        for (d, _, _), pb in VB.items():
            if (c, d) in Pi:
                N[(0, 0)] = N.get((0, 0), 0) + pa * pb * Piw(c, d, u) * v[0]
    pref = 1 * (I * kap) ** 2 * I ** (2 * Vs.phase) * I ** 2; dens = 'll-p'
elif g == 'D06':   # Q02: VA cbar_rho:-p, c^mu:l, h_a:p-l ; VB cbar_sig:-l, c^nu:p, h_b:l-p ; ghost line l, graviton l-p
    Vg = V['V_cb_c_h']; VA = route(Vg, [(0, -1), (1, 0), (-1, 1)]); VB = route(Vg, [(-1, 0), (0, 1), (1, -1)])
    for (rho, mu, a), pa in VA.items():
        for (sg, nu, b), pb in VB.items():
            if (mu, sg) in Gh and (a, b) in Pi:
                N[(rho, nu)] = N.get((rho, nu), 0) + pa * pb * Ghw(mu, sg, u) * Piw(a, b, v)
    pref = 1 * (I * kap) ** 2 * I ** (2 * Vg.phase) * I ** 1; dens = 'll-p'
elif g == 'D13':
```

```
    rules = AR.base_rules(); VK = AR.K_vertex(rules); Vg = V['V_cb_c_h']
    VA = route(VK, [(0, -1), (1, 0), (-1, 1)]); VB = route(Vg, [(-1, 0), (0, 1), (1, -1)])
    for (ap, mu, c), pa in VA.items():
        for (rho, nu, d), pb in VB.items():
            if (mu, rho) in Gh and (c, d) in Pi:
                N[(ap, nu)] = N.get((ap, nu), 0) + pa * pb * Ghw(mu, rho, u) * Piw(c, d, v)
    R = rules['graphs']['D13']
    pref = R['ext_sign'] * R['S_f'] * R['loop_sign'] * (I * kap) ** 2 * I ** (VK.phase + Vg.phase) * I; dens = 'll-p'
elif g == 'D02':
    V4 = V['V4_grav']; VA = route(V4, [(0, 1), (0, -1), (1, 0), (-1, 0)])
    for (a, b, c, d), pa in VA.items():
        if (c, d) in Pi: N[(a, b)] = N.get((a, b), 0) + pa * Piw(c, d, u) * v[0]
    pref = sp.Rational(1, 2) * (I * kap ** 2) * I ** V4.phase * I; dens = 'l'
elif g == 'D12':
    V2s = V['V_hh_phi_phi']; VA = route(V2s, [(0, 1), (0, -1), (1, 0), (-1, 0)])
    for (a, b, _, _), pa in VA.items(): N[(a, b)] = N.get((a, b), 0) + pa * u[0] * v[0]
    pref = sp.Rational(1, 2) * (I * kap ** 2) * I ** V2s.phase * I; dens = 'l'
elif g == 'D05a':
    V2s = V['V_hh_phi_phi']; VB = route(V2s, [(1, 0), (-1, 0), (0, 1), (0, -1)])
    for (c, d, _, _), pa in VB.items():
        if (c, d) in Pi: N[(0, 0)] = N.get((0, 0), 0) + pa * Piw(c, d, u) * v[0]
    pref = sp.Rational(1, 2) * (I * kap ** 2) * I ** V2s.phase * I; dens = 'l'

print('built', g, len(N), round(time.time() - T0, 1), flush=True)
q0, q3 = sp.symbols('q0 q3')
L = LS
sub = {PS[1]: 0, PS[2]: 0, PS[0]: q0, PS[3]: q3}
ETA = [1, -1, -1, -1]
pv = [q0, 0, 0, q3]
lp = sum(ETA[r] * L[r] * pv[r] for r in range(4)); p2 = q0**2 - q3**2
def dfact(n):
    r = 1
    while n > 1: r *= n; n -= 2
    return r
def avg(mon):   # <prod l_r^{n_r}> / (l^2)^k, d=4 Minkowski
    if any(n % 2 for n in mon): return 0
    k = sum(mon) // 2
    c = sp.Integer(1)
    for r, n in enumerate(mon): c *= dfact(n - 1) * ETA[r] ** (n // 2)
    return c / (2**k * sp.factorial(k + 1))
def pole4(expr):
    """coefficient of 1/(4-d) of int d^dl_M/(2pi)^d expr/den  (den = l^2 or l^2 (l-p)^2), UV only: i/(8 pi^2) x <deg -4 part> (l^2)^2"
     ""
    e = sp.expand(expr.subs(sub))
    if e == 0: return 0
    P = sp.Poly(e, *L)
    tot = 0
    for mon, cf in zip(P.monoms(), P.coeffs()):
        kdeg = sum(mon)
        if dens == 'l':
            # mon/l^2: degree kdeg-2 == -4 impossible for kdeg>=0
            continue
        # 1/(l-p)^2 = sum_n (2 l.p - p^2)^n/(l^2)^{n+1}; need kdeg + j - 2n - 4 = -4, j = #(2l.p) factors
        for n in range(0, kdeg + 1):
            j = 2 * n - kdeg
            if j < 0 or j > n: continue
            term = sp.binomial(n, j) * (2 * lp) ** j * (-p2) ** (n - j)
            Q = sp.Poly(sp.expand(term * sp.Mul(*[L[r] ** mon[r] for r in range(4)])), *L)
            for m2, c2 in zip(Q.monoms(), Q.coeffs()):
                tot += cf * c2 * avg(m2)
    return sp.expand(I / (8 * sp.pi**2) * tot)
res = {}
for k, e in N.items():
    e = sp.expand(e.subs({**{x: 1 for x in u}, **{x: 1 for x in v}}))
    if e == 0: continue
    val = sp.factor(sp.simplify(-I * pref * pole4(e)))
    if val != 0: res[str(k)] = str(val)
out = {'graph': g, 'branch': 'FLAT4D', 'pref': str(pref), 'Gamma_pole_coeff_of_1/(4-d)': res,
       'momentum': 'p=(q0,0,0,q3), mostly-minus', 'seconds': round(time.time() - T0, 1)}
import os; os.makedirs('/workspace/XVI-01-CONE/logs/v6', exist_ok=True)
json.dump(out, open('/workspace/XVI-01-CONE/logs/v6/flat_%s.json' % g, 'w'), indent=1)
print(json.dumps(out)[:1500])
```

2.3 flat_vs_p18.py

SHA-256: 4bee961081e64fa4ca7de67baa92bd515bbd3307acaf317390c9fa5c68912561

```
"""v6: validation of the flat 4D pole route (flat_pole.py) against the frozen XVI-P certificate P18_D11 (read-only):
the full lattice array must be proportional (one overall constant) to the 4D (non-evanescent) covariant P18 tensor."""
import json, itertools, sympy as sp
q0, q3, ka = sp.symbols('q0 q3 kappa')
d = json.load(open('/tmp/o1cone_ro/XVI-Q/provenance/XVI-P/certificates/P18_D11.json'))['result']
```

```
c = {k: sp.sympify(v['UV_pole']) for k, v in d.items() if not v['evanescent']}
ETA = [1, -1, -1, -1]; HC = [(0,0),(0,1),(0,2),(0,3),(1,1),(1,2),(1,3),(2,2),(2,3),(3,3)]
pl = [q0, 0, 0, q3]
def T(m, n, r, s, sym, up):
    g = lambda a, b: (ETA[a] if a == b else 0)
    p = (lambda a: ETA[a]*pl[a]) if up else (lambda a: pl[a])
    pp = q0**2 - q3**2
    def S2(f): return (f(m, n, r, s) + f(n, m, r, s)) / 2 if sym else f(m, n, r, s)
    v = c['g4_mn g4_rs (p.p)^2'] * g(m, n) * g(r, s) * pp**2
    v += c['g4_mr g4_ns (p.p)^2'] * S2(lambda m, n, r, s: g(m, r) * g(n, s)) * pp**2 * (2 if sym else 1)
    v += c['g4_mn p_r p_s (p.p)^1'] * g(m, n) * p(r) * p(s) * pp + c['g4_rs p_m p_n (p.p)^1'] * g(r, s) * p(m) * p(n) * pp
    v += c['g4_mr p_n p_s (p.p)^1'] * S2(lambda m, n, r, s: (g(m, r) * p(n) * p(s) + g(n, s) * p(m) * p(r)) / 2) * pp * (2 if sym else
      1)
    v += c['p_m p_n p_r p_s'] * p(m) * p(n) * p(r) * p(s)
    return sp.expand(v)
mine = {eval(k): sp.sympify(v) for k, v in json.load(open('/workspace/XVI-01-CONE/logs/v6/flat_D11.json'))['Gamma_pole_coeff_of_1/(4-d
    )'].items()}
LAT = [0, 3, 9, 4, 7]
best = None
for sym, up, w in itertools.product([True, False], [True, False], [1, 2]):
    K = None; ok = True
    for a in LAT:
        for b in LAT:
            m, n = HC[a]; r, s = HC[b]
            wt = (w if m != n else 1) * (w if r != s else 1)
            t = T(m, n, r, s, sym, up) * wt; v = mine.get((a, b), 0)
            if t == 0 and v == 0: continue
            if t == 0 or v == 0: ok = False; break
            ratio = sp.simplify(v / t)
            if K is None: K = ratio
            elif sp.simplify(ratio - K) != 0: ok = False; break
        if not ok: break
    print('sym', sym, 'up', up, 'offdiag w', w, '->', 'PROPORTIONAL, K =' if ok else 'no', K if ok else '')
    if ok: best = (sym, up, w, str(K))
json.dump({'match': best}, open('/workspace/XVI-01-CONE/logs/v6/flat_vs_p18.json', 'w'), indent=1)
# general fully-symmetrised ansatz fit (normalisation-free ratio comparison)
A = sp.symbols('A1:6')
def Tg(m, n, r, s, up, A):
    g = lambda a, b: (ETA[a] if a == b else 0)
    p = (lambda a: ETA[a]*pl[a]) if up else (lambda a: pl[a]); pp = q0**2 - q3**2
    return (A[0]*g(m,n)*g(r,s)*pp**2 + A[1]*(g(m,r)*g(n,s)+g(m,s)*g(n,r))*pp**2 + A[2]*(g(m,n)*p(r)*p(s)+g(r,s)*p(m)*p(n))*pp
            + A[3]*(g(m,r)*p(n)*p(s)+g(m,s)*p(n)*p(r)+g(n,r)*p(m)*p(s)+g(n,s)*p(m)*p(r))*pp + A[4]*p(m)*p(n)*p(r)*p(s))
res = {}
for up, w in itertools.product([True, False], [1, 2, sp.Rational(1, 2)]):
    eqs = []
    for a in LAT:
        for b in LAT:
            m, n = HC[a]; r, s = HC[b]
            wt = (w if m != n else 1) * (w if r != s else 1)
            eqs += sp.Poly(sp.expand(Tg(m, n, r, s, up, A) * wt - mine.get((a, b), 0)), q0, q3).coeffs()
    sol = sp.solve(eqs, A, dict=True)
    if sol:
        s0 = sol[0]; norm = s0[A[1]]
        rat = [sp.nsimplify(s0[x] / norm) for x in A]
        print('up', up, 'w', w, 'fit ratios A/A2 =', rat, ' (P18 ratios a/a2 = [6, 1, -6, -1, 8])')
        res[f'up={up},w={w}'] = [str(x) for x in rat]
json.dump({'match': best, 'ansatz_fits': res, 'P18_ratios_over_a2': [6, 1, -6, -1, 8]}, open('/workspace/XVI-01-CONE/logs/v6/
    flat_vs_p18.json', 'w'), indent=1)
```

3 Kernels, coincidence limits and heat coefficients

3.1 `kernel.py`

SHA-256: 4678e4bcc963c7368ac8c5156915d0c2f651d5aefae348cdaba309e762461b06

```
"""Exact twisted Friedrichs / geometric-branch kernels on the cone (Euclidean, massless, 4D).
G_b(x,x') = S_b(eta/alpha, psi) / (8 pi^2 alpha r r' sinh eta),
S_b(u,psi) = f_b(u - i psi) + f_{1-b}(u + i psi),  f_b(z) = e^{-b z}/(1-e^{-z}).
For 0<=b<1 this is the Friedrichs mode sum sum_n e^{i(n+b)psi} e^{-|n+b|u};
for b outside [0,1) it is the 'geometric branch' (E0-type flips in the modes with sign change)."""
import mpmath as mp
def f(b,z): return mp.exp(-b*z)/(1-mp.exp(-z))
def S(b,u,psi): return f(b,u-1j*psi)+f(1-b,u+1j*psi)
def S_modes(b,u,psi,N=4000):
    # Friedrichs mode sum, b reduced mod 1
    b=b-mp.floor(b)
    return mp.nsum(lambda n: mp.exp(1j*(n+b)*psi-abs(n+b)*u),[-mp.inf,mp.inf])
def G(b,alpha,x,xp):
    """x=(tau,X,Y,z) developed Cartesian coordinates (lift: angle = atan2/alpha, small sector)"""
    t,X,Y,z=x; tp,Xp,Yp,zp=xp
```

```
    r=mp.sqrt(X*X+Y*Y); rp=mp.sqrt(Xp*Xp+Yp*Yp)
    th=mp.atan2(Y,X); thp=mp.atan2(Yp,Xp)
    psi=(thp-th)/alpha          # lift angle difference x -> x'
    ch=(r*r+rp*rp+(t-tp)**2+(z-zp)**2)/(2*r*rp)
    eta=mp.acosh(ch)
    return S(b,eta/alpha,psi)/(8*mp.pi**2*alpha*r*rp*mp.sinh(eta))
def Gflat(x,xp):
    return 1/(4*mp.pi**2*sum((a-b)**2 for a,b in zip(x,xp)))
```

3.2 tensor.py

SHA-256: a28aea0bb3bd82d37b34cc62ad86954ea99286c52c729339bca9ecaf1ac9f262

```
"""Helicity-projected Feynman / de Donder kernels on the cone (Euclidean, developed Cartesian frame (tau,X,Y,z))."""
import mpmath as mp, itertools, sys; sys.path.insert(0,'/workspace/XVI-01-CONE/code')
from kernel import S, f
def Gb(b,alpha,x,xp,flip=None):
    t,X,Y,z=x; tp,Xp,Yp,zp=xp
    r=mp.sqrt(X*X+Y*Y); rp=mp.sqrt(Xp*Xp+Yp*Yp)
    psi=(mp.atan2(Yp,Xp)-mp.atan2(Y,X))/alpha
    eta=mp.acosh((r*r+rp*rp+(t-tp)**2+(z-zp)**2)/(2*r*rp))
    u=eta/alpha; s=S(b,u,psi)
    if flip is not None:   # flip mode with angular exponent m=n+b (E0 in that channel)
        m=flip; s+= mp.exp(1j*m*psi)*(mp.exp(abs(m)*u)-mp.exp(-abs(m)*u))
    return s/(8*mp.pi**2*alpha*r*rp*mp.sinh(eta))
sq=1/mp.sqrt(2)
U={'t':(mp.mpf(1),0,0,0),'p':(0,sq,1j*sq,0),'m':(0,sq,-1j*sq,0),'z':(0,0,0,mp.mpf(1))}
HEL={'t':0,'p':1,'m':-1,'z':0}
def bgeom(s,alpha): return s*alpha if s>=0 else 1+s*alpha
def bfried(s,alpha):
    b=s*alpha; return b-mp.floor(b)
def vec_kernel(alpha,x,xp,branch=bgeom,flips={}):
    """W[a][b] = sum_A u_A u_A^dagger G_{b(s_A)}"""
    cache={}
    def g(s):
        if s not in cache: cache[s]=Gb(branch(s,alpha),alpha,x,xp,flips.get(s))
        return cache[s]
    W=[[0]*4 for _ in range(4)]
    for A,u in U.items():
        gs=g(HEL[A])
        for a in range(4):
            for b in range(4):
                W[a][b]+=u[a]*mp.conj(u[b])*gs
    return W
def P(a,b,c,d): return (a==c)*(b==d)+(a==d)*(b==c)-(a==b)*(c==d)
def ten_kernel(alpha,x,xp,branch=bgeom,flips={}):
    """W[(a,b)][(c,d)] = sum_{A,B} (u_A x u_B)(u_A x u_B)^dagger P G_{b(s_A+s_B)} ; P commutes with helicity projectors"""
    cache={}
    def g(s):
        if s not in cache: cache[s]=Gb(branch(s,alpha),alpha,x,xp,flips.get(s))
        return cache[s]
    idx=list(itertools.product(range(4),repeat=2))
    # projector-weighted kernel K[(a,b)][(e,f)] = sum_{AB} uA_a uB_b conj(uA_e uB_f) g(sA+sB)
    K={}
    for I in idx:
        for J in idx:
            K[(I,J)]=0
    for A,uA in U.items():
        for B,uB in U.items():
            gs=g(HEL[A]+HEL[B])
            for I in idx:
                vI=uA[I[0]]*uB[I[1]]
                if vI==0: continue
                for J in idx:
                    vJ=mp.conj(uA[J[0]]*uB[J[1]])
                    if vJ!=0: K[(I,J)]+=vI*vJ*gs
    W={}
    for I in idx:
        for J in idx:
            W[(I,J)]=sum(K[(I,L)]*P(L[0],L[1],J[0],J[1]) for L in idx)
    return W
```

3.3 coinc.py

SHA-256: 16d74a50ebdc5257f882b571ad88c60285fa9af31959e5d689f926eb5b02995b

```
"""Exact coincidence (Synge) data of the subtracted twisted cone kernel R_b = G_b - G_flat, from the Bernoulli
generating function  f_b(z) = (1/z) sum_n B_n(1-b) z^n/n!  (independent of the closed-form numerics in kernel.py).
R_b(x,x') = Phi_b(eta, eps) / (8 pi^2 r r'),  eps = developed angle difference theta'-theta,
```

```
cosh eta = (r^2 + r'^2 + s_t dt^2 + dz^2)/(2 r r'),  s_t=+1 Euclidean, s_t=-1 Lorentzian (time-ordered = continued G_E).
Returns A(x), B_i(x), C_ij(x) with R(x, x+d) = A + B.d + d.C.d + O(d^3), in coordinates (t, X, Y, z)."""
import sympy as sp
b, al = sp.symbols('b alpha', positive=True)
eta, eps, rho, ph = sp.symbols('eta epsilon rho phi')
def tail(bb, aa, order):
    t = 0
    for n in range(2, order + 3):
        Bn = sp.bernoulli(n, bb)
        t += Bn / (sp.factorial(n) * aa ** n) * ((-1) ** n * (eta - sp.I * eps) ** (n - 1) + (eta + sp.I * eps) ** (n - 1))
    return sp.expand(sp.expand(t / eta) * sp.series(eta / sp.sinh(eta), eta, 0, order + 1).removeO())
def Phi_series(order=2):
    """Phi_b = T(b,alpha) - T(0,1), T(b,alpha) = sum_{n>=2} B_n(b)/(n! alpha^n) [(-1)^n (eta-i eps)^{n-1} + (eta+i eps)^{n-1}]/sinh
     eta.
    Exact because the n=0 (Hadamard) term is alpha,b-independent in (eta,eps) and Phi_{b=0,alpha=1} = 0 identically;
    the n=1 term vanishes.  Truncated at total degree <= order in (eta, eps)."""
    tot = sp.expand(tail(b, al, order) - tail(0, 1, order))
    out = 0
    for term in sp.Add.make_args(tot):
        if sp.Poly(term, eta, eps).total_degree() <= order: out += term
    return sp.expand(out)
def synge(Phi, lorentz=True):
    t, X, Y, z, lam = sp.symbols('t X Y z lam')
    d = sp.symbols('d0:4')
    r = sp.sqrt(X ** 2 + Y ** 2); Xp = X + lam * d[1]; Yp = Y + lam * d[2]
    rp = sp.sqrt(Xp ** 2 + Yp ** 2)
    st = -1 if lorentz else 1
    w = ((r - rp) ** 2 + st * (lam * d[0]) ** 2 + (lam * d[3]) ** 2) / (2 * r * rp)   # cosh(eta) - 1
    eta2 = 2 * w - w ** 2 / 3
    ep = sp.atan((X * Yp - Y * Xp) / (X * Xp + Y * Yp))
    E2, EP = sp.symbols('E2 EP')
    # Phi is polynomial in eta (even) and eps: substitute eta^2 -> E2
    P2 = sp.expand(Phi).subs(eta ** 2, E2)
    assert not P2.has(eta), P2
    expr = P2.subs({E2: eta2, eps: ep}) / (8 * sp.pi ** 2 * r * rp)
    ser = sp.series(expr, lam, 0, 3).removeO()
    ser = sp.expand(ser)
    A = sp.simplify(ser.coeff(lam, 0)); B1 = ser.coeff(lam, 1); C2 = ser.coeff(lam, 2)
    Bv = [sp.simplify(sp.diff(B1, d[i])) for i in range(4)]
    Cm = [[sp.simplify(sp.diff(C2, d[i], d[j]) / (1 if i == j else 1) / 2) for j in range(4)] for i in range(4)]
    # C2 = sum_ij Cm_ij d_i d_j with Cm symmetric:  d^2/dd_i dd_j C2 = 2 Cm_ij
    return (t, X, Y, z), A, Bv, Cm
if __name__ == '__main__':
    import pickle, time
    T0 = time.time()
    Ph = Phi_series(2)
    print('Phi_b to order 2:', Ph, round(time.time() - T0, 1), flush=True)
    for lor in (True, False):
        co, A, Bv, Cm = synge(Ph, lor)
        print('lorentz', lor, 'A=', sp.simplify(A), flush=True)
        pickle.dump({'Phi': Ph, 'coords': co, 'A': A, 'B': Bv, 'C': Cm}, open('/workspace/XVI-01-CONE/logs/v2_coinc_%s.pkl' % ('L' if
     lor else 'E'), 'wb'))
    print('done', round(time.time() - T0, 1))
```

3.4 xcheck_coinc.py

SHA-256: 03d2a6baae59073ce543fc66ff831ce76bd4afc529c82fb380fcf9650b31c9f1

```
"""(F) Independent cross-check of the exact Synge data (Bernoulli route, coinc.py) against
 (i) the closed-form kernel (kernel.py S_b) and (ii) a direct angular MODE SUM (with the CR E0 mode flipped explicitly),
evaluated at finite separations; residual must scale like lambda^3."""
import pickle, sys, json
import mpmath as mp, sympy as sp
sys.path.insert(0, '/workspace/XVI-01-CONE/code')
mp.mp.dps = 30
from kernel import S as S_closed
D = pickle.load(open('/workspace/XVI-01-CONE/logs/v2_coinc_E.pkl', 'rb'))
t, X, Y, z = D['coords']; bs, als = sp.symbols('b alpha', positive=True)
d = sp.symbols('d0:4')
def S_modes(b, u, psi):
    f = lambda n: mp.exp(1j * (n + b) * psi - abs(n + b) * u)
    s = mp.nsum(f, [-mp.inf, mp.inf])
    # explicit E0 flips: geometric (CR) branch for b>1 flips exponent b-1; for b<0 flips exponent b
    if b > 1: s += mp.exp(1j * (b - 1) * psi) * (mp.exp((b - 1) * u) - mp.exp(-(b - 1) * u))
    if b < 0: s += mp.exp(1j * b * psi) * (mp.exp(-b * u) - mp.exp(b * u))
    return s
def Rnum(b, al, x, xp, how):
    tt, X1, Y1, zz = x; tp, X2, Y2, zp = xp
    r = mp.sqrt(X1 ** 2 + Y1 ** 2); rp = mp.sqrt(X2 ** 2 + Y2 ** 2)
    ep = mp.atan2(Y2, X2) - mp.atan2(Y1, X1); psi = ep / al
    ch = (r * r + rp * rp + (tt - tp) ** 2 + (zz - zp) ** 2) / (2 * r * rp); eta = mp.acosh(ch); u = eta / al
    Sv = S_closed(b, u, psi) if how == 'closed' else S_modes(b, u, psi)
```

```
    return Sv / (8 * mp.pi ** 2 * al * r * rp * mp.sinh(eta)) - 1 / (8 * mp.pi ** 2 * r * rp * (ch - mp.cos(ep)))
out = []
al = mp.mpf('0.70710678811865475244')
x0 = (mp.mpf(0), mp.mpf('1.0'), mp.mpf('0.3'), mp.mpf(0)); dv = (mp.mpf('0.3'), mp.mpf('-0.5'), mp.mpf('0.7'), mp.mpf('0.4'))
labels = {'0': 0, 'alpha': al, '1-alpha': 1 - al, '2alpha(CR)': 2 * al, '1-2alpha(CR)': 1 - 2 * al, '2alpha-1(F)': 2 * al - 1}
for name, bv in labels.items():
    sub = {bs: sp.Float(str(bv), 30), als: sp.Float(str(al), 30), t: 0, X: sp.Float('1.0', 30), Y: sp.Float('0.3', 30), z: 0}
    A = complex(sp.N(D['A'].subs(sub), 25)); Bv = [complex(sp.N(Bi.subs(sub), 25)) for Bi in D['B']]
    Cm = [[complex(sp.N(D['C'][i][j].subs(sub), 25)) for j in range(4)] for i in range(4)]
    row = {'label': name}
    for how in ('closed', 'modes'):
        res = []
        for lam in (mp.mpf('0.04'), mp.mpf('0.02'), mp.mpf('0.01')):
            xp = tuple(x0[i] + lam * dv[i] for i in range(4))
            pred = A + lam * sum(Bv[i] * float(dv[i]) for i in range(4)) + lam ** 2 * sum(Cm[i][j] * float(dv[i]) * float(dv[j]) for i
      in range(4) for j in range(4))
            res.append(float(abs(Rnum(bv, al, x0, xp, how) - pred)))
        row[how] = res; row[how + '_order'] = [mp.nstr(mp.log(res[0] / res[1], 2), 4), mp.nstr(mp.log(res[1] / res[2], 2), 4)]
    out.append(row); print(row, flush=True)
json.dump(out, open('/workspace/XVI-01-CONE/logs/v2_xcheck_coinc.json', 'w'), indent=1)
```

3.5 heatcoef.py

SHA-256: 5b6a313c0049b3e9f7763d0825ab5004a4298aad647987213eb2ba8575ba2467

```
# conical heat coefficient per label a(b) = (1/12)(1/al-al) - b(1-b)/(2 al), from r^2 R_b(x,x) (locality.py, verified 1e-14)
# Compare with Kabat (vector) and Fursaev-Miele eq.(3.47) at d=4 (spin 2), in units where scalar a0 = a(0).
from fractions import Fraction as F
import json
def a(b,al): return F(1,12)*(1/al-al) - b*(1-b)/(2*al)
out=[]
for al in [F(3,10),F(41,100),F(1,2),F(309,500),F(2,3),F(7071,10000),F(4,5),F(8501,10000)]:
    a0=a(0,al)
    vec=2*a0+a(al,al)+a(1-al,al)                  # Friedrichs vector (all alpha: nu1 channel only affects alpha>1/2 E0-vec, not used)
    kab=4*a0-(1-al)
    b2 = 2*al if al<=F(1,2) else 2*al-1           # Friedrichs label of helicity +2 jumps at alpha=1/2
    b2m= 1-2*al if al<=F(1,2) else 2-2*al
    tenF=4*a0+2*(a(al,al)+a(1-al,al))+a(b2,al)+a(b2m,al)
    tenCR=4*a0+2*(a(al,al)+a(1-al,al))+a(2*al,al)+a(1-2*al,al)
    fm=10*a0+6*(al-1)+2
    out.append({'alpha':str(al),'vec_minus_Kabat':str(vec-kab),'tenCR_minus_FM':str(tenCR-fm),'tenF_minus_FM':str(tenF-fm)})
    print(out[-1])
json.dump(out,open('logs/heatcoef.json','w'),indent=1)
```

3.6 weighted_apex.py

SHA-256: a8a3ffd0742a5fdc5d176d8b259d47011051513ff48191fcc22231e94cb2eada

```
"""v6 item (b): r^2-weighted apex heat coefficient D_b:  ApTr[r^2 e^{-sA_b}] = D_b * s  (Abel-subtracted), so that
ApTr[r^2 f(A_b)] = -D_b f'(0) for heat superpositions.  Also the singular-channel shift and orbifold image checks."""
import sympy as sp, json, mpmath as mp
u, al, b, nu = sp.symbols('u alpha b nu', positive=True)
S = sp.cosh((sp.Rational(1, 2) - b) * u / al) / sp.sinh(u / (2 * al))          # sum_n exp(-|n+b|u/alpha)
# int_0^inf z e^{-z cosh u} I_rho(z) dz = -(1/sinh u) d/du [e^{-rho u}/sinh u]; T2/s = sum_n of that
T2 = -(1 / sp.sinh(u)) * sp.diff(S / sp.sinh(u), u)
c = sp.cosh(u) - 1
bulk = al / c**2                                                  # int r^2 e^{-c r^2/2s} dV/(4 pi s) / s
ser = sp.series(T2 - bulk, u, 0, 2).removeO()
Db = sp.factor(sp.simplify(ser.coeff(u, 0)))
print('pole terms (must vanish):', [sp.simplify(ser.coeff(u, -k)) for k in (1, 2, 3, 4)])
print('D_b =', Db)
# singular channel shift: sum over one channel: -(1/sinh u) d/du [(e^{nu u}-e^{-nu u})/sinh u] -> u->0
sh = sp.limit(-(1 / sp.sinh(u)) * sp.diff((sp.exp(nu * u) - sp.exp(-nu * u)) / sp.sinh(u), u), u, 0)
print('flip shift of D (one channel):', sp.simplify(sh))
# orbifold check alpha = 1/N, b=0: images give sum_{k=1}^{N-1} 1/(4 N sin^4(pi k/N))
chk = {}
for N in [2, 3, 4, 5, 7]:
    img = sum(1 / (4 * N * mp.sin(mp.pi * k / N) ** 4) for k in range(1, N))
    val = Db.subs({al: sp.Rational(1, N), b: 0})
    chk[N] = (mp.nstr(img, 15), str(val), mp.nstr(mp.mpf(sp.N(val, 30)) - img, 5))
    print('alpha=1/%d image sum' % N, chk[N])
# twisted orbifold check: alpha=1/N, b=j/N  -> phases e^{2 pi i k b N/N}? holonomy: images weighted by e^{-2 pi i k b} (b = twist)
tchk = {}
for N, j in [(3, 1), (4, 1), (5, 2)]:
    bb = sp.Rational(j, N)
    img = sum(mp.cos(2 * mp.pi * k * j / N * N / N) / (4 * N * mp.sin(mp.pi * k / N) ** 4) for k in range(1, N))
    # holonomy of twisted channel b around full 2pi/alpha... image k carries phase exp(2 pi i k b / alpha * alpha)?? report both
    val = Db.subs({al: sp.Rational(1, N), b: bb})
    tchk[str((N, j))] = (mp.nstr(img, 12), str(val), mp.nstr(mp.mpf(sp.N(val, 30)), 12))
```

```
    print('twisted', N, j, tchk[str((N, j))])
json.dump({'D_b': str(Db), 'flip_shift': str(sp.simplify(sh)), 'orbifold_b0': {str(k): v for k, v in chk.items()}, 'twisted': tchk},
          open('/workspace/XVI-01-CONE/logs/v6/weighted_apex.json', 'w'), indent=1)
```

3.7 weighted_checks.py

SHA-256: a14d14e9112ab3603c4d77bbaed10cf1dc70e67357ac4678d116de89eb317834

```
"""v6: closed-form checks of the r^2-weighted apex coefficient D_b (logs/v6/weighted_apex.json)."""
import json, sympy as sp
a, b = sp.symbols('alpha b', positive=True)
B2 = b**2-b+sp.Rational(1,6); B4 = b**4-2*b**3+b**2-sp.Rational(1,30)
D = -11*a/180 + B2/(3*a) - B4/(6*a**3)
logD = sp.sympify(json.load(open('logs/v6/weighted_apex.json'))['D_b'], locals={'alpha': a, 'b': b})
out = {'D_matches_log': sp.simplify(D-logD) == 0}
out['D0'] = str(sp.factor(D.subs(b, 0)))
out['D_F_S1'] = str(sp.factor(D.subs(b, a)))
out['D_F_S2'] = str(sp.factor(D.subs(b, 2*a-1)))
out['D_CR_S2'] = str(sp.factor(D.subs(b, 2*a)))
nu = 2-1/a
out['CR_minus_F_S2_equals_flip'] = sp.simplify(D.subs(b, 2*a)-D.subs(b, 2*a-1)-2*nu*(1-nu**2)/3) == 0
out['D_CR_S2_at_1'] = str(sp.limit(D.subs(b, 2*a), a, 1))
out['D_F_S2_at_1'] = str(sp.limit(D.subs(b, 2*a-1), a, 1))
# orbifold identity  D_0(1/N) = (1/(4N)) sum_{k=1}^{N-1} csc^4(pi k/N)
chk = {}
for N in range(2, 9):
    s = sum(1/sp.sin(sp.pi*k/N)**4 for k in range(1, N))
    chk[N] = [str(sp.nsimplify(sp.N(s/(4*N), 40), rational=True)), str(D.subs({b: 0, a: sp.Rational(1, N)}))]
out['orbifold_csc4'] = chk
out['csc4_sum_closed_form'] = '(N^2-1)(N^2+11)/45'
print(json.dumps(out, indent=1)); json.dump(out, open('logs/v6/weighted_checks.json', 'w'), indent=1)
```

4 Tadpoles D02, D12, D05a and the hh structures

4.1 cone_tadpole.py

SHA-256: 8a4134732528952ad4ab100762002a6efb42504fc1496abbc21c0430b2f1492c

```
"""(C,D) Apex poles of the one-loop TADPOLE graphs on the cone, from the frozen XVI-P vertex tensors (read-only copy
in /tmp/o1cone_ro/XVI-P, imported, never modified) and the exact Synge data of the subtracted cone kernels (coinc.py).
Graph value convention (XVI-P/XVI-Q): i Gamma_1 = prefactor * int d^dl/(2pi)^d N/(l^2+i0);  position space:
int d^dl/(2pi)^d e^{il(x-x')}/(l^2+i0) = -i G_E(x,x')|_continued, leg momentum k_leg -> -i d at that leg's point.
Cone: <h_c(x) h_d(x')> numerator Pi_cd -> sum_S (Pi_S)_cd R_{b(S)}(x,x'), Pi_S = helicity-S projected de Donder tensor;
flat part scaleless (=0 in DR).  Apex pole: integrand ~ r^{-2} (theta-independent) => Gamma_pole = (1/(4-d)) int_W 2 pi alpha r^2 f.""
    "
import sys, pickle, json, itertools
from fractions import Fraction as Fr
import sympy as sp
sys.path.insert(0, '/tmp/o1cone_ro/XVI-P/code/p5_entry')
import amplitude_rules as AR
F = AR.F
D = pickle.load(open('/workspace/XVI-01-CONE/logs/v2_coinc_L.pkl', 'rb'))
t, X, Y, z = D['coords']; bS, alS = sp.symbols('b alpha', positive=True)
CO = (t, X, Y, z)
AS = sp.Symbol('alpha', positive=True)
HC = list(F.HC)
ETA = (1, -1, -1, -1)
sq = 1 / sp.sqrt(2)
U = {'t': ((1, 0, 0, 0), 0), 'p': ((0, sq, sp.I * sq, 0), 1), 'm': ((0, sq, -sp.I * sq, 0), -1), 'z': ((0, 0, 0, 1), 0)}
def Nfull(m, n, r, s):
    e = lambda a, b: ETA[a] if a == b else 0
    return sp.Rational(1, 2) * (e(m, r) * e(n, s) + e(m, s) * e(n, r)) - sp.Rational(1, 2) * e(m, n) * e(r, s)
def label(S, branch):
    if branch == 'CR': return S * AS if S >= 0 else 1 + S * AS
    if branch == 'F':   # Friedrichs labels for 1/2 < alpha < 1
        return {0: 0, 1: AS, 2: 2 * AS - 1, -1: 1 - AS, -2: 2 - 2 * AS}[S]
    raise ValueError
# ---- Synge coincidence derivatives (symbolic in X, Y, b, alpha)
A = D['A']; B = D['B']; C = D['C']
def coinc(nx, nxp):
    """nx, nxp: tuples of coordinate indices of derivatives on x and on x' (total <= 2)"""
    d = lambda e, i: sp.diff(e, CO[i])
    if len(nx) + len(nxp) > 2: raise ValueError
    if not nx and not nxp: return A
    if not nx and len(nxp) == 1: return B[nxp[0]]
    if len(nx) == 1 and not nxp: return d(A, nx[0]) - B[nx[0]]
    if not nx and len(nxp) == 2: return 2 * C[nxp[0]][nxp[1]]
```

```
    if len(nx) == 1 and len(nxp) == 1: return d(B[nxp[0]], nx[0]) - 2 * C[nx[0]][nxp[0]]
    i, j = nx
    return d(d(A, i), j) - d(B[j], i) - d(B[i], j) + 2 * C[i][j]
_cc = {}
def coinc_b(nx, nxp, bval):
    key = (nx, nxp)
    if key not in _cc: _cc[key] = sp.expand(coinc(nx, nxp))
    return _cc[key].subs(bS, bval)
# ---- projected propagator numerators
def tensor_sectors():
    out = {}
    for (Aa, (ua, sa)), (Bb, (ub, sb)) in itertools.product(U.items(), repeat=2):
        S = sa + sb
        for ci, (m, n) in enumerate(HC):
            for di, (r, s) in enumerate(HC):
                v = 0
                for m2 in range(4):
                    for n2 in range(4):
                        c2 = sp.conjugate(ua[m2] * ub[n2])
                        if c2 == 0: continue
                        v += ua[m] * ub[n] * c2 * Nfull(m2, n2, r, s)
                v = sp.nsimplify(sp.expand(v))
                if v != 0: out.setdefault(S, {}).setdefault((ci, di), 0); out[S][(ci, di)] += v
    for S in out: out[S] = {k: sp.simplify(v) for k, v in out[S].items() if sp.simplify(v) != 0}
    return out
def vector_sectors(G):
    out = {}
    for Aa, (ua, sa) in U.items():
        for mu in range(4):
            for rho in range(4):
                v = sum(ua[mu] * sp.conjugate(ua[n2]) * G.get((n2, rho), 0) for n2 in range(4))
                v = sp.nsimplify(sp.expand(v))
                if v != 0: out.setdefault(sa, {}).setdefault((mu, rho), 0); out[sa][(mu, rho)] += v
    return out
def mono_derivs(mono, legmap):
    """mono: tuple of var ids (4*leg + r) -> dict leg -> tuple of coordinate indices"""
    out = {}
    for v in mono:
        leg, r = divmod(v, 4); out.setdefault(legmap[leg], []).append(r)
    return {k: tuple(sorted(v)) for k, v in out.items()}
def ext_deriv(hexpr, idx):
    e = hexpr
    for i in idx: e = sp.diff(e, CO[i])
    return e
r2 = X ** 2 + Y ** 2
TESTS = {'T1_htt_r2': {0: r2}, 'T2_hzz_r2': {9: r2}, 'T3_PM_delta_r2': {4: r2, 7: r2},
         'T4_PP_xixj': {4: X ** 2 - r2 / 2, 7: Y ** 2 - r2 / 2, 5: X * Y}}
def onepoint(graph, branch='CR'):
    rules = AR.base_rules(); ints = AR.build_integrands(rules); pref = ints[graph]['prefactor']
    V = F.vertices(); Pi = AR.pinv_sparse(); Gh = AR.ghost_num()
    terms = []   # (ext comp a, loop sector label b, numerator weight, derivative dict)
    if graph == 'D14a':
        vt = V['V3_grav']; TS = tensor_sectors()
        chk = {k: 0 for k in Pi}
        for S, dct in TS.items():
            for k, v in dct.items(): chk[k] = chk.get(k, 0) + v
        assert all(sp.simplify(chk.get(k, 0) - Pi.get(k, 0)) == 0 for k in set(chk) | set(Pi)), 'projector sum != Pi'
        legmap = {0: 'ext', 1: 'x', 2: 'xp'}
        for (a, c, d), poly in vt.T.items():
            for S, dct in TS.items():
                wgt = dct.get((c, d))
                if wgt is None: continue
                for mono, cf in poly.d.items(): terms.append((a, label(S, branch), wgt * sp.Rational(cf.numerator, cf.denominator),
     mono_derivs(mono, legmap)))
    elif graph == 'D14b':
        vt = V['V_cb_c_h']; VS = vector_sectors(Gh)   # legs (cb_rho, c^mu, h_a); c at x (first arg), cb at x' (second)
        legmap = {0: 'xp', 1: 'x', 2: 'ext'}
        for (rho, mu, a), poly in vt.T.items():
            for s, dct in VS.items():
                wgt = dct.get((mu, rho))
                if wgt is None: continue
                for mono, cf in poly.d.items(): terms.append((a, label(s, branch), wgt * sp.Rational(cf.numerator, cf.denominator),
     mono_derivs(mono, legmap)))
    elif graph == 'D14c':
        vt = V['V_h_phi_phi']; legmap = {0: 'ext', 1: 'x', 2: 'xp'}
        num = F.propagators()['scalar']['numerator']
        for (a, _, _), poly in vt.T.items():
            for mono, cf in poly.d.items(): terms.append((a, 0, num * sp.Rational(cf.numerator, cf.denominator), mono_derivs(mono,
     legmap)))
    # flat (p=0) reproduction of the exported integrand: sum over sectors at b=0 must equal ints N (legs -> l, -l)
    res = {'graph': graph, 'prefactor': str(pref), 'n_terms': len(terms)}
    for tn, hd in TESTS.items():
        f = 0
        for (a, bval, wgt, dv) in terms:
```

```
            if a not in hd: continue
            e = ext_deriv(hd[a], dv.get('ext', ()))
            if e == 0: continue
            nder = len(dv.get('ext', ())) + len(dv.get('x', ())) + len(dv.get('xp', ()))
            f += wgt * (-sp.I) ** nder * e * coinc_b(dv.get('x', ()), dv.get('xp', ()), bval)
        f = sp.expand(-pref * (-sp.I) * f / sp.I)   # Gamma = (1/i) * pref * (-i) * [..] ; kappa kept symbolic in pref
        v0 = sp.simplify((r2 * f).subs({X: 1, Y: 0})); v1 = sp.simplify((r2 * f).subs({X: sp.Rational(3, 5), Y: sp.Rational(4, 5)}))
        res[tn] = {'r2f_theta0': str(v0), 'theta_independent': sp.simplify(v0 - v1) == 0,
                   'pole_coeff_per_unit_area': str(sp.simplify(2 * sp.pi * AS * v0))}
        print(graph, tn, res[tn], flush=True)
    return res
if __name__ == '__main__':
    g = sys.argv[1]; br = sys.argv[2] if len(sys.argv) > 2 else 'CR'
    out = onepoint(g, br)
    json.dump(out, open('/workspace/XVI-01-CONE/logs/v2_tadpole_%s%s.json' % (g, '' if br == 'CR' else '_' + br), 'w'), indent=1)
```

4.2 tadpole2.py

SHA-256: e7b1246714acc43614e5d725175215ed757407e15d76aef3b8b926cfd701b3db

```
"""v3 item (3): apex poles of the two-point TADPOLE graphs D02 (V4_grav, graviton loop), D12 (V_hh_phi_phi, scalar
loop) and D05a (V_hh_phi_phi, graviton loop, external phi phi) on the cone, CR branch (or F), same coincidence route as
D14 (cone_tadpole.py), with the corrected overall sign Gamma = -prefactor*[...] (varsigma=-1, XVI-P rule).
External fields: general rotation-equivariant, reflection-even n=0 jets up to m=2 (symbolic coefficients),
times exp(+-i(p0 t + p3 z)).  Output: the apex pole bilinear (coefficient of 1/(4-d), per unit area of W).
usage: tadpole2.py <D02|D12|D05a> [CR|F]"""
import sys, json, itertools, time
sys.path.insert(0, '/workspace/XVI-01-CONE/code')
import sympy as sp
import cone_tadpole as CT
AR = CT.AR; F = CT.F; X, Y = CT.X, CT.Y; AS = CT.AS; HC = CT.HC
g = sys.argv[1]; br = sys.argv[2] if len(sys.argv) > 2 else 'CR'
T0 = time.time()
kap = F.KAPPA
p0, p3, lam = sp.symbols('p0 p3 lam')
def hfield(tag):
    c = sp.symbols(' '.join(f'{n}{tag}' for n in ['a0', 'a1', 'b0', 'b1', 'c0', 'c1', 'd', 'e', 'f0', 'f1', 'g']))
    a0, a1, b0, b1, c0, c1, d, e, f0, f1, gg = c; r2 = X**2 + Y**2; xs = (None, X, Y)
    H = {}
    for ci, (m, n) in enumerate(HC):
        if (m, n) == (0, 0): v = a0 + a1 * r2
        elif (m, n) == (0, 3): v = b0 + b1 * r2
        elif (m, n) == (3, 3): v = c0 + c1 * r2
        elif m == 0 and n in (1, 2): v = d * xs[n]
        elif m in (1, 2) and n == 3: v = e * xs[m]
        elif m in (1, 2) and n in (1, 2): v = (f0 + f1 * r2 if m == n else 0) + gg * (xs[m] * xs[n] - (r2 / 2 if m == n else 0))
        else: raise ValueError
        H[ci] = v
    return H, c
def sfield(tag):
    s0, s1 = sp.symbols(f's0{tag} s1{tag}'); return {0: s0 + s1 * (X**2 + Y**2)}, (s0, s1)
CO = (sp.Symbol('t'), X, Y, sp.Symbol('z'))
def extd(expr, idx, sgn):
    e = expr
    for i in idx:
        if i == 0: e = sp.I * sgn * p0 * e
        elif i == 3: e = sp.I * sgn * p3 * e
        else: e = sp.diff(e, CO[i])
    return e
rules = AR.base_rules(); ints = AR.build_integrands(rules); V = F.vertices()
if g == 'D02':
    vt = V['V4_grav']; pref = ints['D02']['prefactor']; loop = 'grav'; legs_ext = (0, 1); legmap = {0: 'e1', 1: 'e2', 2: 'x', 3: 'xp'
     }; ext = 'h'
elif g == 'D12':
    vt = V['V_hh_phi_phi']; pref = ints['D12']['prefactor']; loop = 'scal'; legmap = {0: 'e1', 1: 'e2', 2: 'x', 3: 'xp'}; ext = 'h'
elif g == 'D05a':   # not in XVI-P amplitude_rules: prefactor by analogy with D02/D12: S_f*loop_sign*(i k^2)*phase*i
    vt = V['V_hh_phi_phi']; pref = sp.nsimplify(sp.Rational(1, 2) * (sp.I * kap**2) * (sp.I ** vt.phase) * sp.I)
    loop = 'grav'; legmap = {0: 'x', 1: 'xp', 2: 'e1', 3: 'e2'}; ext = 'phi'
if ext == 'h': E1, c1 = hfield('_1'); E2, c2 = hfield('_2')
else: E1, c1 = sfield('_1'); E2, c2 = sfield('_2')
TS = CT.tensor_sectors() if loop == 'grav' else None
PTS = [(sp.Integer(1), sp.Integer(0)), (sp.Rational(3, 5), sp.Rational(4, 5))]
cache = {}
def coinc_at(nx, nxp, bval, pt):
    key = (nx, nxp, bval, pt)
    if key not in cache:
        v = CT.coinc_b(nx, nxp, bval)
        cache[key] = sp.cancel(v.subs({X: lam * pt[0], Y: lam * pt[1]}))
    return cache[key]
acc = [0, 0]; nterm = 0
for key, poly in vt.T.items():
    # key = component indices per leg
```

```
    if g in ('D02', 'D12'):
        a, b = key[0], key[1]
        if g == 'D02':
            c, d = key[2], key[3]; secs = [(S, w.get((c, d))) for S, w in TS.items() if w.get((c, d)) is not None]
        else: secs = [(0, sp.Integer(1))]
    else:
        c, d = key[0], key[1]; a, b = 0, 0
        secs = [(S, w.get((c, d))) for S, w in TS.items() if w.get((c, d)) is not None]
    if a not in E1 or b not in E2 or not secs: continue
    for mono, cf in poly.d.items():
        dv = CT.mono_derivs(mono, legmap)
        e1 = extd(E1[a], dv.get('e1', ()), +1); e2 = extd(E2[b], dv.get('e2', ()), -1)
        if e1 == 0 or e2 == 0: continue
        nder = sum(len(dv.get(k, ())) for k in ('e1', 'e2', 'x', 'xp'))
        base = sp.Rational(cf.numerator, cf.denominator) * (-sp.I) ** nder * e1 * e2
        for S, w in secs:
            bval = CT.label(S, br) if loop == 'grav' else 0
            for ip, pt in enumerate(PTS):
                ee = base.subs({X: lam * pt[0], Y: lam * pt[1]})
                acc[ip] += w * ee * coinc_at(dv.get('x', ()), dv.get('xp', ()), bval, pt)
        nterm += 1
    if nterm % 2000 == 0: print('terms', nterm, round(time.time() - T0), flush=True)
print('collected', nterm, round(time.time() - T0), flush=True)
out = {'graph': g, 'branch': br, 'prefactor': str(pref), 'sign_rule': 'Gamma=-prefactor*[..]', 'n_terms': nterm}
res = []
for ip in range(2):
    e = sp.expand(-pref * acc[ip] * lam**2 * (PTS[ip][0]**2 + PTS[ip][1]**2))   # r^2 f on the ray, times -pref
    e = sp.together(e); num, den = sp.fraction(e)
    num = sp.Poly(sp.expand(num), lam); den = sp.Poly(sp.expand(den), lam)
    # den should be a monomial in lam
    assert len(den.terms()) == 1, 'nonmonomial denominator'
    (dd,), dc = den.terms()[0]
    c0 = sum(cf * 1 for (k,), cf in num.terms() if k == dd) / dc
    res.append(sp.factor(sp.expand(2 * sp.pi * AS * c0)))
out['theta_independent'] = sp.simplify(res[0] - res[1]) == 0
P = sp.Poly(sp.expand(res[0]), *(list(c1) + list(c2) + [p0, p3]))
out['pole_bilinear'] = {str(m): str(sp.factor(cf)) for m, cf in zip(P.monoms(), P.coeffs())}
out['vars'] = [str(v) for v in list(c1) + list(c2) + [p0, p3]]
print(json.dumps(out)[:3000], flush=True)
json.dump(out, open('/workspace/XVI-01-CONE/logs/v3/tad2_%s_%s.json' % (g, br), 'w'), indent=1)
print('done', round(time.time() - T0), flush=True)
```

4.3 tadpole2_fast.py

SHA-256: d7878153e6c3693ea063cb691fce994a0f3ec91301d4bbf2031331839e66963f

```
"""v3 item (3): apex poles of the two-point TADPOLE graphs D02 (V4_grav, graviton loop), D12 (V_hh_phi_phi, scalar
loop) and D05a (V_hh_phi_phi, graviton loop, external phi phi) on the cone, CR branch (or F), same coincidence route as
D14 (cone_tadpole.py), with the corrected overall sign Gamma = -prefactor*[...] (varsigma=-1, XVI-P rule).
External fields: general rotation-equivariant, reflection-even n=0 jets up to m=2 (symbolic coefficients),
times exp(+-i(p0 t + p3 z)).  Output: the apex pole bilinear (coefficient of 1/(4-d), per unit area of W).
usage: tadpole2.py <D02|D12|D05a> [CR|F]"""
import sys, json, itertools, time
sys.path.insert(0, '/workspace/XVI-01-CONE/code')
import sympy as sp
import cone_tadpole as CT
AR = CT.AR; F = CT.F; X, Y = CT.X, CT.Y; AS = CT.AS; HC = CT.HC
g = sys.argv[1]; br = sys.argv[2] if len(sys.argv) > 2 else 'CR'
T0 = time.time()
kap = F.KAPPA
p0, p3, lam = sp.symbols('p0 p3 lam')
def hfield(tag):
    c = sp.symbols(' '.join(f'{n}{tag}' for n in ['a0', 'a1', 'b0', 'b1', 'c0', 'c1', 'd', 'e', 'f0', 'f1', 'g']))
    a0, a1, b0, b1, c0, c1, d, e, f0, f1, gg = c; r2 = X**2 + Y**2; xs = (None, X, Y)
    H = {}
    for ci, (m, n) in enumerate(HC):
        if (m, n) == (0, 0): v = a0 + a1 * r2
        elif (m, n) == (0, 3): v = b0 + b1 * r2
        elif (m, n) == (3, 3): v = c0 + c1 * r2
        elif m == 0 and n in (1, 2): v = d * xs[n]
        elif m in (1, 2) and n == 3: v = e * xs[m]
        elif m in (1, 2) and n in (1, 2): v = (f0 + f1 * r2 if m == n else 0) + gg * (xs[m] * xs[n] - (r2 / 2 if m == n else 0))
        else: raise ValueError
        H[ci] = v
    return H, c
def sfield(tag):
    s0, s1 = sp.symbols(f's0{tag} s1{tag}'); return {0: s0 + s1 * (X**2 + Y**2)}, (s0, s1)
C0 = (sp.Symbol('t'), X, Y, sp.Symbol('z'))
def extd(expr, idx, sgn):
    e = expr
    for i in idx:
        if i == 0: e = sp.I * sgn * p0 * e
```

```
        elif i == 3: e = sp.I * sgn * p3 * e
        else: e = sp.diff(e, C0[i])
    return e
rules = AR.base_rules(); ints = AR.build_integrands(rules); V = F.vertices()
if g == 'D02':
    vt = V['V4_grav']; pref = ints['D02']['prefactor']; loop = 'grav'; legs_ext = (0, 1); legmap = {0: 'e1', 1: 'e2', 2: 'x', 3: 'xp'
     }; ext = 'h'
elif g == 'D12':
    vt = V['V_hh_phi_phi']; pref = ints['D12']['prefactor']; loop = 'scal'; legmap = {0: 'e1', 1: 'e2', 2: 'x', 3: 'xp'}; ext = 'h'
elif g == 'D05a':   # not in XVI-P amplitude_rules: prefactor by analogy with D02/D12: S_f*loop_sign*(i k^2)*phase*i
    vt = V['V_hh_phi_phi']; pref = sp.nsimplify(sp.Rational(1, 2) * (sp.I * kap**2) * (sp.I ** vt.phase) * sp.I)
    loop = 'grav'; legmap = {0: 'x', 1: 'xp', 2: 'e1', 3: 'e2'}; ext = 'phi'
if ext == 'h': E1, c1 = hfield('_1'); E2, c2 = hfield('_2')
else: E1, c1 = sfield('_1'); E2, c2 = sfield('_2')
TS = CT.tensor_sectors() if loop == 'grav' else None
PTS = [(sp.Integer(1), sp.Integer(0)), (sp.Rational(3, 5), sp.Rational(4, 5))]
cache = {}
def lpoly(expr):
    """Laurent dict {power of lam: coeff} for expr = num/den with monomial den in lam"""
    expr = sp.cancel(sp.together(expr)); num, den = sp.fraction(expr)
    dp = sp.Poly(sp.expand(den), lam); assert len(dp.terms()) == 1, 'nonmonomial denominator'
    (dd,), dc = dp.terms()[0]
    out = {}
    for (k,), cf in sp.Poly(sp.expand(num), lam).terms(): out[k - dd] = out.get(k - dd, 0) + cf / dc
    return out
def coinc_at(nx, nxp, bval, pt):
    key = (nx, nxp, bval, pt)
    if key not in cache:
        v = CT.coinc_b(nx, nxp, bval)
        cache[key] = lpoly(v.subs({X: lam * pt[0], Y: lam * pt[1]}))
    return cache[key]
acc = [0, 0]; nterm = 0
for key, poly in vt.T.items():
    if g in ('D02', 'D12'):
        a, b = key[0], key[1]
        if g == 'D02':
            c, d = key[2], key[3]; secs = [(S, w.get((c, d))) for S, w in TS.items() if w.get((c, d)) is not None]
        else: secs = [(0, sp.Integer(1))]
    else:
        c, d = key[0], key[1]; a, b = 0, 0
        secs = [(S, w.get((c, d))) for S, w in TS.items() if w.get((c, d)) is not None]
    if a not in E1 or b not in E2 or not secs: continue
    for mono, cf in poly.d.items():
        dv = CT.mono_derivs(mono, legmap)
        e1 = extd(E1[a], dv.get('e1', ()), +1); e2 = extd(E2[b], dv.get('e2', ()), -1)
        if e1 == 0 or e2 == 0: continue
        nder = sum(len(dv.get(k, ())) for k in ('e1', 'e2', 'x', 'xp'))
        base = sp.Rational(cf.numerator, cf.denominator) * (-sp.I) ** nder * e1 * e2
        for ip, pt in enumerate(PTS):
            bp = sp.Poly(sp.expand(base.subs({X: lam * pt[0], Y: lam * pt[1]})), lam)
            bd = {k: c_ for (k,), c_ in bp.terms()}
            for S, w in secs:
                bval = CT.label(S, br) if loop == 'grav' else 0
                cd = coinc_at(dv.get('x', ()), dv.get('xp', ()), bval, pt)
                # need total lam power -2 (so that lam^2 * (...) is lam^0)
                tot = 0
                for k, c_ in bd.items():
                    if (-2 - k) in cd: tot += c_ * cd[-2 - k]
                if tot != 0: acc[ip] += w * tot
        nterm += 1
    if nterm % 2000 == 0: print('terms', nterm, round(time.time() - T0), flush=True)
print('collected', nterm, round(time.time() - T0), flush=True)
out = {'graph': g, 'branch': br, 'prefactor': str(pref), 'sign_rule': 'Gamma=-prefactor*[..]', 'n_terms': nterm}
res = []
for ip in range(2):
    c0 = sp.expand(-pref * acc[ip] * (PTS[ip][0]**2 + PTS[ip][1]**2))
    res.append(sp.expand(2 * sp.pi * AS * c0))
out['theta_independent'] = sp.simplify(res[0] - res[1]) == 0
P = sp.Poly(sp.expand(res[0]), *(list(c1) + list(c2) + [p0, p3]))
out['pole_bilinear'] = {str(m): str(sp.factor(cf)) for m, cf in zip(P.monoms(), P.coeffs())}
out['vars'] = [str(v) for v in list(c1) + list(c2) + [p0, p3]]
print(json.dumps(out)[:3000], flush=True)
json.dump(out, open('/workspace/XVI-01-CONE/logs/v3/tad2fast_%s_%s.json' % (g, br), 'w'), indent=1)
print('done', round(time.time() - T0), flush=True)
```

4.4 hh_structures.py

SHA-256: adc0bdbe0482e92d59bd26de05b90c41e5d90e7dcb390c6ef6440f9d3c3f3f23

```
"""v6 item (e): decompose the constant-profile apex pole bilinears (lattice keys, independent coordinates
x = (h00, h03, h33, f0)) into the 6 independent SO(1,1)-invariant symmetric structures quadratic in p
(2D Cayley-Hamilton removes p h h' p):
```

```
 S1 = P^2 trh trh', S2 = P^2 h.h', S3 = (php) trh' + trh (ph'p), S6 = P^2 (f0 trh' + trh f0'), S7 = (php) f0' + f0 (ph'p), S8 = P^2 f0
     f0'.
Exact in alpha; also the gauge-invariant (Ward) combinations."""
import json, sympy as sp
al = sp.Symbol('alpha', positive=True); q0, q3 = sp.symbols('q0 q3')
ETA2 = [1, -1]; P2 = q0**2 - q3**2
x = sp.symbols('x00 x03 x33 xf'); z = sp.symbols('z00 z03 z33 zf')
def mats(v): return [[v[0], v[1]], [v[1], v[2]]], v[3]
p = [q0, q3]   # lower components p_a (a=0,3)
def up(M): return [[ETA2[a]*ETA2[b]*M[a][b] for b in range(2)] for a in range(2)]
def tr(M): return sum(ETA2[a]*M[a][a] for a in range(2))
def php(M):
    pu = [ETA2[a]*p[a] for a in range(2)]
    return sum(pu[a]*M[a][b]*pu[b] for a in range(2) for b in range(2))
def dot(M, N): U = up(M); return sum(U[a][b]*N[a][b] for a in range(2) for b in range(2))
hx, fx = mats(x); hz, fz = mats(z)
S = {'S1': P2*tr(hx)*tr(hz), 'S2': P2*dot(hx, hz), 'S3': php(hx)*tr(hz) + tr(hx)*php(hz),
     'S6': P2*(fx*tr(hz) + tr(hx)*fz), 'S7': php(hx)*fz + fx*php(hz), 'S8': P2*fx*fz}
idx = {0: 0, 3: 1, 9: 2, 4: 3, 7: 3}
def bil(path):
    D = json.load(open(path))['Gamma_pole_coeff_of_1/(4-d)']
    tot = 0
    for k, v in D.items():
        a, b = eval(k)
        if a in idx and b in idx: tot += sp.sympify(v, locals={'alpha': al}) * x[idx[a]] * z[idx[b]]
    return sp.expand(tot)
cs = sp.symbols('b1 b2 b3 b6 b7 b8')
def decompose(B):
    ans = sp.expand(sum(c*S[k] for c, k in zip(cs, S)))
    eqs = sp.Poly(sp.expand(ans - B), *x, *z, q0, q3).coeffs()
    sol = sp.solve(eqs, cs, dict=True)
    if not sol: return None
    return {k: str(sp.factor(sol[0].get(c, c))) for c, k in zip(cs, S)}
out = {}
for br in ['CR', 'F', 'UNIT']:
    out[br] = {}
    tot = 0
    for g in ['D01', 'D02', 'D03', 'D11', 'D12']:
        try: B = bil(f'logs/v4/spec_{g}_{br}.json')
        except FileNotFoundError: out[br][g] = 'MISSING'; continue
        tot += B
        out[br][g] = decompose(B) if B != 0 else 'zero (computed)'
    out[br]['hh_total'] = decompose(sp.expand(tot))
    print(br, json.dumps(out[br], indent=0), flush=True)
json.dump(out, open('logs/v6/hh_structures.json', 'w'), indent=1)
```

5 Spectral apex reduction of the bubbles (v4)

5.1 spectral_apex.py

SHA-256: cb22811e721c932766a576cb15626015584f9cbc3cc3696f793e664c43debde2

```
"""v4 item (1): spectral apex reduction for one-loop kappa^2 two-point graphs on the cone, constant-profile
(n=0, m=0) external fields with longitudinal momentum p.
Hypotheses: (H_I) intertwining of transverse derivatives between helicity bundles on CR; then the apex part of
Tr_perp f(-Lap_b) = C_b f(0) with C_b = (1/12)(1/a - a) - b(1-b)/(2a)  (scale-invariant heat-trace constant).
Hence: apex pole = sum_S C_{b(S)} x [UV pole of the (d-2)-dim LONGITUDINAL loop integral of the flat XVI-P integrand
at k_perp = 0], each internal line weighted by the helicity class of its propagator components.
Graph conventions copied (read-only) from XVI-P amplitude_rules.build_integrands and XVI-Q Q02 bulk_graphs (D04, D06),
D05a (Q02 symmetry factor 1/2).  i Gamma = pref * int d^dl/(2pi)^d N/dens  ->  Gamma = -i pref int ...
Wick: l0 = i y0, l3 = y3;  int d^Dl_M/(2pi)^D f  pole  = i/(8 pi^2 eps) int_0^{2pi} g(theta) dtheta,  eps=(4-d)/2,
where g = lim R^2 * [degree -2 part of f].  Output: coefficient of 1/(4-d) of Gamma_{ab}(p), per unit area of W.
usage: spectral_apex.py <graph> [CR|F]"""
import sys, json, time, itertools
sys.path.insert(0, '/tmp/o1cone_ro/XVI-P/code/p5_entry')
import sympy as sp
import amplitude_rules as AR
F = AR.F
I = sp.I
g = sys.argv[1]; br = sys.argv[2] if len(sys.argv) > 2 else 'CR'
T0 = time.time()
LS, PS = sp.symbols('l0:4'), sp.symbols('p0:4')
NAMES = {**{r: LS[r] for r in range(4)}, **{4 + r: PS[r] for r in range(4)}}
kap = F.KAPPA
al = sp.Symbol('alpha', positive=True)
u = sp.symbols('u0:3'); v = sp.symbols('v0:3')
def sym(poly): return sp.expand(poly.to_sympy(NAMES)) if hasattr(poly, 'to_sympy') else sp.expand(poly)
HC = list(F.HC)
def ctype(i):
    m, n = HC[i]; L = (m in (0, 3)) + (n in (0, 3))
```

```
    return {2: 'L', 1: 'M', 0: 'T'}[L]
def cls(i, j):
    a, b = ctype(i), ctype(j)
    if a == b == 'M': return 1
    if a == b == 'T': return 2
    return 0
Pi = AR.pinv_sparse(); Gh = AR.ghost_num()
def Piw(i, j, mk): return sp.Rational(Pi[(i, j)].numerator, Pi[(i, j)].denominator) * mk[cls(i, j)]
def Ghw(i, j, mk): return sp.Rational(Gh[(i, j)].numerator, Gh[(i, j)].denominator) * mk[0 if i in (0, 3) else 1]
V = F.vertices()
def route(vt, r): return {k: sym(p) for k, p in F.vt_route(vt, r).items()}
def C_of(b): return sp.Rational(1, 12) * (1 / al - al) - b * (1 - b) / (2 * al)
def lab(S):
    if br == 'CR': return S * al if S >= 0 else 1 + S * al
    return {0: 0, 1: al, 2: 2 * al - 1}[S]       # F, 1/2 < alpha < 1 (|S| only; C symmetric under b -> 1-b)
CS = [sp.Integer(1)]*3 if br == "UNIT" else [C_of(lab(S)) for S in range(3)]
N = {}; dens = None
if g == 'D01':
    V3 = V['V3_grav']; VA = route(V3, [(0, 1), (-1, 0), (1, -1)]); VB = route(V3, [(0, -1), (1, 0), (-1, 1)])
    for (a, c, e), pa in VA.items():
        for (b, d, f), pb in VB.items():
            if (c, d) in Pi and (e, f) in Pi:
                N[(a, b)] = N.get((a, b), 0) + pa * pb * Piw(c, d, u) * Piw(e, f, v)
    pref = AR.base_rules()['graphs']['D01']['S_f'] * (I * kap) ** 2 * I ** (2 * V3.phase) * I ** 2; dens = 'll-p'
elif g == 'D03':
    Vg = V['V_cb_c_h']; VA = route(Vg, [(-1, 0), (1, -1), (0, 1)]); VB = route(Vg, [(-1, 1), (1, 0), (0, -1)])
    for (rho, mu, a), pa in VA.items():
        for (s, nu, b), pb in VB.items():
            if (mu, s) in Gh and (nu, rho) in Gh:
                N[(a, b)] = N.get((a, b), 0) + pa * pb * Ghw(mu, s, u) * Ghw(nu, rho, v)
    pref = -1 * (I * kap) ** 2 * I ** (2 * Vg.phase); dens = 'll-p'
elif g == 'D11':
    Vs = V['V_h_phi_phi']; VA = route(Vs, [(0, 1), (-1, 0), (1, -1)]); VB = route(Vs, [(0, -1), (1, 0), (-1, 1)])
    for (a, _, _), pa in VA.items():
        for (b, _, _), pb in VB.items():
            N[(a, b)] = N.get((a, b), 0) + pa * pb * u[0] * v[0]
    pref = sp.Rational(1, 2) * (I * kap) ** 2 * I ** (2 * Vs.phase) * I ** 2; dens = 'll-p'
elif g == 'D04':   # Q02: VA h_c:-l, phi_ext:p, phi:l-p ; VB h_d:l, phi_ext:-p, phi:p-l ; graviton line l, scalar line l-p
    Vs = V['V_h_phi_phi']; VA = route(Vs, [(-1, 0), (0, 1), (1, -1)]); VB = route(Vs, [(1, 0), (0, -1), (-1, 1)])
    for (c, _, _), pa in VA.items():
        for (d, _, _), pb in VB.items():
            if (c, d) in Pi:
                N[(0, 0)] = N.get((0, 0), 0) + pa * pb * Piw(c, d, u) * v[0]
    pref = 1 * (I * kap) ** 2 * I ** (2 * Vs.phase) * I ** 2; dens = 'll-p'
elif g == 'D06':   # Q02: VA cbar_rho:-p, c^mu:l, h_a:p-l ; VB cbar_sig:-l, c^nu:p, h_b:l-p ; ghost line l, graviton l-p
    Vg = V['V_cb_c_h']; VA = route(Vg, [(0, -1), (1, 0), (-1, 1)]); VB = route(Vg, [(-1, 0), (0, 1), (1, -1)])
    for (rho, mu, a), pa in VA.items():
        for (sg, nu, b), pb in VB.items():
            if (mu, sg) in Gh and (a, b) in Pi:
                N[(rho, nu)] = N.get((rho, nu), 0) + pa * pb * Ghw(mu, sg, u) * Piw(a, b, v)
    pref = 1 * (I * kap) ** 2 * I ** (2 * Vg.phase) * I ** 1; dens = 'll-p'
elif g == 'D13':
    rules = AR.base_rules(); VK = AR.K_vertex(rules); Vg = V['V_cb_c_h']
    VA = route(VK, [(0, -1), (1, 0), (-1, 1)]); VB = route(Vg, [(-1, 0), (0, 1), (1, -1)])
    for (ap, mu, c), pa in VA.items():
        for (rho, nu, d), pb in VB.items():
            if (mu, rho) in Gh and (c, d) in Pi:
                N[(ap, nu)] = N.get((ap, nu), 0) + pa * pb * Ghw(mu, rho, u) * Piw(c, d, v)
    R = rules['graphs']['D13']
    pref = R['ext_sign'] * R['S_f'] * R['loop_sign'] * (I * kap) ** 2 * I ** (VK.phase + Vg.phase) * I; dens = 'll-p'
elif g == 'D02':
    V4 = V['V4_grav']; VA = route(V4, [(0, 1), (0, -1), (1, 0), (-1, 0)])
    for (a, b, c, d), pa in VA.items():
        if (c, d) in Pi: N[(a, b)] = N.get((a, b), 0) + pa * Piw(c, d, u) * v[0]
    pref = sp.Rational(1, 2) * (I * kap ** 2) * I ** V4.phase * I; dens = 'l'
elif g == 'D12':
    V2s = V['V_hh_phi_phi']; VA = route(V2s, [(0, 1), (0, -1), (1, 0), (-1, 0)])
    for (a, b, _, _), pa in VA.items(): N[(a, b)] = N.get((a, b), 0) + pa * u[0] * v[0]
    pref = sp.Rational(1, 2) * (I * kap ** 2) * I ** V2s.phase * I; dens = 'l'
elif g == 'D05a':
    V2s = V['V_hh_phi_phi']; VB = route(V2s, [(1, 0), (-1, 0), (0, 1), (0, -1)])
    for (c, d, _, _), pa in VB.items():
        if (c, d) in Pi: N[(0, 0)] = N.get((0, 0), 0) + pa * Piw(c, d, u) * v[0]
    pref = sp.Rational(1, 2) * (I * kap ** 2) * I ** V2s.phase * I; dens = 'l'
print('built', g, len(N), round(time.time() - T0, 1), flush=True)
# longitudinal kinematics, Wick rotation
y0, y3, R, th, w = sp.symbols('y0 y3 R theta w', real=True)
q0, q3 = sp.symbols('q0 q3')         # external Minkowski longitudinal momentum p = (q0, 0, 0, q3)
sub = {LS[1]: 0, LS[2]: 0, PS[1]: 0, PS[2]: 0, PS[0]: q0, PS[3]: q3}
lM0, lM3 = I * R * sp.cos(th), R * sp.sin(th)
def msq(a0, a3): return a0 ** 2 - a3 ** 2
den = msq(lM0, lM3) if dens == 'l' else msq(lM0, lM3) * msq(lM0 - q0, lM3 - q3)
def pole(expr):
    """coefficient of 1/(4-d) of int d^Dl_M/(2pi)^D expr/den, expr polynomial in l0,l3."""
```

```
    e = sp.expand(expr.subs(sub).subs({LS[0]: lM0, LS[3]: lM3}))
    if e == 0: return 0
    f = e / den
    ser = sp.series(f.subs(R, 1 / w), w, 0, 3).removeO()
    g2 = sp.expand(ser.coeff(w, 2))
    g2 = sp.expand(sp.expand_trig(g2.rewrite(sp.exp)))
    integ = sp.integrate(g2, (th, 0, 2 * sp.pi))
    return sp.simplify(I / (4 * sp.pi ** 2) * integ)    # i/(8 pi^2 eps), eps=(4-d)/2
res = {}; mixed_ok = True
for k, e in N.items():
    e = sp.expand(e)
    if e == 0: continue
    P = sp.Poly(e, *u, *v)
    tot = 0
    for mon, cf in zip(P.monoms(), P.coeffs()):
        iu = [i for i in range(3) if mon[i]]; iv = [i for i in range(3) if mon[3 + i]]
        pv = pole(cf)
        if pv == 0: continue
        s1, s2 = iu[0], iv[0]
        if dens == 'l': s2 = s1          # tadpole: v0 is a dummy marker (no second line)
        if s1 != s2:
            mixed_ok = False; print('MIXED nonzero', k, mon, pv, flush=True); continue
        tot += pv * CS[s1]
    val = sp.factor(sp.simplify(-I * pref * tot))
    if val != 0: res[str(k)] = str(val)
out = {'graph': g, 'branch': br, 'pref': str(pref), 'mixed_helicity_cancel': mixed_ok, 'Gamma_pole_coeff_of_1/(4-d)': res,
       'ext_keys': 'HC component indices (graviton) / ghost vector index / 0 for scalar', 'momentum': 'p=(q0,0,0,q3), mostly-minus',
       'seconds': round(time.time() - T0, 1)}
json.dump(out, open('/workspace/XVI-01-CONE/logs/v4/spec_%s_%s.json' % (g, br), 'w'), indent=1)
print(json.dumps(out)[:3000])
```

5.2 spec_compare.py

SHA-256: d681f04c3810f7e6f80b0f51f4489714099fd42fb9e22e09608d42539b501eb7

```
"""compare spectral constant-profile results with coincidence-route (tadpole2) constant-constant terms; S=0 externals only."""
import json, sys, sympy as sp
q0,q3=sp.symbols('q0 q3')
names=['a0','b0','c0','f0']
def bil(S, tag1='_1', tag2='_2'):
    a=[sp.Symbol(n+tag1) for n in names]; b=[sp.Symbol(n+tag2) for n in names]
    H1={0:a[0],3:a[1],9:a[2],4:a[3],7:a[3]}; H2={0:b[0],3:b[1],9:b[2],4:b[3],7:b[3]}
    B=0
    for k,v in S.items():
        x,y=eval(k)
        if x in H1 and y in H2: B+=sp.sympify(v)*H1[x]*H2[y]
    return sp.expand(B.subs({q0:sp.Symbol('p0'),q3:sp.Symbol('p3')}))
def tadconst(T):
    V=T['vars']; keep={n+t for n in names for t in ('_1','_2')}|{'p0','p3'}
    Tt=0
    for k,v in T['pole_bilinear'].items():
        idx=eval(k)
        if all((e==0 or V[i] in keep) for i,e in enumerate(idx)):
            Tt+=sp.sympify(v)*sp.prod([sp.Symbol(V[i])**e for i,e in enumerate(idx)])
    return sp.expand(Tt)
if __name__=='__main__':
    g=sys.argv[1]
    for br in ['CR','F']:
        S=json.load(open(f'logs/v4/spec_{g}_{br}.json'))['Gamma_pole_coeff_of_1/(4-d)']
        tf=f'logs/v3/tad2fast_{g}_{br}.json' if g=='D02' else f'logs/v3/tad2_{g}_CR.json'
        T=json.load(open(tf))
        B=bil(S); Tt=tadconst(T)
        print(g,br,'spectral terms',len(sp.Add.make_args(B)),'coincidence terms',len(sp.Add.make_args(Tt)),'difference:',sp.simplify(B
     -Tt))
```

5.3 catalogue_check.py

SHA-256: 851f295b79cf2a483e6f9de4558d0f842082bd7eff46efd1866d2ad93c3180df

```
"""v6: compare every printed v5 catalogue entry (coefficient form) with the
executed spectral logs logs/v4/spec_<G>_<BR>.json. Keys: HC index
0=00,1=01,2=02,3=03,4=11,5=12,6=13,7=22,8=23,9=33; ghost index 0,3.
Absent log key = computed zero (spectral_apex.py drops exact zeros)."""
import json, sympy as sp
al, ka, q0, q3 = sp.symbols('alpha kappa q0 q3')
I, pi = sp.I, sp.pi
P2 = q0**2 - q3**2
c0 = (1-al**2)/(12*al); c1 = (5*al**2-6*al+1)/(12*al)
C2 = {'CR': (23*al**2-12*al+1)/(12*al), 'F': (23*al**2-36*al+13)/(12*al)}
```

```
def sym(d):
    out = {}
    for (a, b), v in d.items(): out[(a, b)] = v; out[(b, a)] = v
    return out
def expected(br):
    c2 = C2[br]; A = c0+2*c1+4*c2; B = -c0+2*c1
    R = 3*c0+4*c1; T = 4*c0+3*c1; V = c1-c0; W = (c0+2*c1+c2)/2
    k16 = ka**2/(16*pi)
    E = {}
    E['D04'] = {(0, 0): -ka**2/(4*pi)*c0*P2}
    E['D05a'] = {(0, 0): -ka**2/(4*pi)*(c2-c0)*P2}
    E['D11'] = sym({(4, 4): c0/2*P2*k16, (4, 7): c0/2*P2*k16, (7, 7): c0/2*P2*k16})
    def tens(t):  # apply 1<->2 images 4<->7 on single transverse index
        out = dict(t)
        for (a, b), v in t.items():
            for (x, y) in [(a, b)]:
                m = {4: 7}
                if a == 4 and b not in (4, 7): out[(7, b)] = v
                if b == 4 and a not in (4, 7): out[(a, 7)] = v
        return sym(out)
    D03 = {(0, 0): -c0*P2, (0, 9): -c0*P2, (9, 9): -c0*P2, (3, 3): 4*c0*P2,
           (0, 4): c0*(q0**2-3*q3**2), (3, 4): 4*c0*q0*q3, (4, 9): -c0*(3*q0**2-q3**2),
           (4, 4): -(c0+4*c1)*P2, (7, 7): -(c0+4*c1)*P2, (4, 7): -c0*P2,
           (5, 5): -8*c1*P2}  # (5,5) new in v6 (absent from v4/v5 table)
    D01 = {(0, 0): c0*P2, (0, 9): c0*P2, (9, 9): c0*P2, (3, 3): -4*c0*P2,
           (4, 4): (c0-4*c1+c2)*P2, (7, 7): (c0-4*c1+c2)*P2, (4, 7): (c0+c2)*P2,
           (3, 4): -4*(c0+2*c2)*q0*q3, (4, 9): A*q0**2-B*q3**2, (0, 4): -B*q0**2+A*q3**2,
           (5, 5): -8*c1*P2}  # (5,5) new in v6
    E['D03'] = {k: v*k16 for k, v in tens(D03).items()}
    E['D01'] = {k: v*k16 for k, v in tens(D01).items()}
    E['D06'] = {(0, 0): I*ka**2/(8*pi)*R*P2, (3, 3): I*ka**2/(8*pi)*R*P2}
    E['D13'] = {(0, 0): -2*T*q0, (0, 3): 2*V*q3, (3, 0): -R*q3, (3, 3): R*q0,
                (9, 0): -2*V*q0, (9, 3): 2*T*q3}
    E['D13'] = {k: I*ka**2/(16*pi)*v for k, v in E['D13'].items()}
    D02p = {(1, 1): q3**2, (2, 2): q3**2, (5, 5): P2, (6, 6): q0**2, (8, 8): q0**2,
            (1, 6): -q0*q3, (2, 8): -q0*q3, (0, 4): -q3**2/2, (0, 7): -q3**2/2,
            (3, 4): q0*q3, (3, 7): q0*q3, (4, 7): -P2/2, (4, 9): -q0**2/2, (7, 9): -q0**2/2}
    E['D02'] = {k: ka**2/pi*W*v for k, v in sym(D02p).items()}
    E['D12'] = {}
    return E
loc = {'alpha': al, 'kappa': ka, 'q0': q0, 'q3': q3, 'I': I, 'pi': pi}
report = {}
for br in ['CR', 'F']:
    E = expected(br)
    for g, exp in E.items():
        d = json.load(open(f'logs/v4/spec_{g}_{br}.json'))
        got = {tuple(int(x) for x in k.strip('()').split(',')): sp.sympify(v, locals=loc)
               for k, v in d['Gamma_pole_coeff_of_1/(4-d)'].items()}
        keys = set(got) | set(exp)
        bad = [str(k) for k in sorted(keys) if sp.simplify(got.get(k, 0) - exp.get(k, 0)) != 0]
        import re
        try: mixed = sorted(set(re.findall(r'MIXED nonzero (\(\d+, \d+\))', open(f'logs/v4/spec_{g}_{br}.log').read())))
        except FileNotFoundError: mixed = 'no log'
        report[f'{g}_{br}'] = {'mixed_dropped_keys': mixed, 'n_log_keys': len(got), 'n_expected_keys': len([k for k in exp if exp[k]
     != 0]),
                               'mismatch': bad, 'mixed_helicity_flag': d.get('mixed_helicity_cancel')}
        print(g, br, report[f'{g}_{br}'])
json.dump(report, open('logs/v6/catalogue_check.json', 'w'), indent=1)
print('ALL_MATCH', all(not r['mismatch'] for r in report.values()))
```

6 Slavnov–Taylor and Ward diagnostics

6.1 `st_test_const.py`

SHA-256: `dd87dfd55dbf1a730ad31d728fced6d9aa1f35c59afc0b5ab0f8072f9b81f0ee`

```
"""v4: linearised ST (sigma_0) hh-row test on the apex pole, constant-profile S=0 sector (lattice components
h_00,h_03,h_33, transverse trace f0 (h11=h22); ghost c^0,c^3; K_ab).  Row: 2 B_hh(K2 c(p), h(-p)) + lam * G_Kc(E h, c) = 0.
lam (and the K off-diagonal weight w) are convention constants: they must be alpha-INDEPENDENT and fit ALL samples.
B_hh = D01+D02+D03+D11+D12 (spectral, CR or F); G_Kc = D13 (spectral)."""
import json, sys, random, sympy as sp
br = sys.argv[1] if len(sys.argv) > 1 else 'CR'
q0, q3, al = sp.symbols('q0 q3 alpha'); ka = sp.Symbol('kappa')
HC = [(0,0),(0,1),(0,2),(0,3),(1,1),(1,2),(1,3),(2,2),(2,3),(3,3)]
ETA = [1,-1,-1,-1]
def load(g):
    return {eval(k): sp.sympify(v) for k, v in json.load(open(f'/workspace/XVI-01-CONE/logs/v4/spec_{g}_{br}.json'))['
     Gamma_pole_coeff_of_1/(4-d)'].items()}
LAT = {0, 3, 9, 4, 7}
Bhh = {}
```

```
for g in ['D01', 'D02', 'D03', 'D11', 'D12']:
    for k, v in load(g).items():
        if k[0] in LAT and k[1] in LAT: Bhh[k] = Bhh.get(k, 0) + v
GKc = {k: v for k, v in load('D13').items() if k[0] in LAT and k[1] in (0, 3)}
def B(x, y):   # x at +p, y at -p ; dicts comp->value
    return sum(v * x.get(a, 0) * y.get(b, 0) for (a, b), v in Bhh.items())
def G(K, c):
    return sum(v * K.get(a, 0) * c.get(n, 0) for (a, n), v in GKc.items())
def full(hc):  # comp dict -> 4x4 symmetric lower-index matrix
    M = [[0]*4 for _ in range(4)]
    for i, (m, n) in enumerate(HC):
        if i in hc: M[m][n] = M[n][m] = hc[i]
    return M
def comps(M, w):
    return {0: M[0][0], 3: w * M[0][3], 9: M[3][3], 4: M[1][1], 7: M[2][2]}
def E(hc, p):   # linearised Einstein-type operator (overall factor -> lam), lower indices, mostly minus
    h = full(hc); pu = [ETA[i]*p[i] for i in range(4)]
    tr = sum(ETA[i]*h[i][i] for i in range(4)); p2 = sum(ETA[i]*p[i]**2 for i in range(4))
    ph = [sum(pu[r]*h[r][n] for r in range(4)) for n in range(4)]; pph = sum(pu[n]*ph[n] for n in range(4))
    return [[p2*h[m][n] - p[m]*ph[n] - p[n]*ph[m] + p[m]*p[n]*tr + (ETA[m] if m == n else 0)*(pph - p2*tr) for n in range(4)] for m in
      range(4)]
def K2(c, p):   # c^nu upper -> delta h_{mu nu} = p_mu c_nu + p_nu c_mu (factor -> lam)
    cl = [ETA[i]*c.get(i, 0) for i in range(4)]
    return [[p[m]*cl[n] + p[n]*cl[m] for n in range(4)] for m in range(4)]
random.seed(1)
lam = sp.Symbol('lam')
p = [q0, 0, 0, q3]; mp = [-x for x in p]
out = {}
for w in [1, 2, sp.Rational(1, 2)]:
    eqs = []
    for s in range(6):
        hc = {i: random.randint(-5, 5) for i in (0, 3, 9)}; f0 = random.randint(-5, 5); hc[4] = f0; hc[7] = f0
        c = {0: random.randint(-5, 5), 3: random.randint(-5, 5)}
        sub = {q0: random.randint(1, 9), q3: random.randint(1, 9)}
        dh = comps(K2(c, p), 1)
        Eh = comps(E(hc, mp), w)
        r = sp.expand((2 * B(dh, hc) + lam * G(Eh, c)).subs(sub))
        eqs.append(r)
    # solve lam from first nonzero-in-lam eq, test rest as polynomials in alpha
    sol = None
    for e in eqs:
        cl = sp.expand(e).coeff(lam)
        if sp.simplify(cl) != 0:
            sol = sp.simplify(-(e - cl*lam) / cl); break
    res = [sp.factor(sp.simplify(e.subs(lam, sol))) for e in eqs] if sol is not None else None
    out[str(w)] = {'lam': str(sp.factor(sol)) if sol is not None else None, 'residuals': [str(x) for x in res] if res else None,
                   'hh_only_residuals': [str(sp.factor(e.subs(lam, 0))) for e in eqs]}
    print(w, 'lam =', sp.factor(sol) if sol is not None else None, 'residuals:', res, flush=True)
json.dump({'branch': br, 'fits': out}, open(f'/workspace/XVI-01-CONE/logs/v4/st_const_{br}.json', 'w'), indent=1)
```

6.2 st_scan.py

SHA-256: 4a03c966a77b60a028e57163aef7f2645118c797ec3d55fc5c44ac8ee63cf759

```
"""v4: convention scan for the hh ST row. Control branch UNIT (all C_S=1: k_perp=0 reduction of the flat integrand,
pointwise Ward identities => must close). Pick conventions that close UNIT, then apply unchanged to CR and F."""
import json, sys, random, itertools, sympy as sp
sys.argv = [sys.argv[0], 'UNIT']
exec(open('/workspace/XVI-01-CONE/code/st_test_const.py').read().split('random.seed(1)')[0])
p = [q0, 0, 0, q3]; lam = sp.Symbol("lam")
def run(brn, w, sE, cup, tw, sdh, lamfix=None):
    global Bhh, GKc
    Bhh = {}
    for g in ['D01','D02','D03','D11','D12']:
        for k, v in {eval(k): sp.sympify(v) for k, v in json.load(open(f'logs/v4/spec_{g}_{brn}.json'))['Gamma_pole_coeff_of_1/(4-d)'
     ].items()}.items():
            if k[0] in LAT and k[1] in LAT: Bhh[k] = Bhh.get(k, 0) + v
    GKc = {eval(k): sp.sympify(v) for k, v in json.load(open(f'logs/v4/spec_D13_{brn}.json'))['Gamma_pole_coeff_of_1/(4-d)'].items()
     if eval(k)[0] in LAT and eval(k)[1] in (0,3)}
    random.seed(7); eqs = []
    for s in range(8):
        hc = {i: random.randint(-5,5) for i in (0,3,9)}; f0 = random.randint(-5,5); hc[4]=f0; hc[7]=f0
        c = {0: random.randint(-5,5), 3: random.randint(-5,5)}
        cc = c if cup else {0: c[0], 3: -c[3]}
        sub = {q0: random.randint(1,9), q3: random.randint(1,9)}
        pp = [sdh*x for x in p]
        dh = comps(K2(cc, pp), 1)
        Ep = [sE*x for x in p]
        Em = E(hc, Ep); Eh = comps(Em, w); Eh[4] *= tw; Eh[7] *= tw
        eqs.append(sp.expand((2*B(dh, hc) + lam*G(Eh, c)).subs(sub)))
    if lamfix is not None:
        return lamfix, [sp.factor(e.subs(lam, lamfix)) for e in eqs]
```

```
    sol = None
    for e in eqs:
        cl = e.coeff(lam)
        if sp.simplify(cl) != 0: sol = sp.factor(-(e - cl*lam)/cl); break
    if sol is None: return None, [sp.factor(e) for e in eqs]
    return sol, [sp.factor(e.subs(lam, sol)) for e in eqs]
good = []
for w, sE, cup, tw, sdh in itertools.product([1,2,sp.Rational(1,2)], [1,-1], [True, False], [1,2,sp.Rational(1,2)], [1,-1]):
    sol, res = run('UNIT', w, sE, cup, tw, sdh)
    ok = all(r == 0 for r in res)
    if ok: good.append((w, sE, cup, tw, sdh, sol)); print('UNIT closes:', w, sE, cup, tw, sdh, 'lam=', sol, flush=True)
if not good: print('UNIT: no convention closes; sample', sol, res[:3])
out = {'UNIT_closing_conventions': [[str(x) for x in g] for g in good]}
for brn in ['CR', 'F']:
    out[brn] = []
    for (w, sE, cup, tw, sdh, sol) in good:
        lf, res = run(brn, w, sE, cup, tw, sdh)
        l2, res2 = run(brn, w, sE, cup, tw, sdh, lamfix=sol)
        out[brn].append({'conv': [str(w), sE, cup, str(tw), sdh], 'lam_fit': str(lf), 'res_fit': [str(r) for r in res], '
     res_with_UNIT_lam': [str(r) for r in res2]})
        print(brn, (w, sE, cup, tw, sdh), 'lam_fit', lf, '| res(fit)', res[:4], '| res(UNIT lam)', res2[:4], flush=True)
json.dump(out, open('logs/v4/st_scan.json', 'w'), indent=1)
```

6.3 st_norm.py

SHA-256: da1c19a9f75926f26490d44129fc21fa952b97384d4784a7c79219e5f1ff0f31

```
"""v6 item (a): predict the ST convention constant lam from XVI-P tree normalisations (read-only import).
ST (one loop, b eliminated, ghost equation used):  sum_a [dS/dK'_a] [dG1/dh_a] + sum_a [dG1/dK'_a] [dS_inv/dh_a] = 0,
dS/dK'_a = val(V_K_c)(a,mu) c^mu = i*A_GH*(p_m c_n + p_n c_m)   (XVI-P check K_c_equals_a_dc_plus_dc; p = c-leg momentum),
dS_inv/dh_a = N * Eup_a(h)  (fit N on de Donder-satisfying h, where the F^T F gauge-fixing part vanishes).
=> lam_pred = N / (i*A_GH) in the normalisation of st_v6.py."""
import sys, json, sympy as sp
sys.path.insert(0, '/tmp/o1cone_ro/XVI-P/code/p5_entry')
import amplitude_rules as AR
F = AR.F; I = sp.I
q0, q3 = sp.symbols('q0 q3')
V2 = F.grav_vertex(2)
k = [F.ksym(leg) for leg in range(2)]
names = {4 * leg + r: k[leg][r] for leg in range(2) for r in range(4)}
print('V2 phase', V2.phase, 'legs', getattr(V2, 'legs', None), 'A_GH', F.A_GH, 'HC', F.HC)
ETA = [1, -1, -1, -1]; HC = list(F.HC)
p = [q0, 0, 0, q3]
def V2ab(a, b, s0=1):
    P = V2.T.get((a, b))
    if P is None: return 0
    e = (I ** V2.phase) * P.to_sympy(names)
    return sp.expand(e.subs({**{k[0][r]: s0 * p[r] for r in range(4)}, **{k[1][r]: -s0 * p[r] for r in range(4)}}))
LAT = [0, 3, 9, 4, 7]
h = sp.symbols('x0:10')
# de Donder-satisfying lattice h: F_nu = p^mu h_mu nu - 1/2 p_nu tr h = 0
hm = [[0]*4 for _ in range(4)]
for i, (m, n) in enumerate(HC):
    if i in LAT: hm[m][n] = hm[n][m] = (h[4] if i == 7 else h[i])
pu = [ETA[i] * p[i] for i in range(4)]
tr = sum(ETA[i] * hm[i][i] for i in range(4))
Fdd = [sp.expand(sum(pu[m] * hm[m][n] for m in range(4)) - sp.Rational(1, 2) * p[n] * tr) for n in range(4)]
sol = sp.solve([Fdd[0], Fdd[3]], [h[0], h[3]], dict=True)[0]; print('de Donder solution', sol)
hv = {i: (sol.get(h[i], h[i]) if i in LAT else 0) for i in range(10)}
hv[7] = hv[4]  # helicity-zero transverse trace f0
# my E (lower) and Eup with (2-delta)
hm2 = [[0]*4 for _ in range(4)]
for i, (m, n) in enumerate(HC): hm2[m][n] = hm2[n][m] = hv[i]
tr2 = sum(ETA[i] * hm2[i][i] for i in range(4)); p2 = q0**2 - q3**2
ph = [sum(pu[r] * hm2[r][n] for r in range(4)) for n in range(4)]; pph = sum(pu[n] * ph[n] for n in range(4))
El = [[p2*hm2[m][n] - p[m]*ph[n] - p[n]*ph[m] + p[m]*p[n]*tr2 + (ETA[m] if m == n else 0)*(pph - p2*tr2) for n in range(4)] for m in
     range(4)]
Eup = {i: (2 - (m == n)) * ETA[m] * ETA[n] * El[m][n] for i, (m, n) in enumerate(HC)}
out = {}
for s0 in (1, -1):
    Vh = {a: sp.expand(sum(V2ab(a, b, s0) * hv[b] for b in range(10))) for a in range(10)}
    ratios = set()
    for a in range(10):
        if sp.expand(Eup[a]) != 0:
            ratios.add(sp.simplify(Vh[a] / Eup[a]))
        elif Vh[a] != 0: ratios.add('mismatch')
    out[str(s0)] = [str(r) for r in ratios]
    print('momentum sign', s0, 'V2 h / Eup ratios:', ratios)
N = list(sp.sympify(out['1'][0]) for _ in [0])[0] if len(out['1']) == 1 else None
lam = sp.simplify(N / (I * F.A_GH)) if N is not None else None
print('N =', N, ' A_GH =', F.A_GH, ' lam_pred =', lam)
```

```
json.dump({'N': str(N), 'A_GH': str(F.A_GH), 'lam_pred': str(lam), 'ratios': out}, open('/workspace/XVI-01-CONE/logs/v6/st_norm.json',
      'w'), indent=1)
```

6.4 st_v6.py

SHA-256: 5dba49edc69ff70ca01cb0a428aef450262c117f19a6d86074acf4dc89cb6155

```
"""v6 item (a): hh Slavnov-Taylor row on constant-profile apex poles, with the conventions DERIVED from the XVI-P
closed forms (P14 amplitude_rules check 'K_c_equals_a_dc_plus_dc': K'_(mn) a (d_m c_n + d_n c_m), sum over independent (mn)):
   sum_a dh_a(c) B_ab h_b  +  lam * sum_(mn) Gamma_{K'_(mn) c^nu} c^nu * dS/dh_(mn) = 0,
   dh_(mn) = p_m c_n + p_n c_m (independent coordinates, c_n = eta_nn c^n),
   dS/dh_(mn) = (2 - delta_mn) E^{mn}(h)   (E = linearised Einstein operator, indices raised with eta=diag(1,-1,-1,-1)).
Exact symbolic test (no random sampling): residual is a polynomial in h00,h03,h33,f0,c0,c3,q0,q3 with alpha-dependent
coefficients; lam must be one alpha-independent constant.  usage: st_v6.py <path pattern with {g}> <label>"""
import json, sys, sympy as sp
tag, br = sys.argv[1], sys.argv[2]
q0, q3, al, ka = sp.symbols('q0 q3 alpha kappa'); al = sp.Symbol('alpha', positive=True)
ETA = [1, -1, -1, -1]; HC = [(0,0),(0,1),(0,2),(0,3),(1,1),(1,2),(1,3),(2,2),(2,3),(3,3)]
LAT = {0, 3, 9, 4, 7}
def load(g):
    try: D = json.load(open(tag.format(g=g)))['Gamma_pole_coeff_of_1/(4-d)']
    except FileNotFoundError: return None
    return {eval(k): sp.sympify(v, locals={'alpha': al}) for k, v in D.items()}
h00, h03, h33, f0, c0, c3, lam = sp.symbols('h00 h03 h33 f0 c0 c3 lam')
hx = {0: h00, 3: h03, 9: h33, 4: f0, 7: f0}
cu = {0: c0, 3: c3}
p = [q0, 0, 0, q3]
def B_of(graphs):
    Bm = {}
    for g in graphs:
        G = load(g)
        if G is None: print('MISSING', g); return None
        for (a, b), v in G.items():
            if a in LAT and b in LAT: Bm[(a, b)] = Bm.get((a, b), 0) + v
    return Bm
def dh():
    cl = [ETA[i] * cu.get(i, 0) for i in range(4)]
    return {i: p[m] * cl[n] + p[n] * cl[m] for i, (m, n) in enumerate(HC) if i in LAT}
def Eup():
    h = [[0]*4 for _ in range(4)]
    for i, (m, n) in enumerate(HC):
        if i in hx: h[m][n] = h[n][m] = hx[i]
    pu = [ETA[i]*p[i] for i in range(4)]
    tr = sum(ETA[i]*h[i][i] for i in range(4)); p2 = sum(ETA[i]*p[i]**2 for i in range(4))
    ph = [sum(pu[r]*h[r][n] for r in range(4)) for n in range(4)]; pph = sum(pu[n]*ph[n] for n in range(4))
    El = [[p2*h[m][n] - p[m]*ph[n] - p[n]*ph[m] + p[m]*p[n]*tr + (ETA[m] if m == n else 0)*(pph - p2*tr) for n in range(4)] for m in
     range(4)]
    # transversality check of E (lower): p^m E_mn = 0
    assert all(sp.expand(sum(pu[m]*El[m][n] for m in range(4))) == 0 for n in range(4))
    return {i: (2 - (m == n)) * ETA[m] * ETA[n] * El[m][n] for i, (m, n) in enumerate(HC)}
def row(hhgraphs, kc='D13'):
    Bm = B_of(hhgraphs); Gk = load(kc)
    if Bm is None or Gk is None: return None
    x = dh()
    t1 = sp.expand(sum(v * x.get(a, 0) * hx.get(b, 0) for (a, b), v in Bm.items()))
    Eu = Eup()
    t2 = sp.expand(sum(v * Eu.get(a, 0) * cu.get(n, 0) for (a, n), v in Gk.items()))
    return t1, t2
def coeffs(e):
    e = sp.expand(e)
    if e == 0: return {}
    P = sp.Poly(e, h00, h03, h33, f0, c0, c3, q0, q3)
    return {m: sp.factor(c) for m, c in zip(P.monoms(), P.coeffs())}
out = {'branch': br, 'conventions': 'derived (w=-2 on K_(03); c upper in vertex; E invariant)'}
r = row(['D01', 'D02', 'D03', 'D11', 'D12'])
t1, t2 = r
C1, C2 = coeffs(t1), coeffs(t2)
mons = sorted(set(C1) | set(C2))
out['n_equations'] = len(mons)
out['B_dh_h_terms'] = {str(m): str(c) for m, c in C1.items()}
out['G_Eh_c_terms'] = {str(m): str(c) for m, c in C2.items()}
sol = None
for m in mons:
    if C2.get(m, 0) != 0:
        sol = sp.simplify(-C1.get(m, 0) / C2[m]); break
res = {str(m): str(sp.factor(sp.simplify(C1.get(m, 0) + (sol if sol is not None else 0) * C2.get(m, 0)))) for m in mons}
out['lam'] = str(sol); out['residuals'] = res; out['all_zero'] = all(v == '0' for v in res.values())
out['lam_alpha_independent'] = (sol is not None) and (sp.diff(sol, al) == 0)
# matter subsector (scalar loop): D11 + D12 must be transverse by itself (no Kc scalar graph at one loop)
rm = row(['D11', 'D12']) if load('D12') is not None else row(['D11'])
out['matter_B_dh_h'] = str(sp.factor(rm[0]))
print(json.dumps(out, indent=1))
```

```
json.dump(out, open(f'/workspace/XVI-01-CONE/logs/v6/st_v6_{br}.json', 'w'), indent=1)
```

6.5 st_v6_stats.py

SHA-256: 36ec924f70166b004720d547c1bf6cac4ae2283aec8321ea7db2234639241c31

```
"""v6: per-graph contributions to the hh ST row (CR, F): how many coefficient equations are nontrivial cancellations."""
import sys, json, sympy as sp
res = {}
for br in ['CR', 'F']:
    sys.argv = ['st_v6.py', 'logs/v4/spec_{g}_' + br + '.json', br]
    src = open('code/st_v6.py').read().split("out = {'branch'")[0]
    ns = {}; exec(src, ns)
    per = {}
    allm = set()
    for g in ['D01', 'D02', 'D03', 'D11', 'D12']:
        t1, _ = ns['row']([g]) if g != 'D12' else ns['row'](['D12'])
        C = ns['coeffs'](t1); per[g] = C; allm |= set(C)
    _, t2 = ns['row'](['D01']); G = ns['coeffs'](t2); allm |= set(G)
    lam = -2 * sp.I
    rows = []
    for m in sorted(allm):
        contrib = {g: per[g].get(m, 0) for g in per}; gk = G.get(m, 0)
        tot = sp.simplify(sum(contrib.values()) + lam * gk)
        rows.append({'monomial(h00,h03,h33,f0,c0,c3,q0,q3)': str(m), **{g: str(v) for g, v in contrib.items()}, 'lam*G': str(sp.factor
  (lam * gk)), 'total': str(tot)})
    res[br] = {'n_monomials_with_any_nonzero_contribution': len(allm), 'n_cancelling_between_graphs': sum(1 for r in rows if r['lam*G'
  ] == '0'), 'rows': rows}
    print(br, len(allm), 'monomials;', res[br]['n_cancelling_between_graphs'], 'cancel among hh graphs alone; all totals zero:', all(r
  ['total'] == '0' for r in rows))
json.dump(res, open('logs/v6/st_v6_stats.json', 'w'), indent=1)
```

6.6 ward_scan.py

SHA-256: 891d5ff37fa8ff0a562d5e8a679535b4044d00944eac6230bbf35339d2979082

```
# Radial scan of the spin-2 Ward residual (I-B-G-S eq:gravityWard), CR (=bgeom) vs Friedrichs, copied/adapted from I-B-G-S/code/ward.
    py
import mpmath as mp,sys,json
mp.mp.dps=30
sys.path.insert(0,'/workspace/XVI-01-CONE/code')
from tensor import *
al=mp.mpf(sys.argv[1]); mode=sys.argv[2]
br={'geom':bgeom,'fried':bfried}[mode]
xp=(mp.mpf("0.35"),mp.mpf("0.8"),mp.mpf("0.45"),mp.mpf("0.2"))
def dx(fun,i,pt): return mp.diff(lambda s: fun(tuple(pt[k]+(s if k==i else 0) for k in range(4))),0)
half=mp.mpf(1)/2
out={'alpha':str(al),'mode':mode,'scan':[]}
for rr in ["1.0","0.3","0.1","0.03","0.01"]:
    x=(mp.mpf("0.1"),mp.mpf(rr),mp.mpf("0.2"),mp.mpf("0.05"))
    def hbar(y,mu,nu,rho,sg):
        W=ten_kernel(al,y,xp,br); val=W[((mu,nu),(rho,sg))]
        if mu==nu: val-=half*sum(W[((l,l),(rho,sg))] for l in range(4))
        return val
    row={'r':rr}
    for (nu,rho,sg) in [(1,1,1),(2,2,2),(1,1,2)]:
        lhs=sum(dx(lambda y: hbar(y,mu,nu,rho,sg),mu,x) for mu in range(4))
        rhs=dx(lambda y: vec_kernel(al,x,y)[nu][sg],rho,xp)+dx(lambda y: vec_kernel(al,x,y)[nu][rho],sg,xp)
        row[str((nu,rho,sg))]=[mp.nstr(abs(lhs+rhs),6),mp.nstr(abs(rhs),6)]
    out['scan'].append(row); print(row,flush=True)
json.dump(out,open('/workspace/XVI-01-CONE/logs/wardscan_%s_%s.json'%(sys.argv[1],mode),'w'),indent=1)
```

6.7 ward_xcheck.py

SHA-256: c0ba86940b98df10033669b7f05a5e6e25beadaca42cbe7b8237b431819acf2b

```
"""(F) Second implementation of the spin-2 Ward residual: identical residual as ward_scan.py, but with the kernel
S_b(u,psi) evaluated by the explicit angular MODE SUM (with explicit E0 flips for CR labels b>1, b<0) instead of the
closed Bernoulli/exponential form.  usage: ward_xcheck.py <alpha> <geom|fried> <r>"""
import mpmath as mp, sys, json
mp.mp.dps = 20
sys.path.insert(0, '/workspace/XVI-01-CONE/code')
import tensor
def S_modes(b, u, psi):
    f = lambda n: mp.exp(1j * (n + b) * psi - abs(n + b) * u)
    s = mp.nsum(f, [-mp.inf, mp.inf])
```

```
    if b > 1: s += mp.exp(1j * (b - 1) * psi) * (mp.exp((b - 1) * u) - mp.exp(-(b - 1) * u))
    if b < 0: s += mp.exp(1j * b * psi) * (mp.exp(-b * u) - mp.exp(b * u))
    return s
tensor.S = S_modes
from tensor import ten_kernel, vec_kernel, bgeom, bfried
al = mp.mpf(sys.argv[1]); mode = sys.argv[2]; rr = sys.argv[3]
br = {'geom': bgeom, 'fried': bfried}[mode]
xp = (mp.mpf("0.35"), mp.mpf("0.8"), mp.mpf("0.45"), mp.mpf("0.2"))
def dx(fun, i, pt): return mp.diff(lambda s: fun(tuple(pt[k] + (s if k == i else 0) for k in range(4))), 0)
half = mp.mpf(1) / 2
x = (mp.mpf("0.1"), mp.mpf(rr), mp.mpf("0.2"), mp.mpf("0.05"))
def hbar(y, mu, nu, rho, sg):
    W = ten_kernel(al, y, xp, br); val = W[((mu, nu), (rho, sg))]
    if mu == nu: val -= half * sum(W[((l, l), (rho, sg))] for l in range(4))
    return val
row = {'alpha': sys.argv[1], 'mode': mode, 'r': rr, 'impl': 'mode-sum'}
for (nu, rho, sg) in [(1, 1, 1), (2, 2, 2), (1, 1, 2)]:
    lhs = sum(dx(lambda y: hbar(y, mu, nu, rho, sg), mu, x) for mu in range(4))
    rhs = dx(lambda y: vec_kernel(al, x, y)[nu][sg], rho, xp) + dx(lambda y: vec_kernel(al, x, y)[nu][rho], sg, xp)
    row[str((nu, rho, sg))] = [mp.nstr(abs(lhs + rhs), 6), mp.nstr(abs(rhs), 6)]
    print(row, flush=True)
json.dump(row, open('/workspace/XVI-01-CONE/logs/v2_wardx_%s_%s_%s.json' % (sys.argv[1], mode, rr), 'w'), indent=1)
```

7 $\mathbf{H}_I obstruction for D01(v6)$

7.1 hi_d01.py

SHA-256: f3e7ee7f82bde0c6aa85d4fc57e2807a47fe95a53b9af5348d31b206a0e524d5

```
"""v6 item (c): exact obstruction to (H_I) for D01 on CR.  Composite trace of the S=2 -> 3 raising operator
X = (d_r - nu/r) acting on both arguments of the CR-minus-F radial heat-kernel difference in the flipped channel
(nu = 2 - 1/alpha), diagonal integrated with r dr.  Diagonal ~ c0 r^(-2nu-2): NOT integrable (any nu>0).
Mellin (homogeneous) extension R(z) = int r^(1+z) D(r) dr, continued to z=0 (unique iff 2nu not integer).
Scale invariance: R(0) = kappa_*/s.  Prints s*R(0) for several alpha."""
import mpmath as mp, json
mp.mp.dps = 60
def run(alpha, s=mp.mpf(1)):
    nu = 2 - 1/mp.mpf(alpha)
    def dK(r, rp):
        x = r*rp/(2*s)
        return mp.e**(-(r**2+rp**2)/(4*s))/(2*s) * (2*mp.sin(mp.pi*nu)/mp.pi) * mp.besselk(nu, x)
    A = mp.sin(mp.pi*nu)/(mp.pi*s)
    def K(m, x): return mp.besselk(m, x)
    def Kp(x): return -(K(nu-1, x) + K(nu+1, x))/2
    def Kpp(x): return (K(nu-2, x) + 2*K(nu, x) + K(nu+2, x))/4
    def D(r):
        # analytic: dK = A exp(-(r^2+r'^2)/4s) K_nu(r r'/2s); X = (d_r - nu/r), X' = (d_r' - nu/r'), at r'=r
        x = r*r/(2*s); E = mp.e**(-r*r/(2*s)); c = 1/(2*s)
        # derivatives of g(r,r') = exp(-(r^2+r'^2)/4s) K_nu(c r r')
        g = E*K(nu, x)
        gr = E*(-r/(2*s)*K(nu, x) + c*r*Kp(x))          # d/dr (same as d/dr' by symmetry at r=r')
        grr = E*((r/(2*s))**2*K(nu, x) - (r/(2*s))*c*r*Kp(x)*2 + c*Kp(x) + (c*r)**2*Kpp(x))  # d^2/dr dr'
        return A*(grr - 2*nu/r*gr + nu**2/r**2*g)
    c0 = (1/(2*s))*(4*s)**nu*4*nu**2/mp.gamma(1-nu)
    eps = mp.mpf('1e-9')
    c1 = (D(eps) - c0*eps**(-2*nu-2))*eps**(2*nu)          # next term c1 r^(-2nu)
    head = c1*eps**(2-2*nu)/(2-2*nu)                        # int_0^eps r * c1 r^(-2nu)
    inner = head + mp.quad(lambda r: r*(D(r) - c0*r**(-2*nu-2)), [eps, mp.mpf('1e-6'), mp.mpf('1e-3'), mp.mpf('0.25'), 1])
    inner_check = (D(eps*10) - c0*(eps*10)**(-2*nu-2))*(eps*10)**(2*nu)
    print('  c1 stability', mp.nstr(c1, 10), mp.nstr(inner_check, 10), flush=True)
    outer = mp.quad(lambda r: r*D(r), [1, 4, 12, mp.inf])
    R0 = inner + c0/(-2*nu) + outer
    return nu, s*R0
out = {}
for a in ['0.6', '0.7', '0.75', '0.8', '0.9']:
    nu, v = run(mp.mpf(a))
    out[a] = {'nu': mp.nstr(nu, 12), 's*R(0)': mp.nstr(v, 15)}
    print(a, out[a], flush=True)
nu, v2 = run(mp.mpf('0.8'), s=mp.mpf(3)); print('s-check alpha=0.8, s=3:', mp.nstr(v2, 15))
out['s_check_0.8_s3'] = mp.nstr(v2, 15)
json.dump(out, open('/workspace/XVI-01-CONE/logs/v6/hi_d01.json', 'w'), indent=1)
```

7.2 hi_d01_exact.py

SHA-256: c4e7d0271a0890d604e0ae1e8b300bb53bdaa64402e1188faf72247133d0f118

```
"""v6 item (c), exact: Mellin-regularised composite trace R(z) for D01 (H_I) in closed form.
```

```
At s=1/2 (x=r^2): D(r) = A e^{-x}[ x K - 2x K' + K' + x K'' - 2 nu(-K + K') + nu^2 K/x ],  K = K_nu(x),
r dr = dx/2, r^z = x^{z/2}.  Uses  int_0^inf x^{mu-1} e^{-x} K_m(x) dx = sqrt(pi) G(mu+m) G(mu-m) / (2^mu G(mu+1/2))
(DLMF 10.43.22-type Mellin formula, analytically continued in mu).  Tests R(0) = 0 exactly in nu."""
import sympy as sp, json, mpmath as mp
nu, z = sp.symbols('nu z')
def I(mu, m):
    return sp.sqrt(sp.pi)*sp.gamma(mu+m)*sp.gamma(mu-m)/(2**mu*sp.gamma(mu+sp.Rational(1, 2)))
w = z/2 + 1     # x^{z/2} * x^{k} -> mu = z/2 + 1 + k
# K' = -(K_{nu-1}+K_{nu+1})/2 ; K'' = (K_{nu-2} + 2K_nu + K_{nu+2})/4
def Kint(k, kind):  # int x^{z/2+k} e^{-x} [K-derivative](x) dx  (mu = z/2 + k + 1)
    mu = z/2 + k + 1
    if kind == 0: return I(mu, nu)
    if kind == 1: return -(I(mu, nu-1) + I(mu, nu+1))/2
    if kind == 2: return (I(mu, nu-2) + 2*I(mu, nu) + I(mu, nu+2))/4
bracket = (Kint(1, 0) - 2*Kint(1, 1) + Kint(0, 1) + Kint(1, 2) + 2*nu*Kint(0, 0) - 2*nu*Kint(0, 1) + nu**2*Kint(-1, 0))
R = sp.Rational(1, 2)*bracket          # times A = sin(pi nu)/(pi s), omitted (nonzero constant)
out = {}
for nv in ['1/3', '4/7', '2/3', '3/4', '8/9', '0.37', '0.91']:
    f = sp.lambdify(z, R.subs(nu, sp.nsimplify(nv) if '/' in nv else sp.Float(nv, 50)), 'mpmath')
    mp.mp.dps = 50
    out[nv] = mp.nstr(f(mp.mpf('1e-30')), 12)
print('R(z->0) numeric (times 1/A):', out)
# exact: simplify R at z=0 symbolically via gammasimp
R0 = sp.gammasimp(sp.simplify(R.subs(z, 0)))
print('R(0) symbolic:', R0)
Rs = sp.series(R, z, 0, 2).removeO()
out_sym = {'R0_symbolic': str(sp.simplify(R0)), 'R_first_order_in_z': str(sp.simplify(sp.gammasimp(Rs.coeff(z, 1))))}
print(out_sym)
json.dump({'numeric_R0_over_A': out, **out_sym}, open('logs/v6/hi_d01_exact.json', 'w'), indent=1)
# numerical check of the Mellin-Laplace formula in its convergent strip (a=1 limit of DLMF 10.43.22)
mp.mp.dps = 30
fchk = {}
for mu_, m_ in [(2, 0.3), (3.5, 1.7), (2.5, 0.5)]:
    q = mp.quad(lambda x: x**(mu_-1)*mp.e**(-x)*mp.besselk(m_, x), [0, 1, mp.inf])
    f = mp.sqrt(mp.pi)*mp.gamma(mu_+m_)*mp.gamma(mu_-m_)/(2**mu_*mp.gamma(mu_+0.5))
    fchk[str((mu_, m_))] = mp.nstr(q-f, 5)
print('formula check', fchk)
d = json.load(open('logs/v6/hi_d01_exact.json')); d['formula_check_quad_minus_closed'] = fchk
json.dump(d, open('logs/v6/hi_d01_exact.json', 'w'), indent=1)
```

8 Build and artifact scripts for v6

8.1 build_v6.py

SHA-256: 40cd45f3b622260020cfb6f52a91904fc2894a457be599fb12faec260b09663e

```
"""Assemble XVI-01-CONE-v6.tex from read-only v5 + v6src fragments + generated artifact appendix."""
import hashlib, glob, os, subprocess
R = '/workspace/XVI-01-CONE/'
s = open(R + 'XVI-01-CONE-v5.tex').read()
F = lambda n: open(R + 'v6src/' + n).read()
def rep(old, new, count=1):
    global s
    assert s.count(old) == count, (s.count(old), old[:80])
    s = s.replace(old, new)
def between(a, b, new, keep_a=False, keep_b=True):
    """replace text from start of a to start of b"""
    global s
    i = s.index(a); j = s.index(b, i)
    s = s[:i] + ((a if keep_a else '') + new) + s[j:]
# ---------- title / pdf metadata
rep("""  pdftitle={Scoped 0.1 on the Cone:
  Spectral Apex Reduction, Constant-Profile Pole Records,
  and Ward-Identity Boundaries},""",
"""  pdftitle={Scoped 0.1 on the Cone, v6:
  Executed Apex Records, a Closed Constant-Profile Slavnov-Taylor Row,
  and Ward-Identity Boundaries},""")
rep("""\\title{\\textbf{Scoped 0.1 on the Cone}\\\\[5pt]
\\Large Spectral Apex Reduction, Constant-Profile Pole Records,
and Ward-Identity Boundaries\\\\[6pt]
\\large A proof-level normalization audit and conditional reconstruction
of the supplied v4 calculation}""",
"""\\title{\\textbf{Scoped 0.1 on the Cone}\\\\[5pt]
\\Large Executed Apex Records, a Closed Constant-Profile
Slavnov--Taylor Row, Weighted Apex Coefficients,
and Ward-Identity Boundaries\\\\[6pt]
\\large An artifact-verified revision of the v5 proof-level audit}""")
rep("""\\small Version 5; revision of the supplied
\\loc{XVI-01-CONE-v4.tex}}""", """\\small Version 6; revision of \\loc{XVI-01-CONE-v5.tex}}""")
# macros
```

```
rep("\\newcommand{\\UNVALIDATED}{\\textsf{UNVALIDATED}}",
    "\\newcommand{\\UNVALIDATED}{\\textsf{UNVALIDATED}}\n\\newtheorem*{evidence}{Evidence}")
# ---------- abstract
between("\\begin{abstract}", "\\tableofcontents", F('abstract.tex') + "\n\n")
# ---------- evidence classes
between("This revision uses three evidence classes.", "\\subsection{Programme non-transfer}", F('evidence.tex') + "\n")
# ---------- weighted section after flip section
rep("""%====================================================================
\\section{Conditional spectral reduction of a two-point graph}""",
    F('weighted.tex') + """
%====================================================================
\\section{Conditional spectral reduction of a two-point graph}""")
# ---------- H_I subsection
rep("""Merely assuming the already excluded unrestricted tower would not
certify D01.
""", """Merely assuming the already excluded unrestricted tower would not
certify D01.

""" + F('hi.tex'))
# ---------- catalogue
rep("""All formulas below are exact algebraic expressions.
Their graph interpretation is subject to
Assumption~\\ref{ass:reduction} and matching to the frozen vertex
data. No new coefficient has been obtained by an executed program.""", F('cat_status.tex').rstrip())
between("The supplied Q02 comparison reports the D05a symmetry factor", "\\subsection{Tensor two-point sector}", F('cat_q02.tex') + "\n")
rep("""D03:
\\((4,7)\\)
&
\\(\\displaystyle -c_0P^2\\)\\\\[5pt]
""", """D03:
\\((4,7)\\)
&
\\(\\displaystyle -c_0P^2\\)\\\\[5pt]

D03:
\\((5,5)\\) \\emph{(new in v6)}
&
\\(\\displaystyle -8c_1P^2\\)\\\\[5pt]
""")
rep("""D01:
\\((0,4)\\)
&
\\(\\displaystyle -Bq_0^2+Aq_3^2\\)\\\\
""", """D01:
\\((0,4)\\)
&
\\(\\displaystyle -Bq_0^2+Aq_3^2\\)\\\\[5pt]

D01:
\\((5,5)\\) \\emph{(new in v6)}
&
\\(\\displaystyle -8c_1P^2\\)\\\\
""")
rep("""\\caption{The complete set of printed v4 \\(hh\\) bubble entries,
in equivalent apex-coefficient form.""", """\\caption{The \\(hh\\) bubble entries of v4, in equivalent
apex-coefficient form, with the \\((5,5)\\) entries added from the
executed record (v6).""")
rep("""The entries related by \\(1\\leftrightarrow2\\) are identified in the
supplied record:""", """The entries related by \\(1\\leftrightarrow2\\) are identified in the
executed record:""")
between("D02 and D12 were not printed as component formulas in v4.", "\\subsection{Ghost and source-ghost sector}", F('cat_d02.tex') + "\n")
# ---------- branch tail
between("Independent-component weights matter here.", "%====================================================================\n\\section{The ghost-equation row", F('branch_tail.tex') + "\n")
# ---------- ghost remark
rep("""It is not a proof that all source components or all 26
Ward-constrained combinations have been tested.
""", """It is not a proof that all source components or all 26
Ward-constrained combinations have been tested.
""" + F('ghost_add.tex'))
# ---------- ST section
rep("\\section{The \\(hh\\) Slavnov--Taylor diagnostic remains unvalidated}",
    "\\section{The \\texorpdfstring{\\(hh\\)}{hh} Slavnov--Taylor row: v4 diagnostic, diagnosis, and constant-profile closure}")
rep("\\subsection{The supplied attempted row}", "\\subsection{The supplied attempted row (v4, historical)}")
rep("""The code, complete arrays, and scan log are not supplied here.
Consequently these are historical diagnostic reports, not new
executions.""", """In v5 the code, arrays, and scan log were unavailable.
v6 archives them (\\loc{code/st_test_const.py}, \\loc{code/st_scan.py},
\\loc{logs/v4/st_*.json}) and diagnoses the failure in
Section~\\ref{sec:STdiag}.""")
rep("\\begin{proposition}[No regularity verdict follows from this test]",
    "\\begin{proposition}[No regularity verdict follows from the v4 test]")
```

```
between("The correct ending is \\(\\UNVALIDATED\\), not ``cone anomaly found''", "
     %=====================================================================\n\\section{What constant-profile data cannot determine}",
       F('st.tex') + "\n" + F('structures.tex') + "\n")
# ---------- missing section
between("The single computed scalar combination is the longitudinal", "\\subsection{Joint-apex condition \\(\\Jcond\\)}", F('
     missing_scalar.tex') + "\n")
between("It is not an unconditional proof of \\(\\Jcond\\) for all graphs", "\\subsection{Regularity at special cone openings}", F('
     missing_J.tex') + "\n")
between("\\subsection{Evanescent data and finite linear certificates}", "
     %=====================================================================\n\\section{Companion comparisons", F('missing_ev.tex') +
     "\n")
# ---------- repro
rep("\\subsection{Additional proof-level corrections in this revision}", "\\subsection{Proof-level corrections made in v5 (retained)}"
     )
rep("""\\item The full \\(hh\\) tadpole arrays and higher-profile values are
not reconstructed from ``as v3.''""", """\\item The full \\(hh\\) tadpole arrays and higher-profile values are
not reconstructed from ``as v3'' (v6: now executed and printed).""")
rep("""The registry log distinguishes the oct18c report of zero
new identifiers from the later oct18d report of two.
No new registry search is asserted by this revision.""", """The registry log distinguishes the oct18c report of zero
new identifiers from the later oct18d report of two.
v6 rechecked it: 251 identifiers, unchanged.""")
between("\\subsection{Historical artifact inventory}", "%=========================================================================\n\\
     section{Ledger and release}", F('repro.tex') + "\n" + F('response.tex') + "\n")
# ---------- ledger
between("\\begingroup\n\\small\n\\begin{longtable}{@{}L{.57\\linewidth}L{.36\\linewidth}@{}}", "
     %=====================================================================\n\\appendix", F('ledger.tex') + "\n")
# ---------- declarations
between("\\paragraph{Historical execution records.}", "\\paragraph{Submission obligation.}", F('decl_exec.tex') + "\n")
rep("""Direct retrieval of the consulted companion browse endpoints was
unavailable through the research interface used for this revision.
No inaccessible source is claimed to have been read in full.""",
"""Companion manuscripts were not re-read for v6; their citations
carry the mark \\(\\dagger\\).
No inaccessible source is claimed to have been read in full.""")
rep("""The present conditional theorem must not be relabeled an
independently reproduced complete computation.""", """The present conditional theorem must not be relabeled an
independently reproduced complete computation; \\textsc{[exec]} means
reproducible on archived inputs, not independently reviewed.""")
# ---------- artifact appendix before bibliography
s = s.replace("%=====================================================================\n\\begin{thebibliography}{99}",
              "%%ARTIFACT_APPENDIX%%\n%=========================================================================================\n\\begin{thebibliography
     }{99}")
# ---------- bibliography
rep("%=====================================================================\n\\begin{thebibliography}{99}", "
     %=====================================================================\n" + F('bib_legend.tex') + "\\begin{thebibliography}{99}"
     )
s = s.replace('\\textsc{[exec]}', '\\textsf{[exec]}')
marks = {'CheungConeV4': '*', 'CheungP': '*', 'CheungQ': '*', 'CheungR': '\\dagger',
         'Fursaev1994': '\\ddagger', 'FursaevMiele1997': '\\ddagger', 'Vassilevich': '\\ddagger',
         'DLMF': '\\ddagger', 'DerezinskiRichard': '\\ddagger', 'KayStuder': '\\dagger', 'Stockinger': '\\ddagger',
         'HSR': '\\ddagger', 'NielsenAverage': '\\ddagger', 'ReebWolf': '\\ddagger'}
import re
for key in re.findall(r'\\bibitem\{(\w+)\}', s):
    m = marks.get(key, '\\dagger')
    s = s.replace('\\bibitem{%s}\n' % key, '\\bibitem{%s}\n\\(^{%s}\\)\\ ' % (key, m), 1)
rep("""Immediate source of the retained coefficient catalogue and historical
execution reports. Its unavailable code and logs were not rerun here.""",
"""Read in full. Its code and logs are now archived and audited
(Appendix~\\ref{app:artifacts}).""")
rep("""The Q02 D05a symmetry-factor comparison and Q11 scope are retained
as supplied; complete graph artifacts are not included here.""",
"""v6 reads the regenerated Q02 graph record
\\loc{xviq_regen/Q02_bulk_graphs.json} in full and the
read-only certificate \\loc{provenance/XVI-P/certificates/P18_D11.json};
the Q11 scope is retained as supplied.""")
rep("""The later excerpt is a comparison, not a silent contract update.""",
"""The later excerpt is a comparison, not a silent contract update.
Files read in full for v6 (read-only copies, digests equal to the
originals): \\loc{code/p5_entry/amplitude_rules.py},
\\loc{feynman.py}, \\loc{dred.py}.""")
rep("""Complete source version and full cryptographic manifest were not
supplied for independent authentication.""", """Not read for v6; only its full tree digest is recorded
(unchanged before/after).""")
rep("""Chapter 10, especially Sections 10.29, 10.32, and 10.43.""",
    """Chapter 10, especially Sections 10.29, 10.32, and 10.43
(10.43.22 confirmed online for v6).""")
s = s.replace("\\bibitem{CheungP}", """\\bibitem{CheungConeV5}
\\(^{*}\\)\\ N.~Cheung,
\\emph{Scoped O.1 on the Cone, v5: Spectral Apex Reduction,
Constant-Profile Pole Records, and Ward-Identity Boundaries},
18 October 2026. Source \\loc{XVI-01-CONE-v5.tex}
(SHA-256 \\loc{e00e49f5...a6985b}, full digest in
Appendix~\\ref{app:artifacts}), read in full; the base of this revision.
```

```

\\bibitem{CheungP}""", 1)
s = s.replace("\\end{thebibliography}", """\\bibitem{Kabat}
\\(^{\\ddagger}\\)\\ D.~Kabat,
``Black hole entropy and entropy of entanglement,''
\\emph{Nuclear Physics B} \\textbf{453} (1995), 281--299.
DOI: \\nolinkurl{10.1016/0550-3213(95)00443-V}.
arXiv:\\nolinkurl{hep-th/9503016}.
Spin-one horizon contact term; structural analogue only.

\\end{thebibliography}""")
open(R + 'v6build/XVI-01-CONE-v6.pre.tex', 'w').write(s)
print('assembled', len(s.splitlines()))
```

8.2 artifacts_v6.py

SHA-256: d602a6ab9b12105f02baa274c2c212c3624c1c30d96c490cc8bb95e5e9020d00

```
"""Generate the artifact appendix (full SHA-256 + rerun commands) and write the final v6 tex."""
import hashlib, glob, os, subprocess
R = '/workspace/XVI-01-CONE/'
def sha(p): return hashlib.sha256(open(p, 'rb').read()).hexdigest()
def tree(t):
    cmd = f"cd /workspace/XVI-{t} && find . -type f -not -path './.pytest_cache/*' -not -name '*.pyc' | sort | xargs sha256sum | sha256sum"
    return subprocess.run(cmd, shell=True, capture_output=True, text=True).stdout.split()[0]
groups = [
 ('Base source (read-only copy)', ['XVI-01-CONE-v5.tex']),
 ('Programs', ['code/' + f for f in ['spectral_apex.py', 'st_test_const.py', 'st_scan.py', 'catalogue_check.py', 'st_v6.py', 'st_norm.py',
               'st_v6_stats.py', 'flat_pole.py', 'flat_vs_p18.py', 'weighted_apex.py', 'weighted_checks.py', 'hi_d01.py',
               'hi_d01_exact.py', 'hh_structures.py', 'build_v6.py', 'artifacts_v6.py']]),
 ('Spectral logs (v4 session, audited in v6)', sorted(glob.glob('logs/v4/*.json', root_dir=R))),
 ('v6 logs', sorted(glob.glob('logs/v6/*.json', root_dir=R))),
 ('Read-only frozen inputs', ['/tmp/o1cone_ro/XVI-P/code/p5_entry/amplitude_rules.py', '/tmp/o1cone_ro/XVI-P/code/p5_entry/feynman.py',
   '/tmp/o1cone_ro/XVI-P/code/p5_entry/dred.py', '/tmp/o1cone_ro/XVI-Q/provenance/XVI-P/certificates/P18_D11.json',
   'xviq_regen/Q02_bulk_graphs.json']),
]
def tt(path): return path.replace('_', '\\_')
rows = []; manifest = []
for title, files in groups:
    rows.append('\\multicolumn{2}{@{}l}{\\textit{%s}}\\\\' % title)
    for f in files:
        p = f if f.startswith('/') else R + f
        h = sha(p); manifest.append(f'{h}  {f}')
        rows.append('\\texttt{\\footnotesize %s} & \\texttt{\\scriptsize %s\\\\ %s}\\\\' % (tt(f.replace('/tmp/o1cone_ro/', 'RO:')), h[:32], h[32:]) if False else
                    '{\\footnotesize\\loc{%s}} & \\texttt{\\scriptsize %s}\\newline\\texttt{\\scriptsize %s}\\\\' % (f.replace('/tmp/o1cone_ro/', 'RO:'), h[:32], h[32:]))
trees = {t: tree(t) for t in ['P', 'Q', 'R']}
before = dict(l.split()[::-1] if False else (l.split()[0], l.split()[1]) for l in open(R + 'logs/frozen_v6_before.sha').read().split('\n') if l.strip())
trow = []
for t in ['P', 'Q', 'R']:
    b = before.get('XVI' + t, before.get('XVI-' + t, '?'))
    trow.append('XVI-%s & \\texttt{\\scriptsize %s}\\newline\\texttt{\\scriptsize %s} & %s\\\\' % (t, trees[t][:32], trees[t][32:], 'identical' if b == trees[t] else 'DIFFERS'))
app = r"""%=====================================================================
\section{Artifact manifest and rerun commands (new in v6)}
\label{app:artifacts}
%=====================================================================

Paths are relative to \loc{/workspace/XVI-01-CONE/}.
\texttt{RO:} denotes the read-only copies under
\loc{/tmp/o1cone_ro/}; their digests equal those of the originals in
the frozen trees.
The frozen-tree digest is
\begin{center}\footnotesize
\verb!cd /workspace/XVI-$t && find . -type f -not -path './.pytest_cache/*'!\\
\verb!  -not -name '*.pyc' | sort | xargs sha256sum | sha256sum!
\end{center}

\begingroup\small
\begin{longtable}{@{}L{.10\linewidth}L{.52\linewidth}L{.25\linewidth}@{}}
\toprule
Tree & SHA-256 (before and after v6) & before vs.\ after\\
\midrule
\endfirsthead
\bottomrule
\endfoot
""" + '\n'.join(trow) + r"""
```

```
\end{longtable}
\endgroup

\begingroup\small
\begin{longtable}{@{}L{.50\linewidth}L{.44\linewidth}@{}}
\toprule
File & SHA-256\\
\midrule
\endfirsthead
\toprule
File & SHA-256\\
\midrule
\endhead
\bottomrule
\endfoot
""" + '\n'.join(rows) + r"""
\end{longtable}
\endgroup

\paragraph{Rerun commands.}
All commands run from \loc{/workspace/XVI-01-CONE}.
The spectral runs were launched as detached jobs with logs because
the machine has about 7\,GB of memory.
\begin{center}\footnotesize
\begin{tabular}{@{}l@{}}
\verb!for g in D01 D02 D03 D04 D05a D06 D11 D12 D13; do for b in CR F; do!\\
\verb!  setsid python3 code/spectral_apex.py $g $b > logs/v4/spec_${g}_${b}.log 2>&1; done; done!\\
\verb!for g in D01 D02 D03 D06 D11 D12 D13; do setsid python3 code/spectral_apex.py $g UNIT \!\\
\verb!  > logs/v4/spec_${g}_UNIT.log 2>&1; done!\\
\verb!python3 code/catalogue_check.py          # -> logs/v6/catalogue_check.json!\\
\verb!python3 code/st_norm.py                  # -> logs/v6/st_norm.json!\\
\verb!for b in CR F UNIT; do python3 code/st_v6.py "logs/v4/spec_{g}_$b.json" $b; done!\\
\verb!for g in D01 D02 D03 D06 D11 D12 D13; do python3 code/flat_pole.py $g; done!\\
\verb!python3 code/st_v6.py "logs/v6/flat_{g}.json" FLAT!\\
\verb!python3 code/st_v6_stats.py; python3 code/flat_vs_p18.py!\\
\verb!python3 code/weighted_apex.py; python3 code/weighted_checks.py!\\
\verb!python3 code/hi_d01.py; python3 code/hi_d01_exact.py!\\
\verb!python3 code/hh_structures.py!\\
\verb!python3 code/build_v6.py && python3 code/artifacts_v6.py!\\
\verb!cd v6build && pdflatex XVI-01-CONE-v6.tex && pdflatex XVI-01-CONE-v6.tex!
\end{tabular}
\end{center}
The spectral runs (first four lines) were executed during the v4 session
and were not repeated for v6; every other command was run for v6.
"""
s = open(R + 'v6build/XVI-01-CONE-v6.pre.tex').read()
assert '%%ARTIFACT_APPENDIX%%' in s
s = s.replace('%%ARTIFACT_APPENDIX%%', app)
open(R + 'v6build/XVI-01-CONE-v6.tex', 'w').write(s)
open(R + 'logs/v6/artifact_manifest.sha256', 'w').write('\n'.join(manifest) + '\n')
open(R + 'logs/frozen_v6_after.sha', 'w').write('\n'.join(f'XVI{t} {trees[t]}' for t in trees) + '\n')
print(trees, before)
```

9 Debug scratch

9.1 dbg.py

SHA-256: 4b64316cba7749472bcafe48d59c840671e3fe449a801ea557764dbd7b52aa19

```
import sys, random
from fractions import Fraction as Fr
sys.path.insert(0,'.')
import apex_jets as AJ
from apex_jets import *
S=Setup(Fr(7071,10000),'CR'); random.seed(5); P=AJ.P
def rq(): return [random.randrange(1, 1000), random.randrange(1, 1000)]
hh=candidates(S,'h','h',4); hc=candidates(S,'h','c',5); kc=candidates(S,'K','c',4)
sm=[(rand_cfg(S,'h',6),rand_cfg(S,'c',6),rq()) for _ in range(80)]
Gb=[[eval_sym(c,a,b,q,False) for (a,b,q) in sm] for c in hc]
r0,_=rank_and_basis(Gb); print('hc basis rank',r0,len(hc))
bad=0
for i,c in enumerate(hh):
    img=[2*eval_sym(c,a,Ops(S,q).K2(b),q,True)%P for (a,b,q) in sm]
    r1,_=rank_and_basis(Gb+[img])
    if r1>r0:
        bad+=1
        if bad<4: print('HH cand not in span:',c[0],c[1],c[4],c[5])
print('HH bad',bad,'of',len(hh))
bad=0
for c in kc:
```

```
    img=[eval_sym(c,Ops(S,negq(q)).E(a),b,q,False) for (a,b,q) in sm]
    r1,_=rank_and_basis(Gb+[img])
    if r1>r0:
        bad+=1
        if bad<4: print('CK cand not in span:',c[0],c[1],c[4],c[5])
print('CK bad',bad,'of',len(kc))
```

9.2 dbg2.py

SHA-256: cfb19972bd07e4121da0b2c1ffd795609a0887935fb20dc5f26544003356b579

```
import sys, random
from fractions import Fraction as Fr
sys.path.insert(0,'.')
import apex_jets as AJ
from apex_jets import *
S=Setup(Fr(7071,10000),'CR'); random.seed(7); P=AJ.P
def rq(): return [random.randrange(1, 1000), random.randrange(1, 1000)]
bc=candidates(S,'b','c',5); cb=candidates(S,'cb','c',4); kc=candidates(S,'K','c',4)
sm=[(rand_cfg(S,'b',6),rand_cfg(S,'c',6),rq()) for _ in range(60)]
Gb=[[eval_sym(c,a,b,q,False) for (a,b,q) in sm] for c in bc]
r0,_=rank_and_basis(Gb); print('bc rank',r0,len(bc))
for nm,cl,op in [('CB',cb,None),('CK',kc,'C2T')]:
    bad=0
    for c in cl:
        img=[eval_sym(c,(a if op is None else Ops(S,negq(q)).C2T(a)),b,q,False) for (a,b,q) in sm]
        r1,_=rank_and_basis(Gb+[img])
        if r1>r0:
            bad+=1
            if bad<3: print(nm,'not in span',c[0],c[1],c[4])
    print(nm,'bad',bad,len(cl))
```